

π Tantum

alias Tempus Tantum

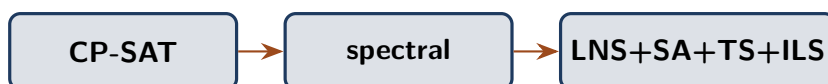
sistema di generazione orario scolastico

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”
Lucio Anneo Seneca, *Epistulae morales ad Lucilium*, I, 1

Manuale

Un metodo per l'orario delle scuole italiane

Da un'idea di Fernando Gargiulo, Giovanni Borghi,
Matteo Mariani, Stefano Bertozzi



Giovanni Borghi

7 maggio 2026

Repository: <https://github.com/gborghi/timetable>

Documentazione sorgente: docs/

Colophon

Questo volume è composto in EB Garamond per il corpo del testo e in Lato per i titoli, con Inconsolata per i blocchi di codice. La compilazione è stata eseguita con `lualatex` (TeX Live), `biber` per la bibliografia e `makeindex` per l'indice analitico. Il sorgente $\LaTeX$ risiede in `docs/manual.tex` con i capitoli in `docs/manual/chapters/`; il bibliografico è in `docs/manual/bibliography.bib`. La pipeline di build è in `docs/build_manual.sh` (Linux/macOS/Git Bash) e `docs/build_manual.bat` (Windows).

Tipografia: pacchetti `ebgaramond`, `lato`, `inconsolata`, `microtype`.

Layout: KOMA-Script `scrbook`, `geometry`, `scrlayer-scrpage`.

Editoriale: `tcolorbox`, `lettrine`, `marginnote`, `epigraph`, `tikz`, `pgfplots`.

Bibliografia e indici: `biblatex` backend `biber`, `imakeidx`.

Versione del repository corrispondente al commit visibile nel log `git log --oneline -- docs/`. Eventuali errori e suggerimenti possono essere segnalati come *issue* sul repository GitHub all'indirizzo <https://github.com/gborghi/timetable>.

Tutti i diritti del progetto sono di Giovanni Borghi. Documento prodotto a maggio 2026.

Indice

Elenco delle figure	9
Elenco delle tabelle	10
Introduzione	11
1 Il problema dell'orario scolastico	12
1.1 Quante combinazioni possibili	12
1.2 Vincoli rigidi e vincoli morbidi	12
1.3 Una breve storia delle idee	13
1.4 Le quattro famiglie di approcci	14
1.5 La filosofia di π Tantum	14
1.6 Le peculiarità del caso italiano	15
2 Una panoramica di πTantum	16
2.1 Cosa fa il sistema	16
2.2 Tre layer in un unico repository	16
2.3 Diagramma a blocchi del sistema	17
2.4 Il flusso d'uso quotidiano	17
2.5 Quando il sistema non basta da solo	18
2.6 Le tecnologie scelte e perché	19
3 L'orario come oggetto strutturale	20
3.1 I docenti	20
3.2 Le classi	20
3.3 Le materie	21
3.4 Le aule	21
3.5 Gli indirizzi di studio	21
3.6 Gli studenti	22
3.7 I gruppi articolati	22
3.8 Le relazioni fra le entità	22
3.9 I vincoli ricorrenti del caso italiano	23
3.10 Cosa rende peculiare il sistema italiano	23
4 Installazione	24
4.1 Windows	24
4.2 Linux (Ubuntu, Debian, Fedora, Arch)	25
4.3 macOS (Intel + Apple Silicon)	25
4.4 Verifica installazione	26
4.5 Aggiornamento	27
4.6 Disinstallazione	27
5 Il modello a vincoli	28
5.1 La tassonomia dei cinque stati	28
5.2 Le matrici di disponibilità	29
5.3 I vincoli logici disgiuntivi	29
5.4 I toggle HARD per classe	31
5.5 Il rilevamento dei conflitti	31
5.6 Tag delle aule	32
5.7 Le preferenze di giorno libero per i docenti	32

5.8	Le preferenze di giorno libero per le classi	32
5.9	Compresenze, sostegno e potenziamento	33
5.10	Religione, alternativa e parallelismi intra-classe	34
5.11	Gruppi inter-classe: seconda lingua, recupero	34
5.12	Lock nativi: blindare lezioni già sistemate	35
5.13	Pre-flight checks: errori prima della run	36
5.14	Plessi: più sedi nello stesso istituto	36
5.15	Pattern canonici e seed legacy via DSL (Step 3b-3e)	38
6	Workflow di ottimizzazione	40
6.1	Schema delle fasi	40
6.2	Phase A: assegnazione (cattedre)	40
6.3	Phase B: scheduling	40
6.4	Metaeuristiche	42
6.5	Phase 4: assegnazione aule	42
6.6	Drag-and-drop con preview live	43
6.7	Slot picker matriciale (Monitor)	43
6.8	Punteggio di graduatoria e criterio Anzianita	44
7	Guida UI	45
7.1	Funzionalità trasversali	45
7.2	Tab per tab	48
8	Launcher	49
8.1	Windows: start.bat	49
8.2	Linux / macOS: start.sh + stop.sh	49
9	CP-SAT, il motore che cerca, deduce, ritratta	50
9.1	Il problema concreto	50
9.2	Una storia in cinque tappe	51
9.3	Gli elementi della modellazione	52
9.4	Un esempio in miniatura	54
9.5	Generalizzazione alla scuola reale	55
9.6	Quando CP-SAT brilla, quando incontra i propri limiti	55
9.7	Le connessioni con il resto del sistema	56
10	La decomposizione spettrale, spezzare per regnare	57
10.1	Da dove viene l'idea	57
10.2	Il grafo bipartito classe-docente	57
10.3	Il Laplaciano e il vettore di Fiedler	58
10.4	Un esempio in miniatura	58
10.5	La generalizzazione al caso reale	59
10.6	Quando il metodo funziona, quando no	59
10.7	Le connessioni con il resto del sistema	60
10.8	Tre alternative alla decomposizione spettrale	60
11	Le metaeuristiche, scolpire una soluzione	62
11.1	La radice comune: la ricerca locale	62
11.2	Simulated Annealing: l'analogia con la metallurgia	62
11.3	Tabu Search: il taccuino delle mosse vietate	63
11.4	Iterated Local Search: perturbare e ricominciare	63
11.5	Variable Neighborhood Search: cambiare obiettivo	63
11.6	Large Neighborhood Search: distruggere e ricostruire	64
11.7	Adaptive LNS: scolpire con scalpelli adattivi	64
11.8	Quale scegliere	65
11.9	Un esempio concreto su otto lezioni	65
11.10	Limiti e parametri da tarare	65
11.11	Le connessioni con il resto del sistema	65

12 Il pre-controllo di Hall	67
12.1 Il teorema dei matrimoni	67
12.2 La traduzione operativa	67
12.3 Un esempio in miniatura	68
12.4 L'integrazione nell'interfaccia	68
12.5 Limiti del pre-controllo	69
12.6 Approfondimento: l'algoritmo di Hopcroft-Karp	69
12.7 Le connessioni con il resto del sistema	69
13 Lagrangian Relaxation e Column Generation	70
13.1 Rilassamento lagrangiano: dualizzare i ponti	70
13.2 Column Generation: il catalogo immobiliare	72
13.3 Connessioni	74
14 Monte Carlo: simulare per capire	75
14.1 La storia del nome	75
14.2 Sensitivity analysis: cosa misurare	75
14.3 Schema operativo	76
14.4 Quando è utile	76
14.5 Quando non basta	76
14.6 Connessioni	77
15 Analisi del grafo bipartito: vedere la struttura	78
15.1 Modularità: quanto è "a cluster" la scuola	78
15.2 Betweenness centrality: i "ponti" del sistema	79
15.3 Densità: quanto è "pieno" il grafo	79
15.4 Visualizzazione e interpretazione	79
15.5 Connessioni	80
16 Statistica applicata: leggere i risultati	81
16.1 Distribuzioni: vedere la forma dei dati	81
16.2 Correlazioni e regressioni	82
16.3 Chi-quadro: indipendenza	83
16.4 Visualizzazione interattiva	83
16.5 Connessioni	83
17 Il DSL generico: un linguaggio per i vincoli	84
17.1 Filosofia: "un parser, molti compilatori"	84
17.2 Grammatica (sintesi)	84
17.3 Parser e AST	85
17.4 Tre back-end	85
17.5 Estensione: aggiungere un built-in	86
17.6 Performance	86
17.7 Roadmap	86
17.8 Connessioni	87
18 Architettura	88
18.1 Stack	88
18.2 Diagramma a blocchi	88
18.3 Struttura cartelle	88
18.4 Lifecycle del backend	89
18.5 Architettura del solver: dal DSL a CP-SAT	89
19 Modello dati	91
19.1 Famiglie di tabelle	91
19.2 Diagramma relazioni	92
19.3 Migrazioni idempotenti	92
19.4 Pickle dell'engine	92

20 DSL generico per i vincoli	93
20.1 Filosofia	93
20.2 Grammatica BNF	93
20.3 Sorgenti iterabili	94
20.4 Galleria di esempi	94
20.5 Scope: dove salvare il vincolo	98
20.6 Predicati built-in: consecutive_days, same_day, consecutive	98
20.7 Aritmetica nel DSL	99
20.8 Casi d'uso reali (Rossi e dintorni)	99
20.9 Errori comuni	100
20.10 Compilazione DSL $\rightarrow$ CP-SAT (Step 1)	101
20.11 Predicati di slot: consecutive, same_day	101
20.12 Path l.classroom.plesso	102
20.13 Vocabolario di pattern canonici (Step 3b)	102
20.14 Plesso commuting via DSL (Step 3c)	102
20.15 Seed legacy HARD via DSL (Step 3d-3e)	103
20.16 Vincoli SOFT con peso	103
20.17 Le quattro famiglie di compilazione	104
20.18 Logica DNF vs forall/count: due famiglie a confronto	104
20.19 Quattordici pragma canonici per la Phase A e la Phase B	105
20.20 Workflow: dal tab Docente al solver	106
20.21 Tre casi narrati	107
20.22 Diagramma dell'architettura DSL	108
21 Tecniche di ottimizzazione avanzate	110
21.1 Adaptive Large Neighborhood Search (ALNS)	110
21.2 Variable Neighborhood Search (VNS)	110
21.3 Hall's theorem pre-check	110
21.4 Column Generation e Branch-and-Price avanzato	110
21.5 Lagrangian Relaxation	111
21.6 Branch-and-Price MVP scaffold (Step 4)	111
22 Diagnostica statistica	112
22.1 Sensitivity Monte Carlo	112
22.2 Analisi bipartito	112
22.3 Correlazioni e regressioni	112
22.4 Distribuzioni	112
22.5 Run telemetry	112
23 API REST	113
23.1 Pattern CRUD	113
23.2 Esempi curl	113
23.3 Riferimento gruppi di endpoint	114
24 Estendere il sistema	115
24.1 Aggiungere una nuova tabella + CRUD	115
24.2 Migrazione idempotente	115
24.3 Aggiungere un nuovo tipo di vincolo	115
24.4 Aggiungere un nuovo step di ottimizzazione	116
24.5 Testing	116
A Repository GitHub	117
B Benchmark e risultati	118
B.1 I cinque profili	118
B.2 Pipeline end-to-end: tempi totali	118
B.3 Componenti dell'obiettivo SOFT	119
B.4 Hall pre-check: tempo trascurabile	119

B.5	Decomposizione spettrale: speedup misurato	120
B.6	Metaeuristiche: convergenza dell'obiettivo	120
B.7	Lettura dei risultati	120
B.8	Confronto cross-method	121
B.9	Branch-and-Price su HUGE e MEGA	121
B.10	Quando il Branch-and-Price conviene	123
C	Lessons learned	125
C.1	Cosa ha funzionato	125
C.2	Cosa non ha funzionato (al primo tentativo)	126
C.3	Quello che ancora si può migliorare	127
C.4	Una riflessione finale	127
D	Glossario discorsivo dei metodi	128
D.1	CP-SAT (Constraint Programming over SAT)	128
D.2	Decomposizione spettrale	128
D.3	LNS (Large Neighborhood Search)	129
D.4	ALNS (Adaptive LNS)	129
D.5	SA (Simulated Annealing)	130
D.6	TS (Tabu Search)	130
D.7	ILS (Iterated Local Search)	130
D.8	VNS (Variable Neighborhood Search)	131
D.9	Hall's theorem pre-check	131
D.10	Lagrangian Relaxation con sub-gradient	131
D.11	Column Generation (Dantzig-Wolfe)	132
D.12	Monte Carlo Sensitivity	132
D.13	Modularità, betweenness, densità (analisi del bipartito)	132
D.14	Distribuzioni e istogrammi	133
D.15	Correlazioni e regressioni	133
D.16	DSL generico	134
D.17	Stati 5-colori delle matrici	134
D.18	Tag aule e tag studenti	134
D.19	Run telemetry e visualizzazione live	135
D.20	Lockati: lezioni che non si toccano	135
E	Reference	136
	Bibliografia	137
	Indice analitico	139

Elenco delle figure

2.1	Architettura a blocchi di π Tantum: il frontend parla al backend tramite REST, il backend persiste su SQLite e dialoga con il solver attraverso un convertitore esplicito basato su pickle, il solver invoca OR-Tools per le risoluzioni CP-SAT.	17
6.1	Pipeline a 4 fasi.	40
9.1	Il ciclo di funzionamento di CP-SAT. Dopo aver propagato il modello iniziale, il solver alterna decisioni e propagazioni; ogni conflitto produce una clausola appresa che evita di ripetere lo stesso errore.	53
10.1	Esempio di grafo bipartito con due cluster naturali e un “ponte” debole. Il vettore di Fiedler assegna componenti di segno opposto ai due cluster.	58
18.1	Architettura a blocchi della webapp.	88
19.1	Relazioni rilevanti del modello dati (semplificato).	92
20.1	I 14 pragma canonici, raggruppati per layer della pipeline in cui sono compilati. I pragma di Phase A sono i mattoni di DayCountModel; quelli di Phase B alimentano PhaseB-DaySolver; le ultime due forme (class_subject_same_day e teacher_class_consecutive_days) sono espresse direttamente dalla famiglia general_dsl senza un canonical helper ad hoc.	105
20.2	Pipeline del DSL: dal modal nell’UI fino alle quattro famiglie di backend. Il pulsante <i>validate</i> attiva solo il parser (e ritorna gli errori in tempo reale); il <i>save</i> attiva il loop completo: persistenza in constraints_general, ricarica al successivo POST /api/optimize, valutazione post-hoc al termine.	106
20.3	Architettura del DSL generico: stessa grammatica e stesso AST, quattro back-end di traduzione. La differenza fra “valutare una soluzione” (evaluator post-hoc) e “costruire un modello” (compiler verso CP-SAT) si traduce in due attraversamenti diversi dello stesso albero.	109
B.1	Tempo totale della pipeline (scala logaritmica). Si nota la salita lineare in scala log fino a huge, con il salto previsto su superhuge dovuto alla dimensione del Phase A.	119
B.2	Confronto del tempo di Phase B con e senza decomposizione spettrale, sui profili medio-grandi.	120
B.3	Convergenza di ALNS sul profilo huge: il miglioramento si concentra nei primi 1000–2000 step e poi satura, tipico delle metaeuristiche.	121
B.4	Tempo di convergenza del Branch-and-Price (scala logaritmica). Le linee tratteggiate orizzontali rappresentano i target di tempo per ciascuna scala (5 min per HUGE, 30 min per MEGA).	122
B.5	Dimensione finale del pool di colonne dopo il loop Branch-and-Price.	122

Elenco delle tabelle

5.1	I cinque stati della tassonomia di π Tantum, con colori, semantica e contributo alla funzione obiettivo.	28
20.1	Quattro famiglie di compilazione, una sola grammatica. Storicamente erano quattro sub-DSL separati. Dalla vo.4 il parser è unico (<code>general_dsl.py</code>) e produce un AST condiviso da cui ciascuna famiglia estrae ciò che le serve.	104
20.2	La DNF logica è un <i>subset</i> della grammatica generale. Migrare un vincolo dalla forma DNF alla forma generale è sempre possibile (<code>helper logical_unavailability_to_dsl</code>); migrare nell'altro verso richiede un' <i>abstraction</i> del vincolo che spesso non esiste.	104
B.1	Tempi della pipeline end-to-end per profilo. Phase A include assegnazione docenti–classi e CP-SAT su matching, Phase B include decomposizione spettrale (per huge/superhuge) e CP-SAT su sotto-problemi.	118
B.2	Componenti dell'obiettivo SOFT di Phase A. <code>glib_pen=0</code> significa che ogni docente ha ricevuto il giorno libero di prima preferenza (tranne in medium, dove 1–3 docenti hanno preso la seconda preferenza per overlap di vincoli).	119
B.3	Tempo del pre-check di Hall in millisecondi. Trascurabile su tutti i profili.	119
B.4	Speedup misurato della decomposizione spettrale su Phase B. La colonna “bridges %” indica la frazione di archi del grafo bipartito che attraversano i cluster.	120
B.5	Tempi totali con diverse combinazioni di metodi. Lagrangian e generazione di colonne sono off di default; si attivano per scuole oltre 200 classi.	121
B.6	Tempi del Branch-and-Price su scenari sintetici HUGE/MEGA. Entrambi finiscono molto sotto target (5 min/30 min) perché lo scenario sintetico è strutturalmente facile (cattedra = blocco di 4 ore in un solo giorno). Su istanze reali con distribuzione giornaliera meno regolare il tempo per iterazione cresce; il target 30 min per MEGA è calibrato su quel regime.	121
B.7	Branch-and-Price su MEGA reale prima e dopo il fix DW seeding. Il soft cost crolla di un fattore ~ 12 perché il BP ora può effettivamente esplorare colonne migliorative. Il totale aumenta perché Phase A in v2 ha esaurito il budget (stop FEASIBLE non OPTIMAL: la differenza dipende dal seed parallelo del CP-SAT) ma rimane sotto il target di 30 min e ben lontano dall'hard cap di 60 min. Sorgente: <code>tests/benchmarks/results/mega_bp_real_v2.json</code>	123
B.8	Confronto delle tre run sullo stesso MEGA reale. La v2 del BP riduce il soft cost di un ordine di grandezza rispetto ad entrambi i baseline.	123

Introduzione

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”

Lucio Anneo Seneca, *Epistulae morales ad Lucilium*, I, 1

π Tantum è un'applicazione web che assiste nella composizione dell'orario delle scuole italiane. Nasce da idee discusse e sviluppate da Fernando Gargiulo, Giovanni Borghi, Matteo Mariani e Stefano Bertozzi.

Il problema dell'orario scolastico consiste nell'assegnare a ogni cattedra, vale a dire alla coppia formata da un docente e da una classe per una determinata materia, un insieme di ore settimanali distribuite nei giorni e negli slot orari, in modo che siano rispettati i vincoli di disponibilità dei docenti, di copertura delle ore curricolari, di compatibilità delle aule e delle attrezzature, e i vincoli derivanti dal contratto collettivo nazionale del settore scolastico. Si tratta di un problema combinatorio classificato come NP-difficile, il che significa che all'aumentare delle dimensioni della scuola non esiste un algoritmo veloce in grado di trovare una soluzione ottimale in modo esatto.

π Tantum si appoggia, per la risoluzione del problema, sul solver CP-SAT della libreria Google OR-Tools, scelto per la sua maturità nel constraint programming applicato a problemi di scheduling. Per scalare oltre le dimensioni che il solver gestirebbe direttamente, il sistema decompone il problema in sotto-problemi più piccoli, attraverso quattro strategie alternative: la decomposizione spettrale del grafo bipartito classe-docente, la decomposizione temporale per giorno della settimana, la decomposizione per curriculum, e il partizionamento METIS k -bilanciato. A queste si affianca una cascata di metaeuristiche — Large Neighborhood Search, Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search e Variable Neighborhood Search — che migliora la qualità della soluzione trovata.

Il backend dell'applicazione è scritto in Python con il framework FastAPI; la persistenza dei dati è affidata a SQLite. L'interfaccia utente è basata su SvelteKit, in italiano, organizzata in sedici tab che coprono l'intero ciclo di vita di un orario: importazione dell'anagrafica, configurazione dei vincoli, generazione, editing manuale con controllo di fattibilità in tempo reale, gestione delle assenze e delle supplenze quotidiane.

I capitoli seguenti descrivono il problema in modo formale, le entità coinvolte, l'interfaccia utente, e ciascuno dei metodi algoritmici impiegati.

1 Il problema dell'orario scolastico

“Educational timetabling is one of the most studied problems in operations research, yet remains one of the hardest in practice.”

Andrea Schaerf, *A survey of automated timetabling* (Schaerf 1999)

Il problema consiste nell'assegnare a ogni lezione settimanale uno slot di tempo e un'aula, in modo che nessun docente, nessuna classe e nessuna aula si trovino impegnati in due luoghi nello stesso istante, e che il monte ore di ogni materia in ogni classe sia rispettato. Una scuola di dimensioni medie produce un'esplosione combinatoria di possibilità che rende impraticabile l'enumerazione esaustiva, e le richieste personali dei docenti, le incompatibilità delle aule, i vincoli sindacali e le delibere collegiali si sommano gli uni agli altri.

1.1 Quante combinazioni possibili

Si consideri una scuola italiana di dimensione media: trenta classi, sessanta docenti, sei giorni di lezione, sei ore al giorno. Ogni classe ha tipicamente una trentina di ore settimanali, e l'insieme delle lezioni da collocare è dunque dell'ordine di novecento. Per ognuna di queste lezioni è necessario decidere quale materia mettere in quale slot e quale docente farla insegnare. Anche limitando le scelte ai soli docenti compatibili (cioè quelli abilitati alla materia in questione e con sufficienti ore residue), per ogni lezione restano in media una decina di possibilità. Lo spazio delle soluzioni cresce dunque come dieci elevato a novecento, un numero che si scrive con un uno seguito da novecento zeri e che non ha alcun significato fisico immaginabile. Per dare un termine di paragone, si stima che il numero di atomi nell'universo osservabile sia attorno a dieci elevato alla ottantesima potenza: lo spazio delle assegnazioni di una scuola media è quindi 10^{820} volte più grande dell'universo intero. Esplorarlo per enumerazione cieca non è soltanto impraticabile sui computer di oggi: è fisicamente impossibile.

Questo è il motivo per cui non basta “provare le combinazioni”, anche con un computer veloce. Servono algoritmi che, anziché esaminare ogni possibilità, scartino sotto-spazi enormi senza neppure visitarli, sulla base di deduzioni logiche elementari. È quello che fa la *propagazione di vincoli*, una tecnica che vedremo nel dettaglio nel capitolo dedicato a CP-SAT (cap. 9). In alternativa, si possono accettare soluzioni *non garantite ottime*, ottenute attraverso processi di ricerca locale che migliorano una soluzione iniziale per piccoli passi: è l'approccio delle metaeuristiche, di cui parlerà il cap. 11. Si può anche spezzare il problema in sotto-problemi più piccoli e risolverli quasi indipendentemente, ricucendoli alla fine: è il principio della decomposizione, che π Tantum applica in diverse forme (cap. 10 per la decomposizione spettrale, cap. 13 per quella lagrangiana). Tutte queste tecniche convivono e collaborano nella pipeline del sistema, che combina ciò che ognuna fa meglio.

1.2 Vincoli rigidi e vincoli morbidi

Una difficoltà almeno altrettanto grave della dimensione combinatoria deriva dalla *natura* dei vincoli che intervengono. Non sono tutti uguali: alcuni sono assoluti, non negoziabili, e una soluzione che li violi non è neppure una soluzione, mentre altri esprimono preferenze, desideri o abitudini di servizio, e si vorrebbero rispettare ma su di essi si può transigere se le circostanze lo richiedono. Ignorare questa distinzione e trattare tutto come ugualmente inderogabile produce, come si può immaginare, sistemi infattibili al primo conflitto fra preferenze, e l'utente si ritrova davanti a un solver che dichiara “nessuna soluzione esistente” senza la minima indicazione di cosa fare.

In π Tantum si è adottata, fin dall'inizio, una distinzione canonica della letteratura (Carter e Laporte 1998; Schaefer 1999) fra vincoli rigidi e morbidi, raffinata in cinque livelli distinti rappresentati nell'interfaccia da una piccola palette di colori. Il livello *HARD* (rosso) rappresenta i vincoli inderogabili: i due esempi più immediati sono “due classi non possono trovarsi nella stessa aula nello stesso slot” e “un docente non può tenere due lezioni contemporaneamente”. Il livello *SOFT* (giallo) esprime una preferenza negativa: si vorrebbe evitare quella configurazione (per esempio una sesta ora di matematica al sabato pomeriggio) ma se serve per chiudere l'orario la si può accettare, pagando un costo che entra nella funzione obiettivo. Il livello *PREFERRED* (blu) è simmetrico rispetto al precedente: indica una preferenza positiva, una configurazione che si vorrebbe vedere ricorrere (per esempio il laboratorio di fisica in due ore consecutive) e che riduce il costo se viene rispettata. Il livello *ENFORCED* (verde scuro) è ancora più rigido di *HARD* ma in senso opposto: si dichiara che quel particolare slot *deve* essere usato, non è ammessa l'alternativa. Infine il livello *ALLOWED* (verde chiaro) rappresenta la situazione neutra di default: nessuna preferenza in nessun senso.

Questa stratificazione, che a prima vista può apparire sovrabbondante, si rivela quasi sempre necessaria nella pratica. Una scuola italiana ha decine di vincoli che appartengono naturalmente al livello *SOFT* (le preferenze mattutine dei docenti, la concentrazione delle materie teoriche nelle prime ore, l'evitamento dei buchi nelle classi), accanto a quelli inderogabili. Fingere che siano tutti *HARD* vorrebbe dire produrre sistemi cronicamente infattibili; fingere che siano tutti *SOFT* vorrebbe dire produrre sistemi che violano regole contrattuali. La distinzione precisa è ciò che permette di ottenere soluzioni che funzionano davvero.

1.3 Una breve storia delle idee

Il problema del timetabling è oggetto di studio sistematico fin dagli anni sessanta del Novecento, quando le scuole più grandi cominciarono a cercare metodi sistematici per evitare i lunghi pomeriggi di settembre passati a comporre l'orario con scotch e cartoncini ritagliati. I primi tentativi furono basati su algoritmi di colorazione di grafi, perché una versione semplificata del problema (assegnare slot in modo che lezioni in conflitto ricevano slot diversi) si lascia ricondurre con eleganza al colorare i nodi di un grafo in modo che nodi adiacenti abbiano colori distinti. Il problema della colorazione dei grafi è stato uno dei primi classificati come *NP-completo* da Karp nel 1972, e per riduzione anche il timetabling fu identificato come problema intrinsecamente difficile. La sigla NP-completo, per chi non è abituato al gergo della complessità computazionale, vuole dire che non si conosce alcun algoritmo capace di risolvere *tutte* le istanze del problema in tempo crescente polinomialmente con la loro dimensione, e che questa difficoltà è condivisa con un'intera famiglia di problemi ritenuti intrattabili in generale (Garey e Johnson 1979).

Fortunatamente le istanze *reali* di una scuola italiana non sono i casi peggiori previsti dalla teoria: la struttura dei vincoli è molto regolare (gli indirizzi sono pochi, le classi di concorso sono ancora più poche, le aule specializzate sono distribuite in modo prevedibile), e questa regolarità si lascia sfruttare dagli algoritmi moderni in modo molto efficace. Negli anni ottanta e novanta, alle tecniche di backtracking si aggiunsero le *metaeuristiche*: il *simulated annealing* di Kirkpatrick, Gelatt e Vecchi del 1983 (Kirkpatrick et al. 1983), ispirato alla metallurgia, e il *tabu search* di Glover del 1986 (Glover 1986), che introdusse l'idea di una memoria delle mosse recenti per guidare la ricerca. Negli anni duemila i *constraint solver* acquisirono maturità industriale: prodotti come GeCode, ECLiPSe e poi CP-SAT di Google permisero di affrontare istanze prima inarrivabili. Pisinger e Ropke nel 2006 introdussero la *Adaptive Large Neighborhood Search* (Ropke e Pisinger 2006), che combina distruzioni e ricostruzioni intelligenti su porzioni significative della soluzione. La conferenza biennale PATAT (*Practice and Theory of Automated Timetabling*), attiva dal 1995, divenne il riferimento mondiale per i ricercatori del settore (PATAT 1995–2024).

π Tantum si colloca in questa tradizione. Non inventa metodi nuovi: combina con cura quelli esistenti in una pipeline orchestrata che cerca di sfruttare i punti di forza di ciascuno. CP-SAT viene usato come motore principale per la fase di soddisfacimento dei vincoli rigidi, sia nella fase di matching docente–classe sia nella collocazione settimanale. La decomposizione spettrale, ispirata alle intuizioni di Fiedler del 1973 (Fiedler 1973), viene usata per spezzare le scuole più grandi in cluster naturali, in modo che CP-SAT possa lavorare su sotto-problemi trattabili. Le metaeuristiche raffinano la soluzione ottenuta, riducendo le violazioni dei vincoli morbidi. Il pre-controllo di Hall, basato su un teorema del 1935 che sembra antico ma che resta sorprendentemente attuale, verifica in pochi millisecondi se il problema è almeno formalmente fattibile, evitando di lanciare il solver su istanze infeasibili dall'inizio.

1.4 Le quattro famiglie di approcci

La letteratura sul timetabling distingue tradizionalmente quattro famiglie di approcci, ciascuna con i propri vantaggi e i propri limiti. La prima è la *programmazione a vincoli*, che modella il problema come insieme di variabili con domini finiti e di vincoli che le legano, lasciando poi che un solver come CP-SAT si occupi di trovare un'assegnazione che li soddisfi tutti. Il vantaggio di questo approccio è la sua estrema flessibilità: aggiungere un vincolo nuovo significa aggiungere una riga di codice e nessuna riprogettazione strutturale del modello. Lo svantaggio è che la scalabilità diventa critica oltre la soglia di una settantina di classi: il numero di variabili booleane equivalenti che il solver deve gestire cresce e i tempi di risoluzione esplodono.

La seconda famiglia è la *programmazione lineare intera* (MILP, *Mixed Integer Linear Programming*), che formula il problema come sistema di disuguaglianze lineari con variabili binarie e lo risolve con prodotti commerciali come Gurobi o CPLEX, oppure con alternative open-source come HiGHS. Il vantaggio teorico è che si possono ottenere certificati di ottimalità, almeno su istanze piccole. Lo svantaggio pratico è che la formulazione MILP del timetabling tende a richiedere milioni di variabili binarie e diventa pesante. π Tantum non utilizza MILP nella pipeline principale ma prevede uno scheletro di generazione di colonne (*column generation*) che si appoggia a un master LP risolto con HiGHS, applicabile alle istanze più grandi (cap. 13).

La terza famiglia è quella delle *metaeuristiche*. Si parte da una soluzione iniziale, ottenuta con un metodo qualunque (anche un greedy molto rudimentale), e la si migliora iterativamente attraverso operatori di ricerca locale: scambi di lezioni, perturbazioni casuali, accettazioni parziali di mosse peggioranti. Il vantaggio è la scalabilità: le metaeuristiche restituiscono *sempre* una soluzione, anche su istanze enormi, e in tempi controllabili. Lo svantaggio è che non si ha alcuna garanzia di ottimalità: la soluzione finale è di solito buona, ma raramente la migliore possibile.

La quarta famiglia è quella delle *decomposizioni*. Invece di affrontare il problema monolitico, lo si spezza in sotto-problemi più piccoli, lo si risolve indipendentemente, e lo si ricuce alla fine. La decomposizione può essere *spettrale*, basata sulla struttura algebrica del grafo bipartito che lega classi e docenti; *lagrangiana*, basata sul rilassamento dei vincoli che legano i sotto-problemi e sulla loro penalizzazione progressiva nell'obiettivo; *di Dantzig-Wolfe*, basata sulla generazione iterativa di nuove variabili nel master problem. π Tantum implementa tutte e tre, con preferenze diverse a seconda della dimensione dell'istanza.

1.5 La filosofia di π Tantum

La scelta progettuale fondamentale di π Tantum è di *non scegliere* fra le quattro famiglie sopra descritte, ma di combinarle in una pipeline a quattro fasi orchestrate. Ogni fase utilizza il metodo più appropriato al suo compito e produce un risultato che la fase successiva può usare come punto di partenza. Si tratta di un approccio non particolarmente originale dal punto di vista della ricerca (è ciò che molti sistemi commerciali e accademici fanno), ma di un approccio efficace, e che ha il pregio di rendere il sistema modulare: si può sostituire una fase con un'implementazione alternativa senza dover toccare le altre.

La prima fase, denominata *Phase A* nella terminologia del progetto, si occupa dell'*assegnamento*: per ogni classe e per ogni materia, decidere quale docente la insegnerà e quante ore. È un problema di matching nel grafo bipartito classi-docenti, vincolato dalle compatibilità fra classi di concorso e materie, dai monte ore disponibili per ciascun docente, e dalle indisponibilità per giorno. Si risolve con un algoritmo greedy seguito da un passo CP-SAT che ottimizza la distribuzione. Prima di lanciare la fase, π Tantum esegue il *pre-controllo di Hall*, che verifica in pochi millisecondi se il problema è almeno formalmente fattibile dal punto di vista dei matching: se non lo è, il sistema lo segnala all'utente con l'indicazione del sottoinsieme problematico, evitando un'attesa inutile su CP-SAT.

La seconda fase, *Phase B*, si occupa del *timetabling vero e proprio*: collocare ogni lezione in uno slot della settimana. Per le scuole grandi, prima di lanciare CP-SAT si applica la *decomposizione spettrale*, che spezza il problema in cluster relativamente indipendenti. CP-SAT risolve allora ogni cluster in poche decine di secondi, e un passo finale di ricicatura armonizza le interfacce fra cluster.

La terza fase, *Phase C*, si occupa del *raffinamento*: ALNS, simulated annealing, tabu search e iterated local search lavorano sulla soluzione di Phase B per ridurre ulteriormente il valore della funzione obiettivo SOFT.

Si tratta di una fase opzionale ma quasi sempre attivata, che riduce il costo SOFT di un dieci-quindici per cento in pochi minuti aggiuntivi.

La quarta fase, *Phase D*, si occupa dell'*assegnazione delle aule*: dato che ogni lezione ha già uno slot e una classe, resta da decidere in quale aula si terrà, rispettando i tipi (lab, palestra, aula normale), le capienze, le preferenze classe-aula e le compatibilità materia-aula. Si tratta di un problema CP-SAT relativamente piccolo, che si risolve in pochi secondi.

L'intera pipeline, dal momento in cui l'utente preme il bottone "Pipeline completa" al momento in cui il sistema mostra l'orario finito, dura tipicamente un minuto su una scuola piccola, due minuti su una media, dieci minuti su una grande, e fino a un quarto d'ora sulle più grandi testate. Sono numeri che andranno verificati nel cap. B, dove riporto i risultati sperimentali ottenuti su una macchina consumer con i cinque profili di test.

1.6 Le peculiarità del caso italiano

Vale la pena chiudere questo capitolo con un'osservazione che tornerà spesso nelle pagine successive: i benchmark internazionali del timetabling, codificati in formati standard come ITC-2007, ITC-2011 e XHSTT (McCollum et al. 2010), sono importanti per il confronto fra ricercatori, ma non catturano alcune peculiarità del sistema scolastico italiano che invece π Tantum deve modellare con precisione. La prima peculiarità sono gli *indirizzi di studio* (Liceo Classico, Liceo Scientifico, Linguistico, Istituto Tecnico nelle sue varie articolazioni, Istituti Professionali), che hanno monte ore diversi anche per la stessa materia. Una classe "3A scientifico" fa matematica con un orario diverso da una "3A classico" o da una "3A ITIS Informatica", nonostante che il nome della materia sia lo stesso. La seconda peculiarità sono le *classi di concorso*, codificate con sigle alfanumeriche (A027, A019, eccetera), che limitano quali docenti possono insegnare quali materie e in quali indirizzi. Un docente di A027 può insegnare matematica e fisica al biennio scientifico ma non lettere al classico, e queste compatibilità sono codificate in un grande database ministeriale che π Tantum sintetizza nelle sue tabelle.

La terza peculiarità è la presenza diffusa di *compresenze*: in molti indirizzi tecnici, le ore di laboratorio richiedono che due docenti diversi siano nella stessa classe nello stesso slot (l'insegnante teorico e l'insegnante tecnico-pratico). La compresenza è un vincolo HARD non banale, perché non si limita a chiedere "due docenti" ma chiede "proprio quei due docenti, contemporaneamente". La quarta peculiarità sono i *gruppi articolati*: un sottoinsieme di studenti, eventualmente proveniente da più classi di base, che si riuniscono in slot dedicati per attività specifiche (la seconda lingua straniera del linguistico, la lezione di religione cattolica con l'alternativa per chi non si avvale, i corsi di recupero in itinere). La quinta peculiarità sono i *vincoli sindacali*: il giorno libero contrattuale CCNL, il monte ore massimo settimanale, la priorità basata sul punteggio di graduatoria. Sono vincoli HARD non negoziabili, ai quali si aggiungono i vincoli logici espressi via DSL dall'utente.

Tutte queste peculiarità sono codificate come entità o vincoli di prima classe nel modello dati di π Tantum (cap. 19).

2 Una panoramica di π Tantum

2.1 Cosa fa il sistema

π Tantum è un sistema software che, ricevuti in input l'anagrafica di una scuola (docenti, classi, materie, aule, indirizzi di studio, gruppi articolati, studenti), il monte ore richiesto per ciascun indirizzo e per ciascuna materia, le preferenze e le indisponibilità, produce in output un orario settimanale completo che soddisfa tutti i vincoli rigidi e minimizza la somma pesata delle violazioni dei vincoli morbidi. Una volta prodotto l'orario, lo stesso sistema lo gestisce in corso d'anno: consente modifiche manuali con controllo di fattibilità in tempo reale, supporta la gestione delle assenze e delle supplenze quotidiane, e offre viste alternative per docente, per classe, per aula, per slot. Tutte le operazioni avvengono attraverso un'interfaccia web a sedici tab, e tutti i dati sono persistiti in un singolo file SQLite che può essere esportato, importato, e versionato come qualunque altro file.

2.2 Tre layer in un unico repository

Il sistema è organizzato in tre strati distinti, ciascuno con il proprio dominio di responsabilità, e tutti contenuti in un unico repository Git per semplicità di sviluppo e di distribuzione. Questa scelta architetturale è meditata: se i tre layer fossero in repository separati, il loro co-versionamento richiederebbe attenzioni che, per un progetto di questa scala, non aggiungono valore. Mantenere tutto in un solo repository semplifica anche la pipeline di integrazione continua e l'esperienza dello sviluppatore, che può navigare l'intero codice da un'unica radice.

Il primo layer, denominato *solver layer*, risiede nella cartella `experiments/` e contiene tutto il codice algoritmico in Python puro. È uno strato deliberatamente ignaro del database e dell'interfaccia utente: legge un dizionario di input e produce un dizionario di output, secondo un'interfaccia ben definita. Le sue componenti principali sono il modulo `cpsat_v2_assignment.py` per la fase di assegnamento, il modulo `cpsat_v2_timetable.py` per la fase di timetabling vero e proprio, il modulo `decomposition_spectral_v2.py` per il clustering spettrale, il modulo `metaheuristics.py` che raggruppa le quattro varianti classiche (LNS, SA, TS, ILS), e i moduli `alns.py`, `vns.py`, `hall_check.py`, `lagrangian.py`, `column_generation.py`, `classroom_assignment.py` che implementano le tecniche aggiuntive descritte nei capitoli successivi.

Il secondo layer, denominato *web UI layer*, risiede nella cartella `webui/` e contiene il backend FastAPI e il frontend SvelteKit. Il backend espone tutte le operazioni del solver come API REST e gestisce la persistenza in SQLite; il frontend fornisce l'interfaccia web a sedici tab che copre l'intero ciclo di vita di un orario, dall'inserimento dei dati anagrafici alla generazione finale, all'editing manuale, alla gestione operativa delle assenze e delle supplenze. La comunicazione fra frontend e backend avviene via HTTP/JSON; lo storage è un singolo file `webui/data/timetable.db`; le migrazioni del database sono idempotenti ed eseguite automaticamente a ogni avvio del backend.

Il terzo layer, denominato *legacy layer*, risiede nella cartella `schedule/` e contiene gli script monolitici che hanno fatto da prototipo per il solver attuale. Sono mantenuti per riferimento storico e per la riproducibilità dei benchmark contro le versioni precedenti, ma non fanno più parte del flusso operativo. Un nuovo utente del sistema può ignorarli completamente.

2.3 Diagramma a blocchi del sistema

Il diagramma in figura 2.1 riassume in forma grafica le relazioni fra i tre layer. Il frontend SvelteKit, in cima, parla al backend FastAPI attraverso chiamate REST sotto il path `/api/`, instradate dal proxy Vite in modalità di sviluppo o dal server di produzione in modalità di rilascio. Il backend, a sua volta, persiste i dati su SQLite via SQLAlchemy 2.0 e invoca il solver layer attraverso un convertitore esplicito (`engine_io.py`) che traduce le entità relazionali del database in dizionari Python adatti al solver, e viceversa. Il solver, infine, usa la libreria OR-Tools di Google per le risoluzioni CP-SAT.

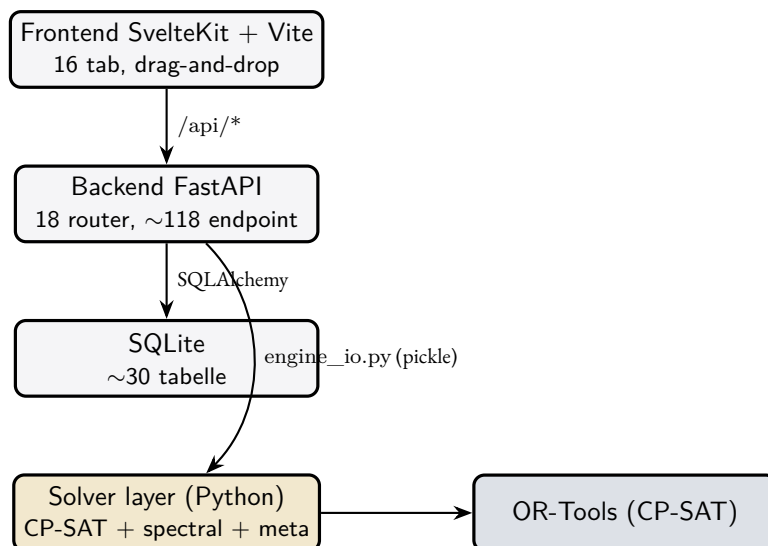


Figura 2.1 – Architettura a blocchi di π Tantum: il frontend parla al backend tramite REST, il backend persiste su SQLite e dialoga con il solver attraverso un convertitore esplicito basato su pickle, il solver invoca OR-Tools per le risoluzioni CP-SAT.

Si noti che non vi è alcuna connessione diretta fra frontend e solver: ogni richiesta dell'utente attraversa sempre il backend, che la valida, la persiste, ed eventualmente la inoltra al solver. Questa scelta garantisce che lo stato del sistema sia sempre coerente e che ogni operazione lasci una traccia nel database, utile sia per l'audit sia per la possibilità di rieseguire le pipeline su dati storici.

2.4 Il flusso d'uso quotidiano

Un coordinatore d'orario alla prima esecuzione del sistema segue tipicamente il flusso che ora descriviamo. Si tratta di una sequenza ricorrente, che si ripete grosso modo identica all'inizio di ogni anno scolastico, e che diventa familiare dopo le prime due o tre esperienze. Le variazioni nascono soprattutto dalla quantità e dal tipo di vincoli particolari che la scuola desidera esprimere; il flusso di base resta lo stesso.

Il primo passo è l'*importazione dell'anagrafica*. L'utente può partire da un template Excel o CSV (un foglio per ogni entità: docenti, classi, materie, aule, indirizzi, studenti, gruppi articolati), che il sistema fornisce come download dalla pagina *Importa*. Può anche partire, soprattutto per fini di prova, da un dataset mock generato deterministicamente da π Tantum stesso, in una delle cinque taglie predefinite (small, medium, big, huge, superhuge). Una volta importati i dati, l'utente li verifica e li completa nei tab specifici (Docenti, Classi, Materie, eccetera), correggendo errori, aggiungendo indisponibilità tramite le matrici 5-stati, settando preferenze, eccetera.

Il secondo passo è l'*aggiunta dei vincoli logici* specifici. Mentre i vincoli di base (non sovrapposizione, copertura, classi di concorso) sono codificati nel sistema e non richiedono intervento dell'utente, i vincoli particolari della singola scuola ("il prof Bianchi non lavora sabato", "la palestra è occupata dall'altro istituto il martedì mattina", "in 1A alternativa religione cattolica una volta al mese al giovedì") vengono espressi tramite il DSL generico, descritto in dettaglio nel cap. 17. La documentazione è accessibile dall'interfaccia stessa con un bottone di aiuto, e ogni vincolo è validato in tempo reale prima di essere salvato.

Il terzo passo è il *pre-controllo di Hall*. Prima di lanciare la pipeline vera e propria, conviene verificare che il problema sia almeno formalmente fattibile: la copertura delle ore è sostenuta dalla disponibilità dei docenti? In caso contrario, il sistema mostra il sotto-insieme di classi o materie che presentano un deficit, e l'utente può intervenire prima di consumare tempo di calcolo. Il pre-controllo è accessibile da tre punti distinti dell'interfaccia (un bottone inline nella card di Phase A, una riga nel pannello *tecniche avanzate*, e una sezione del tab Diagnostica), tutti che invocano lo stesso modulo `hall_check.py` e che producono lo stesso output, descritto nel cap. 12.

Il quarto passo è il *lancio della pipeline*. Dal tab Workflow, l'utente preme il bottone “Pipeline completa” e il sistema esegue in sequenza Phase A (assegnazione), Phase B (decomposizione + CP-SAT), Phase C (metaeuristiche) e Phase D (aule). Il log di esecuzione viene mostrato in tempo reale attraverso un canale Server-Sent Events; l'utente può seguire l'avanzamento, leggere i messaggi del solver, ed eventualmente fermare la pipeline se decide di intervenire. Al termine, il sistema mostra il valore della funzione obiettivo, le eventuali violazioni dei vincoli morbidi, e una panoramica dei tempi di esecuzione.

Il quinto passo è l'*editing manuale*. Dal tab Orario, l'utente può trascinare le lezioni da uno slot all'altro per correggere a mano dettagli che il solver non aveva considerato o che non corrispondono ai desideri ultimi del coordinatore. Mentre l'utente trascina, il sistema verifica in tempo reale se la mossa è fattibile dal punto di vista dei vincoli rigidi, e mostra come cambia la funzione obiettivo morbida. Tutto questo avviene senza dover rilanciare il solver: è una valutazione differenziale (*delta-evaluation*) che sfrutta la struttura della funzione obiettivo per ricalcolare solo le componenti che cambiano, in tempo costante.

Il sesto e ultimo passo, ricorrente nel corso dell'anno, è la *gestione delle assenze e delle supplenze*. Dal tab omonimo, l'utente segna i docenti assenti per ogni giornata, vede in tempo reale quali classi rimangono scoperte, e trascina i docenti disponibili come supplenti. Il sistema calcola automaticamente le compatibilità (chi può sostituire chi, considerando classi di concorso e disponibilità) e mostra le opzioni con un'evidenziazione cromatica. Una cella rossa significa “ancora scoperta”; una cella verde significa “coperta”.

Caso di studio

Liceo “Galileo”, 32 classi, settembre. Il vicepresidente parte da un import Excel con sessantasette docenti, trentadue classi, undici indirizzi (quattro scientifico, tre classico, due linguistico, due scienze applicate). Aggiunge quattro vincoli logici specifici (un docente che non lavora sabato, il laboratorio di fisica disponibile solo nel triennio, due compresenze fisse, una classe quinta che il martedì mattina ha attività esterna) e lancia la pipeline. In nove minuti π Tantum produce un orario completo che soddisfa tutti i vincoli rigidi e minimizza una funzione obiettivo composta da sette termini SOFT pesati. Un'occhiata al log mostra che Phase A ha impiegato dodici secondi, Phase B quattro minuti (la scuola è stata spezzata in cinque cluster da sei–otto classi), Phase C tre minuti di LNS più un minuto di simulated annealing, Phase D quaranta secondi. L'obiettivo finale è a 11.2 punti, con zero violazioni rigide e ventitré violazioni morbide, di cui quattordici sono “ore isolate” di docente che il vicepresidente decide di accettare consapevolmente.

2.5 Quando il sistema non basta da solo

Sarebbe disonesto presentare π Tantum come una soluzione completa che risolve tutti i casi senza intervento umano. Vi sono almeno tre situazioni ricorrenti in cui il sistema da solo non basta, e nelle quali la collaborazione fra software e coordinatore diventa indispensabile.

La prima situazione è quella dei *dati di input incompleti o contraddittori*. Se un docente è abilitato alla classe di concorso A027 ma il suo monte ore residuo totale supera la somma delle ore di matematica disponibili, il pre-controllo di Hall fallisce, e nessun algoritmo del mondo può costruire un orario fattibile a partire da quel dato. È un problema dei dati, non del solver: π Tantum non può “inventare” classi che non esistono o ore che non ci sono. La soluzione richiede l'intervento del coordinatore, che dovrà o assumere un docente in più, o ridurre il monte ore, o convocare delle supplenze straordinarie.

La seconda situazione è quella dei *vincoli sociali impliciti*. “La professoressa Rossi vorrebbe avere tutte le sue ore al mattino” non è un dato che il sistema possa indovinare: deve essere esplicitato come matrice di preferenza con celle pomeridiane in SOFT. Se la richiesta non viene codificata, il solver non la conosce, e

produrrà una soluzione che la ignora completamente. La regola pratica è: ogni richiesta personale o sociale deve essere tradotta in un vincolo prima di essere considerata.

La terza situazione è quella dell'*iterazione mancante con il committente*. La prima soluzione prodotta dal sistema produce sempre, in pratica, almeno qualche richiesta di modifica da parte del collegio dei docenti. Per questo π Tantum dispone di un editor manuale e di un sistema di vincoli logici incrementale: la pipeline non va rilanciata da zero ogni volta. Una richiesta del tipo “spostare l’inglese di 2B dal venerdì al giovedì” si gestisce con un drag-and-drop, e il sistema verifica in tempo reale che la mossa sia fattibile e ne calcola il delta sull’obiettivo. Ignorare questa modalità iterativa porta a un’esperienza frustrante per tutti.

2.6 Le tecnologie scelte e perché

L’ultima sezione di questa panoramica documenta le tecnologie sulle quali π Tantum è costruito e le ragioni della loro scelta. Tutte le tecnologie selezionate sono *open source* e prive di vincoli di licenza commerciale: nessuna scuola dovrebbe rinunciare al sistema per motivi economici, e nessun coordinatore dovrebbe trovarsi nella posizione di dover convincere il dirigente ad acquistare una licenza.

Il linguaggio principale è Python 3.11 o superiore. Si tratta del linguaggio dominante per il calcolo scientifico e per la modellazione a vincoli, e dispone di librerie mature per ogni esigenza che il progetto può avere. Le sue performance sono accettabili per questo dominio applicativo, perché la parte computazionalmente pesante (la risoluzione CP-SAT) è implementata in C++ all’interno della libreria OR-Tools, mentre il codice Python si limita a guidarla.

Il solver è OR-Tools di Google, in particolare il modulo CP-SAT. Si tratta di uno dei migliori solver al mondo per problemi combinatori di medie e grandi dimensioni, gratuito, con API Python pulite e documentate, e con prestazioni allineate ai prodotti commerciali più costosi. Ha vinto le edizioni 2018, 2019, 2020 e 2021 della MiniZinc Challenge, la competizione internazionale fra constraint solver. La sua maturità e la sua diffusione lo rendono una scelta naturale per chiunque voglia costruire un sistema di scheduling oggi.

Il backend è FastAPI, che fornisce un framework HTTP asincrono moderno con validazione automatica via Pydantic, generazione automatica della documentazione OpenAPI/Swagger, e ottime performance senza sacrificare la chiarezza del codice. La sua adozione semplifica drasticamente la scrittura e il mantenimento delle API REST.

Lo storage è SQLite, scelto per la sua zero-configuration: è un singolo file, non richiede un servizio dedicato, ed è più che sufficiente per i carichi di lavoro tipici di una scuola, anche grande. Le migrazioni sono scritte come funzioni Python idempotenti che vengono eseguite a ogni avvio del backend, evitando la necessità di un sistema di migration esterno come Alembic. Si tratta di una scelta consapevole che privilegia la semplicità operativa rispetto alla pulizia formale.

Il frontend è SvelteKit con Tailwind CSS, una combinazione che produce bundle finali piccoli, hot-reload veloce in sviluppo, e codice leggibile. La scelta di Svelte rispetto a React o Vue è legata alla sua filosofia “compiler-first”: il codice scritto viene tradotto in JavaScript vanilla ottimizzato, riducendo l’overhead a runtime. Vite, infine, fornisce il dev-server con proxy nativo verso il backend e un ciclo di compilazione di pochi millisecondi.

Le alternative considerate sono state numerose: Django o Flask invece di FastAPI, React o Vue invece di Svelte, PostgreSQL invece di SQLite, Gurobi commerciale invece di OR-Tools, MiniZinc come linguaggio di modellazione invece di Python diretto. Tutte sono state scartate per ragioni di leggerezza, costo, facilità di deployment in scuole italiane con infrastruttura modesta. La regola implicita è stata: ridurre al minimo le dipendenze e i punti di fallimento, preferire sempre la soluzione più semplice se sufficiente al caso d’uso.

3 L'orario come oggetto strutturale

Le entità coinvolte nella composizione di un orario scolastico sono sette: docenti, classi, materie, aule, indirizzi di studio, studenti e gruppi articolati. Ognuna possiede una propria anagrafica, una serie di attributi che la caratterizzano, e una collezione di relazioni con le altre entità. Le sezioni che seguono descrivono ciascuna entità e le relazioni principali che le legano.

3.1 I docenti

Il docente è l'entità forse più ricca di attributi del sistema, perché su di esso convergono numerose costrizioni contrattuali e organizzative. La sua anagrafica include il cognome, il nome, ed eventualmente un nickname per le visualizzazioni compatte. A ogni docente sono associate una o più *classi di concorso*, codificate con sigle alfanumeriche del tipo A027 o A019, che determinano quali materie egli sia abilitato a insegnare. Il monte ore massimo settimanale, tipicamente diciotto per il ruolo ordinario, costituisce un vincolo HARD invalicabile dal solver. Un giorno libero contrattuale, di norma sabato o lunedì a seconda della scuola, va parimenti rispettato in modo inderogabile.

Accanto a questi attributi obbligatori, il docente possiede una *matrice di disponibilità* di sei righe per sei colonne (sei giorni della settimana per sei ore al giorno), in cui ogni cella esprime uno dei cinque stati della palette di vincoli: ALLOWED per la disponibilità normale, SOFT per una preferenza negativa ("preferirei non lavorare in questo slot"), HARD per un'indisponibilità assoluta, PREFERRED per una preferenza positiva ("vorrei lavorare in questo slot"), ENFORCED per un'obbligazione ("devo lavorare in questo slot"). La matrice viene compilata dall'utente nell'interfaccia del tab Docenti, e ogni cella si modifica con un singolo click.

Il docente possiede anche una *matrice di preferenza per le aule*, sempre a cinque stati, che indica in quali aule egli preferisca insegnare; un *punteggio di graduatoria* usato dal solver per ordinare le scelte secondo il criterio dell'anzianità; e un'eventuale lista di *tre giorni preferiti per essere libero*, ordinati per priorità, che il sistema cerca di soddisfare in ordine. Ogni informazione di questo elenco si traduce in un contributo specifico alla funzione obiettivo o in un vincolo HARD per il solver.

3.2 Le classi

La classe rappresenta l'unità didattica di base. Ogni classe ha un nome (per esempio "3A scientifico"), un anno di corso da uno a cinque, un indirizzo di studio (che rimanda all'entità Indirizzo descritta più avanti), un numero di studenti attesi, e una *home class*, ovvero l'aula in cui di norma le sue lezioni si svolgono. Anche la classe possiede una propria matrice di disponibilità a cinque stati e a sei per sei celle, che codifica eventuali indisponibilità (per esempio l'attività di gita una volta al mese) o preferenze.

Sulla classe è possibile impostare un attributo `max_hours_per_day`, che specifica quante ore al giorno la classe possa al massimo svolgere. Il valore di default è sei, ma alcune scuole adottano cinque o, in casi particolari, sette ore al giorno; il vincolo è HARD se specificato. Un peso SOFT separato controlla quanto si voglia evitare la sesta ora come ultima ora del giorno: alcuni istituti privilegiano la quinta ora come termine canonico, relegando la sesta a slot da usare solo se necessario. Anche per le classi sono ammessi tre giorni preferiti per essere libere, con la stessa semantica usata per i docenti.

3.3 Le materie

La materia è descritta dal suo nome (Matematica, Italiano, Storia, Religione, Scienze Motorie, e così via) e da una *matrice di compatibilità con le aule*. Questa matrice, anch'essa a cinque stati, esprime per ogni coppia materia-aula il livello di adeguatezza: alcune materie richiedono aule specializzate (la fisica deve stare nel laboratorio di fisica, la chimica nel laboratorio di chimica, le scienze motorie in palestra), e per esse l'incompatibilità con le aule normali è un vincolo HARD; altre materie sono più flessibili e ammettono qualsiasi aula con preferenze sfumate.

A ogni materia è associato un peso che ne quantifica il “carico cognitivo”, usato dal solver per evitare di mettere in sequenza due materie particolarmente impegnative. Una matrice ulteriore, denominata *subject-group-weight matrix*, codifica quali classi di concorso siano abilitate a insegnare la materia in questione, raffinando la corrispondenza grezza nome-materia. È un'informazione necessaria perché nel sistema scolastico italiano la relazione fra materia e classe di concorso non è biunivoca: “Matematica” può essere insegnata da docenti di A026 (matematica nel biennio scientifico) o di A027 (matematica e fisica nel triennio scientifico) a seconda dell'anno di corso e dell'indirizzo.

3.4 Le aule

L'aula è descritta dal proprio nome (per esempio “Aula 12” o “Lab Fisica 1”), dal proprio *tipo* (aula normale, laboratorio di fisica, laboratorio di chimica, laboratorio di informatica, palestra, aula magna, e così via), dalla propria capienza, e da un flag *multi-class* che indica se l'aula possa ospitare più classi contemporaneamente, come accade tipicamente per l'aula magna o per i laboratori molto grandi. Anche per le aule esiste una matrice di disponibilità a cinque stati, che codifica eventuali periodi di manutenzione o di occupazione da parte di altre attività.

Ogni aula ha inoltre una propria *matrice di preferenze per le classi*, che indica quanto sia preferibile ospitare ciascuna classe specifica. È lo strumento con cui si codifica, per esempio, “l'aula 17 è la home della 3A” (preferenza alta per la 3A) o “l'aula 12 è troppo lontana dal laboratorio di chimica per le classi del triennio scientifico” (preferenza bassa per quelle classi). Si aggiunge infine una lista di *tag* liberi che il coordinatore può associare all'aula: stringhe come “matematica”, “piano_terra”, “accessibile_disabili” che servono come metadato per filtri e per vincoli logici DSL espressi in modo compatto.

3.5 Gli indirizzi di studio

L'indirizzo, che gli ordinamenti italiani designano anche con i sinonimi “curriculum” o “ordinamento”, rappresenta il programma di studi di una particolare articolazione scolastica: Liceo Classico, Liceo Scientifico, Liceo Linguistico, Istituto Tecnico Industriale (con le sue varie specializzazioni), Istituto Professionale per i Servizi, e così via. Per ogni anno di corso (dal primo al quinto) e per ogni materia, l'indirizzo specifica il *numero di ore settimanali* che la classe di quell'anno deve dedicare alla materia in questione. Sono questi monte ore che, sommati sulle classi che adottano l'indirizzo, generano il volume totale di lezioni che la pipeline deve collocare.

L'indirizzo può avere associati propri vincoli logici specifici, espressi nel DSL generico, che si applicano a tutte le classi dell'indirizzo. Esempi tipici sono “in prima classico la matematica non si tiene mai dopo la quarta ora” o “nel triennio scientifico le ore di laboratorio devono essere consecutive”. Questi vincoli si combinano con quelli specifici delle singole classi e con quelli generali della scuola, formando il livello più esterno della gerarchia di vincoli che il solver rispetta.

3.6 Gli studenti

Lo studente è un'entità anagrafica che appartiene di norma a una classe di base, ma che può anche essere membro di uno o più *gruppi articolati*, come la seconda lingua straniera del linguistico, la lezione di religione cattolica con l'alternativa per chi non si avvale, o un corso di recupero pomeridiano. A ogni studente possono essere associati dei *tag* liberi, stringhe come BES (Bisogni Educativi Speciali), DSA (Disturbi Specifici dell'Apprendimento), debito *matematica_4* per indicare un debito formativo specifico, o studente *atleta* per segnalare un percorso scolastico-sportivo. I tag sono metadato passivo che il sistema usa per filtri e per costruire vincoli logici espressi in modo conciso, senza modificare la struttura dei gruppi.

3.7 I gruppi articolati

Il gruppo articolato è l'entità che permette di rappresentare attività didattiche che attraversano i confini delle classi di base. Si tratta di insiemi di studenti, anche provenienti da classi diverse, che si incontrano in slot dedicati per un'attività comune. Esempi tipici sono il gruppo della seconda lingua straniera nel liceo linguistico, in cui gli studenti delle classi 4A e 4B si dividono fra francese, tedesco e spagnolo; oppure il gruppo IRC nei licei, in cui gli studenti che si avvalgono dell'insegnamento di religione cattolica si riuniscono in un'aula mentre quelli che hanno scelto l'alternativa vanno in un'altra; oppure ancora i corsi di recupero in itinere che il collegio docenti decide di attivare a metà anno per gli studenti con difficoltà in una particolare materia.

I gruppi articolati richiedono uno slot dedicato e un'aula specifica, e il loro funzionamento deve essere coordinato con l'orario delle classi di base dei loro membri: nessuno studente può essere richiesto in due luoghi contemporaneamente, e il sistema verifica questa coerenza come vincolo HARD. La semantica è quella che π Tantum chiama *Tipo C*, dal nome interno della prima implementazione: i membri possono provenire da qualunque combinazione di classi, e il gruppo non è mai un sottoinsieme rigido di una sola classe.

3.8 Le relazioni fra le entità

Le entità sopra descritte sono collegate da una rete di relazioni che costituisce il vero substrato del timetabling. La relazione fondamentale è la *cattedra*, che collega un docente a una classe e a una materia con l'indicazione del numero di ore settimanali. È l'unità elementare con cui si esprime “il professor Borghi insegna quattro ore di matematica alla 3A”. Una cattedra esiste soltanto se le classi di concorso del docente sono compatibili con la materia, e se il monte ore della classe per quella materia richiede effettivamente la copertura.

La *compresenza* è una relazione che lega due o più docenti alla stessa classe, alla stessa materia, e allo stesso slot. È un vincolo tipico delle materie tecnico-pratiche degli istituti tecnici, dove un insegnante teorico e un insegnante tecnico-pratico devono essere contemporaneamente in classe. La compresenza è un vincolo HARD non banale, perché non si limita a chiedere “due docenti” ma chiede “proprio quei due docenti, contemporaneamente, nella stessa aula”.

La *lezione* è l'istanza concreta che la pipeline produce: una specifica cattedra (o compresenza) collocata in uno slot specifico della settimana, con un'aula assegnata. Una soluzione completa di π Tantum non è altro che l'insieme delle lezioni che ricoprono interamente le ore richieste dalle cattedre, rispettando tutti i vincoli HARD e minimizzando il valore della funzione obiettivo SOFT.

L'*indisponibilità HARD* non è una relazione fra entità in senso stretto, ma una proprietà delle matrici sei per sei dei docenti, delle classi e delle aule. Tecnicamente è implementata come una collezione di celle marcate, e i vincoli che ne derivano vengono iniettati direttamente nel modello CP-SAT al momento della risoluzione. Il *vincolo logico DSL*, infine, è un'espressione del DSL generico (descritto nel cap. 17) che lega entità in qualunque combinazione, ed è il meccanismo principale con cui l'utente esprime regole specifiche della sua scuola che il modello dati pre-confezionato non cattura.

3.9 I vincoli ricorrenti del caso italiano

Alcuni vincoli ritornano in tutte le scuole italiane, anche se con sfumature diverse, e meritano di essere descritti come categoria. Il vincolo di *copertura* richiede che ogni materia riceva il proprio monte ore in ogni classe, e costituisce la finalità stessa della pipeline: senza copertura completa non c'è neppure un orario. Il vincolo di *non sovrapposizione* richiede che docente, classe e aula siano impegnati in al più una lezione per slot, ed è naturalmente HARD. Il vincolo di *compatibilità per classe di concorso* richiede che soltanto i docenti abilitati possano insegnare una determinata materia.

I vincoli di *compatibilità aula-materia* sono per lo piuttosto rigidi quando l'attività richiede strumentazione specifica (la fisica nel laboratorio di fisica, le scienze motorie in palestra, il disegno tecnico nell'aula da disegno), ma diventano preferenze morbide nei casi più flessibili. Il *giorno libero contrattuale* è un vincolo HARD per contratto collettivo nazionale, che ogni docente ha diritto di vedere rispettato in uno specifico giorno della settimana.

I vincoli sull'*evitamento dei buchi* (le ore vuote nel mezzo della giornata di una classe o di un docente) sono HARD nelle scuole più rigide e SOFT in quelle più flessibili, e π Tantum li configura per default come HARD per le classi (perché una classe con buchi crea problemi organizzativi reali) e SOFT per i docenti (perché i buchi nei docenti sono fastidiosi ma non impediscono il funzionamento della scuola). I vincoli sulla *sesta ora*, sulla *distribuzione settimanale di una stessa materia*, e sulle *coppie di ore consecutive* per le materie che lo richiedono completano il quadro dei vincoli ricorrenti.

3.10 Cosa rende peculiare il sistema italiano

Il caso italiano si distingue dai benchmark accademici internazionali, codificati in formati standard come ITC-2007, ITC-2011 e XHSTT, per l'esistenza degli indirizzi di studio come entità di prim'ordine e la loro diversificazione di monte ore anche per materie nominalmente identiche. Si distingue anche per l'esistenza delle classi di concorso esplicite e gerarchiche, che vincolano in modo fine la corrispondenza fra docenti e materie. Si distingue ancora per la presenza diffusa di compresenze come vincoli HARD ricorrenti, per i gruppi articolati che attraversano le classi di base, e per i vincoli sindacali derivanti dal contratto collettivo nazionale.

Caso di studio

IIS “Galileo Ferraris” di Torino, ventotto classi distribuite su tre indirizzi (ITIS Informatica, ITIS Elettronica, IPS Servizi Commerciali). Le tre articolazioni hanno monte ore diversi anche per materie comuni come matematica e italiano. I docenti di matematica delle classi di concorso Ao26 e Ao27 coprono tutte e tre le articolazioni; i docenti specifici, come quello di Sistemi e Reti per l'indirizzo Informatica, sono limitati a una sola articolazione. Le compresenze sono numerose, in particolare fra matematica e laboratorio di informatica nell'indirizzo Informatica e fra fisica e laboratorio di fisica nell'indirizzo Elettronica. Le settimane di stage in azienda si concentrano nelle ultime settimane dell'anno scolastico, e per esse π Tantum gestisce la situazione attraverso un vincolo logico DSL che svuota i pomeriggi di martedì e giovedì delle classi quinte nel periodo aprile-maggio.

Tutte queste peculiarità sono codificate come entità di prima classe o come vincoli espliciti nel modello dati di π Tantum, descritto in dettaglio nel cap. 19. Il progetto è stato disegnato a partire da queste specificità, e non come adattamento postumo di un sistema accademico generico. È forse il contributo più distintivo della filosofia su cui il software è costruito: non un solver più veloce in assoluto, ma un solver che parla la lingua del sistema scolastico italiano e che si lascia istruire con il vocabolario di chi quel sistema lo conosce.

4 Installazione

Guida cross-platform per installare ed eseguire piTantum su Windows, Linux e macOS. La versione *di riferimento* di questa guida è docs/installation.md nel repo (Markdown, sempre aggiornato); qui se ne riproduce il contenuto in forma compatta per il manuale PDF. In caso di divergenza, fa fede il file Markdown.

Architettura del lancio. Il progetto è composto da due processi:

- **Backend:** uvicorn che serve l'app FastAPI in webui/backend sulla porta 8000.
- **Frontend:** server di sviluppo vite che serve l'app SvelteKit in webui/frontend sulla porta 5173.

Uno script unico (start.bat su Windows, start.sh su Linux/macOS) li avvia entrambi. Al primo lancio crea la *venv* Python, installa i requirements, fa npm install; ai lanci successivi parte in pochi secondi.

4.1 Windows

Prerequisiti.

- Python ≥ 3.11 – installer ufficiale: <https://www.python.org/downloads/windows/>
- Node.js ≥ 20 LTS – installer ufficiale: <https://nodejs.org/en/download> (scegliere “LTS”).
- Git – installer ufficiale: <https://git-scm.com/download/win>.

Note importanti.

- Sull’installer Python, **spuntare** “Add python.exe to PATH”. Senza, lo script non trova il comando python.
- Riavviare il terminale dopo le installazioni: il PATH non si aggiorna nelle finestre già aperte.

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
webui\start.bat
```

Lo script apre due finestre cmd (“piTantum – backend” e “piTantum – frontend”). Aprire il browser su <http://localhost:5173>.

Stop. Chiudere le due finestre cmd, oppure Ctrl+C in ciascuna.

Troubleshooting Windows.

- “python non riconosciuto”: l’installer Python non è stato eseguito con “Add to PATH”. Reinstallare oppure aggiungere C:\Users\tu\AppData\Local\Programs\Python\Python311 al PATH.
- Porta 8000 occupata: netstat -ano | findstr :8000 + taskkill /PID <pid> /F.
- Antivirus blocca start.bat: aggiungere alle eccezioni di Windows Defender.

4.2 Linux (Ubuntu, Debian, Fedora, Arch)

Prerequisiti. Python ≥ 3.11 , Node ≥ 20 LTS, Git, build-essential.

Ubuntu / Debian.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nodejs npm \
    git build-essential
```

I repo apt di Ubuntu 22.04 / Debian 12 hanno Node 12-18, troppo vecchio per SvelteKit 2 (richiede 20+). In quel caso usare **nvm**:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts && nvm use --lts
```

Fedora.

```
sudo dnf install -y python3 python3-virtualenv nodejs npm git \
    gcc make
```

Arch / Manjaro.

```
sudo pacman -S python python-pip nodejs npm git base-devel
```

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
chmod +x webui/start.sh webui/stop.sh
./webui/start.sh # background; log in webui/logs/
./webui/start.sh --foreground # log live in console (Ctrl+C ferma)
```

Stop.

```
./webui/stop.sh
```

Troubleshooting Linux.

- Porta in uso: `sudo lsof -i :8000 + kill <pid>` (eventualmente `kill -9 <pid>`).
- python3 mancante su distro minimal: usare l'eseguibile chiamato `python (3.x)`.
- WSL2: `start.sh` funziona normalmente. Aprire il browser dal lato Windows su `http://localhost:5173`; WSL2 inoltra le porte automaticamente.

4.3 macOS (Intel + Apple Silicon)

Prerequisiti.

- Xcode Command Line Tools: `xcode-select --install`.
- Homebrew: vedere <https://brew.sh/>.
- Python ≥ 3.11 , Node ≥ 20 LTS, Git: tutti via Homebrew.

Installazione.

```
# Homebrew (se non presente)
/bin/bash -c "$(curl -fsSL
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Toolchain
brew install python@3.12 node git

# Verifica
python3 --version # >= 3.11
node --version # >= v20
```

Note Apple Silicon.

- ortools dalla v9.7 spedisce wheel ARM64 nativi: Rosetta non serve.
- Se Python è installato da python.org invece che Homebrew, scegliere l'installer "macOS 64-bit universal2".

Clone + lancio + stop. Identico a Linux.

Troubleshooting macOS.

- Gatekeeper blocca start.sh: `xattr -d com.apple.quarantine webui/start.sh webui/stop.sh`.
- ortools senza wheel ARM (versioni vecchie): `pip install --upgrade ortools` dentro la venv del backend.
- brew non in PATH: eval `"$(/opt/homebrew/bin/brew shellenv)"` in `~/zprofile`.

4.4 Verifica installazione

Dopo il primo lancio andato a buon fine:

```
python --version # >= 3.11 (Linux/macOS: spesso python3)
node --version # >= v20
npm --version # >= 10

# Backend health
curl http://127.0.0.1:8000/api/health
# atteso: {"status":"ok","name":"pitantum","version":"0.1.0"}

# Async layer (Section 2.5 P2)
curl http://127.0.0.1:8000/api/health/async
# atteso: {"status":"ok","async":true,"dialect":"sqlite"}

# Frontend
curl -I http://127.0.0.1:5173
# atteso: HTTP/1.1 200 OK
```

4.5 Aggiornamento

Quando si fa `git pull` per portare a casa nuovi commit:

```
cd /path/to/timetable
git pull origin main

# se sono cambiate dipendenze Python:
webui/backend/.venv/bin/pip install -r webui/backend/requirements.txt
# (Windows: webui\backend\.venv\Scripts\pip install ...)

# se sono cambiate dipendenze frontend:
cd webui/frontend && npm install && cd ../..

# se ci sono migration DB:
cd webui/backend && ../.venv/bin/alembic upgrade head && cd ../..
# (Windows: webui\backend\.venv\Scripts\alembic upgrade head)
```

In pratica `start.sh/start.bat` rilanciato dopo un `git pull` gestisce automaticamente `pip install` e `npm install` quando servono. Le migration DB sono coperte da `__apply__lightweight__migrations()` allo startup come `safety-net` (vedi capitolo *Modello dati*); per la via canonica usare `alembic upgrade head`.

Dopo un aggiornamento backend, fare un **hard reload** del browser (Ctrl+Shift+R) per pulire chunk JavaScript cached.

4.6 Disinstallazione

```
cd /path/to/timetable
./webui/stop.sh # Linux/macOS
# (Windows: chiudere le finestre cmd di start.bat)

rm -rf webui/backend/.venv # ~150 MB
rm -rf webui/frontend/node_modules # ~400 MB
rm -rf webui/data/timetable.db # DATI UTENTE (salvarli prima!)
```

Per rimuovere completamente, cancellare la cartella `timetable/`. Python, Node, Git restano installati nel sistema; rimuoverli dal gestore di pacchetti solo se non servono ad altri progetti.

5 Il modello a vincoli

“Constraints are not the enemy of creativity. Constraints are creativity’s best friend.”

Twyla Tharp, *The Creative Habit*

COSTRUIRE un orario significa, dal punto di vista formale, soddisfare un sistema di vincoli. Alcuni di essi devono valere senza eccezione, altri esprimono preferenze che il sistema cerca di rispettare il più possibile pur sapendo che, in caso di conflitto, sarà necessario un compromesso. Questo capitolo descrive con ordine la tassonomia dei vincoli che π Tantum riconosce, il modo in cui essi vengono espressi nell’interfaccia, e la loro traduzione interna in vincoli per il solver CP-SAT.

La distinzione fra vincoli rigidi e morbidi non è un dettaglio implementativo: è la chiave che permette al sistema di restituire soluzioni utilizzabili. Trattare ogni richiesta dell’utente come inderogabile produrrebbe sistemi cronicamente infattibili, mentre trattare ogni vincolo come puramente preferenziale violerebbe le regole contrattuali e le esigenze didattiche di base. La gradazione in cinque livelli che ora descriveremo è il risultato di un compromesso meditato fra espressività per l’utente e trattabilità per il solver.

5.1 La tassonomia dei cinque stati

I vincoli di π Tantum sono organizzati in una tassonomia uniforme di cinque stati, applicata sia alle matrici di disponibilità (cella per cella, per docenti, classi e aule) sia alle preferenze di aula. I vincoli logici disgiuntivi, espressi nel DSL del sistema, riconoscono quattro di questi stati (l’ALLOWED non si applica a quel contesto). La tabella 5.1 riassume i cinque stati con il loro colore convenzionale, la loro semantica e l’effetto che producono sul solver.

Stato	Colore	Semantica
ALLOWED	verde chiaro	default neutro, applicabile soltanto alle griglie materia–aula e docente–aula; non contribuisce all’obiettivo
SOFT	giallo	penalità positiva quando lo slot viene utilizzato; il valore +penalty viene sommato all’obiettivo che il solver minimizza
HARD	rosso	vincolo inderogabile: il solver deve evitare lo slot oppure soddisfare il vincolo logico in modo assoluto
PREFERRED	blu	bonus negativo quando lo slot viene utilizzato o quando il vincolo logico è soddisfatto; il -bonus riduce l’obiettivo
ENFORCED	verde scuro	il solver deve forzare l’occorrenza dello slot o la soddisfazione del vincolo, con <i>presence required</i> attiva

Tabella 5.1 – I cinque stati della tassonomia di π Tantum, con colori, semantica e contributo alla funzione obiettivo.

Ognuno di questi stati ha un significato preciso che vale la pena commentare. Lo stato ALLOWED, codificato in verde chiaro, rappresenta la situazione di partenza: quella in cui nessuna preferenza esplicita è stata espressa. È applicabile soltanto a quelle griglie in cui ha senso distinguere fra “ammesso” e “vietato”, come la compatibilità fra una materia e un’aula. Per le matrici di disponibilità settimanale di docenti, classi e aule, lo stato di partenza è implicitamente la disponibilità, e ALLOWED non viene mai utilizzato.

Lo stato SOFT, codificato in giallo, esprime una preferenza negativa: l'utente preferirebbe non utilizzare quello slot, ma se serve si può accettare. Quantitativamente, l'uso dello slot aggiunge un costo positivo all'obiettivo, e il solver cerca di evitarlo nei limiti del possibile. Il valore del costo è configurabile, e l'utente lo regola attraverso un campo numerico che appare nella cella stessa quando essa viene marcata come gialla.

Lo stato HARD, codificato in rosso, esprime un divieto assoluto. Il solver *non può* utilizzare quello slot, o equivalentemente *non può* violare il vincolo logico associato. Una soluzione che viola anche un solo vincolo HARD non è una soluzione: viene scartata immediatamente dal solver. Lo stato HARD si usa per codificare le indisponibilità contrattuali, le incompatibilità fisiche, e tutte le richieste che non ammettono eccezioni.

Lo stato PREFERRED, codificato in blu, è simmetrico rispetto al SOFT: esprime una preferenza positiva, una configurazione che si vorrebbe vedere accadere. L'uso dello slot porta un bonus negativo all'obiettivo, e il solver è quindi spinto a scegliere quella configurazione quando ne ha possibilità. È utile per esprimere desideri come “preferirei avere il laboratorio in due ore consecutive” o “preferirei avere la palestra il giovedì”.

Lo stato ENFORCED, codificato in verde scuro, esprime una richiesta assoluta in senso opposto a HARD: il solver *deve* far accadere quella configurazione, non soltanto deve permetterla. È lo strumento con cui si codifica, per esempio, “la riunione di consiglio si tiene giovedì alla quinta ora” (lo slot è *pre-allocato*). È un livello da usare con parsimonia, perché ENFORCED multipli e in conflitto fra loro rendono il problema infeasibile, e il sistema non può fare altro che segnalare l'impossibilità.

5.2 Le matrici di disponibilità

Le matrici di disponibilità sono lo strumento principale con cui l'utente esprime preferenze e indisponibilità per docenti, classi e aule. Ognuna di queste entità possiede una matrice di sei righe per sei colonne, che corrispondono ai sei giorni della settimana scolastica e alle sei ore di lezione tipiche di una giornata. Cliccando su una cella con il mouse, l'utente la fa ciclare attraverso gli stati disponibili nella sequenza libero → giallo (SOFT) → rosso (HARD) → blu (PREFERRED) → verde scuro (ENFORCED) → libero, in modo da poter raggiungere qualunque stato con al massimo cinque click.

Quando una cella viene marcata come SOFT (gialla) o PREFERRED (blu), accanto al colore appare un campo numerico che permette di editare la penalità o il bonus associati. La logica dei segni è gestita automaticamente dal sistema: i valori positivi sono ammessi soltanto sulle celle gialle e i valori negativi soltanto su quelle blu, e il backend applica un *sign-clamp* al salvataggio per evitare incoerenze. Trascinando il mouse mentre si tiene premuto il tasto, si può applicare lo stesso stato a un blocco contiguo di celle, comodità che velocizza l'editing quando un'intera giornata o un'intera fascia oraria deve essere marcata in modo uniforme.

Una funzionalità di sincronizzazione automatica governa il rapporto fra il campo *free day* del docente (il giorno libero contrattuale) e la matrice. Quando il *free__day* è impostato, tutte e sei le ore di quel giorno appaiono automaticamente HARD nella matrice, in modo visivo, anche se l'utente non le ha marcate esplicitamente. Viceversa, se l'utente edita la matrice manualmente e tutte le ore di un giorno diventano HARD, il backend deduce da questo che il *free__day* è quel giorno e lo aggiorna di conseguenza. La logica è implementata nel modulo `webui/backend/db.py` e garantisce che le due rappresentazioni restino sempre coerenti.

La traduzione interna delle matrici nel modello CP-SAT avviene nel modulo `optimization.py`, all'interno della funzione `__availability_constraints`. Per ogni entità (docente, classe, aula), la matrice viene analizzata e le sue celle si traducono in tre strutture dati: un insieme di tuple `<entity>__hard` che rappresentano le *forbidden assignments*, un dizionario `<entity>__soft` che mappa tuple a penalità (con segno già corretto), e un insieme di tuple `<entity>__enforced` che rappresentano le *presence required* nel solver. CP-SAT usa queste strutture per costruire i vincoli durante la fase di risoluzione.

5.3 I vincoli logici disgiuntivi

Accanto alle matrici di disponibilità, il sistema offre agli utenti uno strumento più espressivo per codificare vincoli che non si lascino ridurre a singole celle: i *vincoli logici disgiuntivi*, spesso indicati con l'acronimo DNF dall'inglese *Disjunctive Normal Form*. Si tratta di espressioni logiche che combinano slot e predicati

attraverso gli operatori AND, OR e NOT, e che si applicano a una entità (docente, classe, aula, indirizzo) con un determinato livello di rigidità (HARD, SOFT, PREFERRED, ENFORCED).

5.3.1 La grammatica delle espressioni

La sintassi delle espressioni logiche di π Tantum è stata pensata per essere compatta e leggibile. La forma BNF semplificata è mostrata nel listato che segue, e descrive gli elementi che si possono combinare.

```
expr := or_expr
or_expr := and_expr ('OR' and_expr)*
and_expr := not_expr ('AND' not_expr)*
not_expr := ('NOT' | 'mai') not_expr | atom
atom := slot | predicate | '(' expr ')'
slot := <giorno><ora>
predicate:= <kind> ':' <name> ('@' <giorno> <ora>)?
kind := aula | materia | classe | gruppo | docente | prof
giorno := lun|mar|mer|gio|ven|sab (anche lunedì|...|sabato,
        monday|...|saturday)
ora := 1..6 (ordinale, 1=08:00) | 8..13 (assoluta)
```

La grammatica accetta anche le forme abbreviate & per AND, | per OR e ! per NOT, oltre all’alias italiano mai per NOT che produce espressioni più naturali nei casi tipici. Per esempio, mai aula:LabFisica@mar si legge in modo trasparente come “mai nel laboratorio di fisica il martedì”.

5.3.2 I predicate atoms

Oltre agli slot temporali grezzi, le espressioni possono contenere *predicate atoms* che si riferiscono ad entità specifiche. La sintassi dei predicate atoms è illustrata negli esempi seguenti.

```
aula:LabFisica -- usa l'aula LabFisica a qualunque slot
aula:LabFisica@mar -- usa LabFisica il martedì a qualunque ora
aula:LabFisica@mar4 -- usa LabFisica martedì alla 4a ora (11:00)
materia:Religione@lun8 -- lezione di religione lunedì alle 8
classe:1A_Scientifico -- riferimento alla classe 1A scientifico
gruppo:IRC -- riferimento al gruppo IRC
```

I predicate atoms vengono parsati e persistiti correttamente dal sistema. Il solver li tratta in modo coerente per le restrizioni temporali; l’integrazione completa con la fase di assegnazione delle aule è oggetto di sviluppo incrementale, descritto nella roadmap del progetto.

5.3.3 Esempi pratici

Il modo migliore per familiarizzare con la sintassi dei vincoli logici è leggere alcuni esempi reali di uso. I seguenti esempi sono presi dalla pratica di alcune scuole che usano π Tantum.

```
% Per docenti
lun8 AND lun9
(lun8 AND lun9) OR (mar8 AND mar9)
gio11 OR gio12 OR gio13
NOT (mer3 AND mer4)
mai aula:LabFisica@mar
mai materia:Religione@lun8

% Per classi
mer4 AND mer5 AND mer6
```

```
mai aula:Palestra@gio  
aula:LabInformatica@mar OR aula:LabInformatica@gio
```

```
% Per aule  
gio4 AND gio5 AND gio6  
mai materia:Religione  
gruppo:IRC OR gruppo:Alternativa
```

Il primo esempio per i docenti, `lun8 AND lun9`, significa che il docente deve essere presente lunedì alle otto e alle nove (per esempio per una compresenza obbligatoria). Il secondo esempio, con `OR` fra due congiunzioni, ammette due alternative: o lunedì alle otto-nove, o martedì alle otto-nove. Il quarto, con `NOT`, vieta la combinazione mercoledì terza-quarta ora. I tre esempi con `mai` esprimono divieti su entità specifiche, leggibili in modo quasi colloquiale.

5.3.4 Mapping CP-SAT

Per ogni regola con DNF [`clause1, clause2, ...`], il solver applica una traduzione che dipende dal kind del vincolo. Una regola `HARD` impone che almeno una clausola sia *fully active*, ovvero che tutti i suoi atomi siano veri. Una regola `SOFT` permette che nessuna clausola sia attiva, ma in tal caso paga la penalità positiva `+soft_penalty`. Una regola `PREFERRED` dota di un bonus negativo `-soft_penalty` la situazione in cui almeno una clausola è attiva, riducendo così l'obiettivo da minimizzare. Una regola `ENFORCED` si comporta come una `HARD` ma con *presence required*: almeno uno degli slot della DNF deve coincidere con un'ora di lezione attiva per l'entità in questione, non basta che il vincolo sia formalmente soddisfatto sui domini.

5.4 I toggle `HARD` per classe

Oltre ai vincoli espressi nelle matrici e ai vincoli logici, il sistema mette a disposizione una serie di vincoli `HARD` predefiniti, espressi come booleani nella tabella `school_classes`, che la maggior parte delle scuole usa con valori standard. Sono sette: `hard_entry_at_8` (impone che la prima lezione cominci alle otto), `hard_exit_after_12` (impone che l'ultima lezione non finisca prima delle dodici), `hard_no_holes` (vieta i buchi nell'orario della classe), `hard_dual_math` (impone che la matematica si tenga in coppie di ore consecutive), `hard_dual_italian` (idem per italiano), `hard_motorie_pairs` (idem per educazione fisica), e `hard_max_6_per_day` (impone il limite di sei ore al giorno).

A questi sette toggle si affianca il parametro `SOFT soft_minimize_sixth_weight`, che regola il peso dell'obiettivo di evitare la sesta ora come ultima del giorno. Il valore predefinito è tarato per esprimere una preferenza moderata; valori più alti fanno sì che il solver eviti la sesta ora con maggiore insistenza, valori più bassi la accettano più liberamente.

5.5 Il rilevamento dei conflitti

Il tab `Vincoli` dell'interfaccia fornisce un bottone denominato “Cerca conflitti” che, attivato, esegue una diagnostica sul sistema di vincoli correntemente espresso e segnala incoerenze rilevabili senza dover lanciare il solver. Il bottone fa una chiamata `HTTP` a `GET /api/monitor/conflicts`, e l'output viene mostrato all'utente in una tabella con tre tipi di errore.

Il primo tipo, denominato `matrix_hard_enforced`, segnala le celle marcate sia come `HARD` sia come `ENFORCED` contemporaneamente. Si tratta di una contraddizione diretta, perché una stessa cella non può essere allo stesso tempo proibita e obbligatoria. Il secondo tipo, denominato `enforced_on_free_day`, segnala le celle `ENFORCED` del docente che cadono nel suo giorno libero contrattuale: anche questa è una contraddizione, risolvibile o spostando l'`ENFORCED` o cambiando il `free_day`. Il terzo tipo, denominato `logical_unsatisfiable`, segnala i vincoli logici `HARD` o `ENFORCED` la cui DNF non è soddisfacibile dato l'insieme degli slot `HARD` del proprietario. È un caso che richiede al coordinatore di rivedere il vincolo o di rilassare le indisponibilità.

5.6 Tag delle aule

Per esprimere caratteristiche delle aule che non si lasciano ridurre al solo nome o tipo (“questa aula ha il proiettore”, “questa aula è adatta alla matematica perché ha la lavagna grande”, “questa aula è una sala riunioni”), il sistema offre i *tag delle aule*: si tratta di etichette libere, vere e proprie stringhe, che il coordinatore associa a ciascuna aula dalla scheda Aule. Ogni aula può avere quanti tag desidera, e ogni tag può essere riutilizzato su più aule. La struttura non impone una tassonomia gerarchica: i tag formano un semplice insieme.

L'utilità dei tag emerge soprattutto nella scrittura dei vincoli. Invece di scrivere “solo queste tre aule specifiche”, enumerandole per nome (un elenco che andrebbe aggiornato ogni volta che le aule cambiano), il coordinatore può scrivere “aule taggate matematica”, e il vincolo seguirà automaticamente le etichettature correnti. Se domani viene aggiunta una nuova aula taggata matematica, il vincolo la includerà senza ulteriori interventi.

Un esempio pratico chiarisce la logica. Supponiamo di volere che la matematica si tenga soltanto in aule taggate matematica. Una volta etichettate dalla scheda Aule le aule che vogliamo includere, il vincolo nel DSL generico si scrive così:

```
forall l in lessons where l.subject == Matematica:  
    "matematica" in l.classroom.tags
```

Il sistema offre anche una serie di tag automatici prodotti dalla generazione mock. Per esempio, le aule home class ricevono il tag dell'indirizzo della classe assegnata, permettendo di scrivere vincoli del tipo “le lezioni della 3A devono stare in aule taggate scientifico”. I dettagli della generazione automatica si trovano nel modulo `webui/backend/mock_classrooms.py`.

5.7 Le preferenze di giorno libero per i docenti

I docenti hanno diritto, per contratto collettivo nazionale del settore scolastico italiano, a un certo numero di giorni liberi a settimana, tipicamente uno per il personale a tempo pieno. π Tantum gestisce questa esigenza in modo flessibile attraverso un meccanismo a tre slot di preferenza ordinati. Il primo slot rappresenta il giorno libero ideale del docente, per esempio il sabato; il secondo rappresenta il giorno di ripiego se il primo non è realizzabile, per esempio il mercoledì; il terzo rappresenta l'ultima alternativa, per esempio il lunedì.

Per ciascuna delle tre preferenze, il docente o il coordinatore può scegliere se essa sia di tipo HARD (assoluta, non negoziabile) o di tipo SOFT con un peso configurabile. A questo si aggiunge il campo `required_free_days_count`, che indica quanti giorni liberi servano in totale: il valore predefinito è uno, conforme al contratto collettivo, ma scuole con docenti a tempo parziale possono richiederne di più.

Un esempio illustra la flessibilità del meccanismo. Si consideri un docente con le preferenze [sabato HARD, mercoledì SOFT con peso 50, lunedì SOFT con peso 20] e con `required_free_days_count = 1`. Il sistema cercherà sicuramente di mantenere libero il sabato, perché è HARD; se non riuscisse, in seconda istanza cercherà di liberare il mercoledì (con un costo SOFT di 50 se non ci riesce); in terza istanza cercherà di liberare il lunedì (con un costo di 20). La gradazione permette al sistema di scegliere il compromesso più ragionevole quando le preferenze multiple dei vari docenti entrano in conflitto fra loro.

5.8 Le preferenze di giorno libero per le classi

Anche le classi possono avere giorni preferiti di non attività, e la struttura del meccanismo è analoga a quella dei docenti: tre slot ordinati con flag HARD o SOFT, più un `required_free_days_count`. La motivazione è diversa: una classe può desiderare di avere il sabato libero per attività di stage, oppure il lunedì per evitare il rientro tipicamente faticoso dopo il fine settimana.

In aggiunta alle preferenze di giorno libero, le classi possiedono il campo `max_hours_per_day`, che impone un tetto HARD al numero di ore in un giorno. Il valore predefinito è cinque, ma alcuni istituti adottano quattro per le classi del biennio (per ridurre il carico giornaliero) o sei o sette per casi particolari. Riducendo

il tetto si ottengono giornate più leggere, ma se il monte ore complessivo della classe è alto sarà necessario estendere il calendario settimanale al sabato per non sforare. Il sistema gestisce automaticamente questa redistribuzione, e segnala all'utente i casi in cui un tetto troppo basso renderebbe il problema infeasibile.

5.9 Compresenze, sostegno e potenziamento

La scuola italiana ha tre famiglie di vincoli che il sistema modella nativamente: le **compresenze** (il laboratorio dove lavorano due docenti insieme), il **sostegno** (il prof DVA che segue uno studente specifico), e il **potenziamento** previsto dalla Legge 107 del 2015 (le ore dell'organico potenziato, senza cattedra fissa). Sono entrati nel sistema a partire dal Task C1 e sono enforced come constraint CP-SAT, non gestiti post-hoc. Le violazioni sono restituite come errore al POST con un messaggio specifico, prima che la run parta.

5.9.1 Compresenze “shared” (lab di chimica)

Caso d'uso: laboratorio di chimica del 2C, 4 ore alla settimana, di cui 2 in compresenza con l'assistente di laboratorio. La 2C ha le sue 4 ore di chimica, ma in 2 di queste è prevista la presenza simultanea di due docenti.

Configurazione dalla UI:

1. Tab /assignments: la cattedra principale di ProfChim (4 ore) è già presente.
2. Tab /coteaching: nuovo gruppo (2C, Chimica, n_hours=2, kind=shared, required=True).
3. Tab /assignments: aggiungi una nuova cattedra ProfAss (2 ore di Chimica in 2C) e collegala al gruppo di compresenza appena creato.

Cosa fa il solver: introduce una variabile `coday_count[gid, d]` per ogni giorno, vincolata a sommare a `n_hours` sull'arco settimanale. Il docente principale (quello con più ore, in questo caso ProfChim) ha `day_count[principal, d] >= coday_count[gid, d]`: le sue ore di lezione coprono sempre quelle di compresenza. Il co-docente (ProfAss) ha invece `day_count[codoc, d] == coday_count[gid, d]`: le sue 2 ore coincidono esattamente con le ore di compresenza. A livello di slot orario, i due docenti vengono messi nella stessa cella tramite una variabile booleana di slot condiviso.

5.9.2 Sostegno: il prof DVA che segue lo studente

Caso d'uso: il prof Bianchi è il docente di sostegno per uno studente DVA della 2A, e ha una cattedra da 18 ore di sostegno. Il prof segue le lezioni della classe (non ne aggiunge di proprie) e deve essere presente solo quando 2A ha lezione con un altro docente.

Configurazione:

1. Tab /teachers: ProfBianchi esiste con la materia “sostegno” nelle abilitazioni.
2. Tab /assignments: nuova cattedra (ProfBianchi, 2A, sostegno, ore=18, is_support=True).

Cosa fa il solver: la cattedra di sostegno NON contribuisce all'orario della 2A (le 18 ore non vanno a sommarsi alle ore curricolari). Il vincolo è `slot[ProfBianchi, 2A, sostegno, h] <= 2A_busy[h]`: il prof di sostegno è presente solo dove la 2A ha già una lezione con un altro docente. La somma delle sue ore settimanali è fissata a 18, distribuita in modo che il profilo sia ragionevole.

5.9.3 Potenzamento (Legge 107)

Caso d'uso: la prof Verdi è nell'organico potenziato e ha 18 ore settimanali di potenziamento, senza una cattedra fissa. Le ore vengono usate per progetti, recupero o supplenze.

Configurazione:

1. Tab /assignments: nuova cattedra (ProfVerdi, classe=NULL, materia=Potenziamento, ore=18, is_potenziamento=True). La cattedra viene visualizzata in una sezione dedicata in fondo al tab, **Potenziamento (Legge 107)**.

Cosa fa il solver: introduce `pot_day_count[ProfVerdi, d]` per ogni giorno, con somma settimanale pari a 18. Il vincolo `HARD pot_day_count[d] + ore_di_cattedra[d] <= 5` garantisce che la prof non superi il tetto giornaliero. Cap settimanale `HARD`: 30 ore (cinque giorni per cinque ore ciascuno, escludendo il sabato).

Visibilità nel tab Assenze/Supplenze: quando un docente è assente, il sistema cerca un supplente fra i disponibili. I docenti di potenziamento vengono mostrati per primi, con un badge **POT** viola e un bordo dello stesso colore: l'idea è che la loro disponibilità sia specificamente progettata per coprire questi buchi.

5.10 Religione, alternativa e parallelismi intra-classe

La classe 3B il martedì alla terza ora ha religione cattolica con un docente, e contemporaneamente alternativa con un altro docente, perché gli studenti sono divisi. Dal punto di vista dell'orario della 3B la classe è "occupata" in quello slot (gli studenti non sono disponibili per altre lezioni), ma sono in atto due lezioni parallele, non una.

Configurazione (Task C2):

1. Tab /assignments: due cattedre, (ProfRel, 3B, Religione, 1) e (ProfAlt, 3B, Alternativa, 1).
2. Marca entrambe con lo stesso `parallel_group_id` (id arbitrario; serve solo a legarle insieme nel solver).

Cosa fa il solver: le ore dei due docenti sono legate slot-per-slot (`slot[ProfRel, h] == slot[ProfAlt, h]`); a livello di occupazione classe, le due lezioni condividono la stessa chiave di busy, per cui la 3B risulta occupata UNA sola volta in quello slot, anche se sono presenti due docenti.

5.11 Gruppi inter-classe: seconda lingua, recupero

Caso più complesso, introdotto dal Task C3: un gruppo che attraversa più classi. Esempio: il gruppo di Spagnolo è formato da 5 studenti della 2A e da 7 studenti della 2B, si riunisce 3 ore alla settimana, ed è tenuto da ProfSpa. Quando il gruppo si riunisce, le due classi-madre 2A e 2B sono "occupate" (i loro studenti sono nel gruppo, non possono fare lezione in classe), anche se la lezione non è formalmente una loro lezione.

Configurazione:

1. Tab /students: assicurarsi che gli studenti del gruppo abbiano correttamente la classe-madre 2A o 2B.
2. Tab /groups: nuovo StudyGroup di nome "Spagnolo 2A+2B" con `kind=language`. Aggiungere i 12 studenti come membri e dichiarare le ore-materia con (`Spagnolo, hours_per_week=3`).
3. Tab /assignments: nuova cattedra, selezionando il radio button **Target = gruppo (StudyGroup)**, scegliere "Spagnolo 2A+2B" dal dropdown, docente "ProfSpa", materia "Spagnolo", ore "3". La cattedra appare in una sezione dedicata di colore ciano, con badge **GRP**, separata sia dalle cattedre delle classi che dalla sezione Potenzamento.

Cosa fa il solver: per ogni ora in cui il gruppo Spagnolo si riunisce, sia 2A che 2B vengono marcate come occupate. L'invariante somma di `subj_busy == pr` del Phase B garantisce che, quando il gruppo è attivo in uno slot, la classe-madre non possa avere lezioni regolari nello stesso slot. Il vincolo per-day capacity in Phase A (`cl_day_load + sum(group_day_count) <= 6`) previene il caso degenerare in cui curriculum e gruppo sommerebbero più di sei ore in un solo giorno.

5.11.1 Compresenza nel gruppo

Se il gruppo Spagnolo prevede due docenti (es. ProfSpa1 + ProfSpa2 in compresenza), si configura un CoteachGroup con `group_id` valorizzato (XOR rispetto a `class_id`) e si crea una seconda cattedra con `coteach_group_id` sul gruppo. Il modello CP-SAT applica la stessa logica principal/codoc che già usa per le compresenze sulle classi.

5.11.2 Sostegno nel gruppo

Se uno studente DVA appartiene al gruppo Tedesco (3A+3B), il prof di sostegno deve seguire lo studente **nel gruppo** (non in classe-madre, perché in quei tre slot lo studente non è nella sua classe). Si configura una cattedra (ProfSost, `group_id=tedesco`, `sostegno`, `ore=3`, `is_support=True`). Il vincolo `slot[ProfSost, h] <= group_busy[h]` garantisce che il prof sia presente solo nei tre slot in cui il gruppo si riunisce, senza aggiungere ore di occupazione né alla classe-madre né al gruppo.

5.11.3 Pipeline e metaeuristiche con i gruppi

I gruppi inter-classe sono propagati a TUTTE le pipeline di ottimizzazione e a TUTTE le metaeuristiche. La tabella seguente riassume:

- **Phase B monolitica:** supporto nativo dei `group_slot`.
- **Decomposizione temporale:** thread-through completo, scheduling per giorno in parallelo.
- **Decomposizione spettrale, curriculum, METIS:** quando ci sono gruppi, il sistema usa un percorso “forced-monolithic” per giorno (mantiene la cache della distribuzione master ma salta gli stage A/B/C che non modellano i gruppi).
- **Column generation:** i pattern generati includono le ore-gruppo; il completion solver finale preserva `n_hours` e l'invariante classe-madre.
- **LNS, SA, TS, ILS, ALNS, VNS, Lagrangian:** le mosse atomiche e le riparazioni CP-SAT rispettano i vincoli di gruppo. Una mossa che violerebbe l'esclusione mutua gruppo/classe-madre viene rifiutata.

In pratica: l'utente può configurare i gruppi una volta e poi rilanciare qualunque pipeline o metaeuristica senza preoccuparsi di rompere il vincolo di gruppo. La suite di test del sistema include casi end-to-end per ognuna delle 8+ varianti.

5.12 Lock nativi: blindare lezioni già sistemate

Il pulsante a forma di lucchetto in /assignments (e l'analogo nel monitor delle lezioni) marca una cattedra come “locked”. Quando il solver viene rilanciato per ricalcolare l'orario, le lezioni di una cattedra lockata sono trattate come constraint nativi: il sistema non può spostarle da dove si trovano, deve risolvere intorno a esse.

Vantaggio rispetto al vecchio meccanismo “snapshot/restore”: il fail è immediato e parlante. Se i lock sono incompatibili con altri vincoli (es. il giorno libero del docente, il tetto giornaliero della classe, una compresenza HARD), il sistema rifiuta la run con un errore specifico, ad esempio:

Phase A INFEASIBLE: i lock attivi sono incompatibili con i vincoli correnti. Rimuovi o adatta i lock (es. sblocca eventi nel monitor) e ritenta. Lock attivi: 7 cattedra-giorno.

Senza lock nativi, lo stesso scenario sarebbe risultato in una run apparentemente riuscita, con i lock silenziosamente mossi a posteriori. I lock nativi sono propagati a tutte le pipeline (monolitica, decomposizione temporale, spettrale, curriculum, METIS, column generation) e a tutte le metaeuristiche (LNS, SA, TS, ILS, ALNS, VNS, Lagrangian).

5.13 Pre-flight checks: errori prima della run

Quando l'utente lancia un'ottimizzazione, il sistema esegue una serie di controlli sincroni **prima** di schedare la run. Le violazioni vengono restituite come HTTP 400 con un elenco specifico, in italiano. Categorie controllate:

- **Lock vs vincoli HARD:** lock che cadono nel giorno libero del docente, oltre il tetto giornaliero della classe, ecc.
- **Compresenze:** numero di ore del principal $\geq$ n_hours; numero di ore del codoc esattamente n_hours.
- **Sostegno:** deve avere una classe valida (oppure un gruppo valido nel caso C₃).
- **Potenziamento:** class_id deve essere NULL; somma delle ore non deve superare 30.
- **Gruppi (C₃):** XOR class_id/group_id; il gruppo deve avere almeno uno studente; ogni studente deve avere classe-madre; ore > 0.

L'utente vede un toast di errore con la lista delle anomalie. Può correggerle dai tab pertinenti (/assignments, /groups, /coteaching) e rilanciare.

5.14 Plessi: più sedi nello stesso istituto

In Italia molti istituti scolastici sono distribuiti su più sedi fisiche – la *Sede Centrale*, una o più *succursali* (per esempio *Succursale Via Garibaldi*), un *plesso* dedicato a un indirizzo specifico (per esempio *Plesso ITT* per l'indirizzo tecnico-tecnologico). Spostare un docente o una classe da una sede all'altra durante la giornata costa tempo reale e introduce vincoli che non si possono modellare con i soli vincoli H₁/H₂/H₃.

Il modulo **Plessi** (tab /plessi) gestisce:

- l'**anagrafica delle sedi** (nome, codice, indirizzo);
- l'**associazione aula** → **plesso** (ogni aula appartiene a una sola sede);
- due tipi di regole comportamentali: **commuting rules** (gap minimo per spostarsi tra due sedi) e **entity policies** (vincoli globali su quante sedi può frequentare un docente o una classe).

5.14.1 Commuting rules: il gap fra due plessi

Una *commuting rule* si applica a una coppia ordinata (plesso__origine, plesso__destinazione) e dice quante ore minime devono separare due lezioni consecutive del docente quando l'aula cambia plesso. Campi rilevanti:

- from_plesso_id, to_plesso_id: la coppia di sedi.
- entity_kind: a chi si applica (teacher, class, group).
- entity_id: opzionale – se NULL la regola è *kind-wide* (vale per tutti i docenti / tutte le classi / tutti i gruppi); se valorizzato vale solo per quel docente / classe / gruppo specifico.
- min_gap_hours: numero di ore di *buco* minimo richiesto fra l'ultima lezione nel plesso A e la prima lezione nel plesso B.
- symmetric: se true la regola vale anche nel verso opposto (B→A).

- *priority*: chi vince in caso di più regole sovrapposte.

Esempio italiano tipico: tra *Sede Centrale* e *Succursale Via Garibaldi* servono almeno 60 minuti per spostarsi. La regola corrispondente è (*Centrale*, *Garibaldi*, *kind*=teacher, *entity_id*=NULL, *min_gap_hours*=1, *symmetric*=true): nessun docente può avere lezione in Garibaldi alle ore 9 se ne ha una in Centrale alle ore 8.

La risoluzione è sempre *specifico vince su generale*: se esiste una regola con *entity_id*=42 per il docente Rossi, quella prevale sulla regola *kind-wide*. A parità di specificità vince la *priority* più alta.

5.14.2 Entity policies: vincoli globali sulla persona

Una *entity policy* è un vincolo *globale* (non legato a una coppia di plessi) sul comportamento di un docente o di una classe. Tre policy sono supportate:

- *any* – nessun vincolo globale, valgono solo le *commuting rules*. È il default.
- *single_plesso_per_day* – nell’arco di una giornata l’entità può avere lezioni in al massimo un plesso. Più stringente delle *commuting rules*: anche con *gap* = 1 ora, il cambio di sede nello stesso giorno è proibito.
- *single_plesso_total* – l’entità frequenta *solo* il plesso *plesso_id* per tutta la settimana. Tutte le aule assegnate a quell’entità devono appartenere a quel plesso.

Esempi italiani concreti:

- *Prof Rossi non vuole spostarsi a metà giornata*: la segreteria crea una policy (*entity_kind*=teacher, *entity_id*=Rossi, *policy*= *single_plesso_per_day*). Lunedì tutte le ore di Rossi sono in Centrale; martedì tutte in Garibaldi; mai un giorno misto.
- *La classe 3A indirizzo tecnico frequenta solo il Plesso ITT*: policy (*entity_kind*=class, *entity_id*=3A, *policy*=*single_plesso_total*, *plesso_id*=ITT). Tutte le aule che ricevono lezioni della 3A devono essere nel Plesso ITT.
- *Tutti i docenti, di default, evitano cambi di sede infrasettimanali*: policy (*entity_kind*=teacher, *entity_id*=NULL, *policy*= *single_plesso_per_day*) – vale per tutti i docenti salvo quelli con *override* più specifico.

Anche per le *entity policies* vale *specifico vince su generale* (*override* per singolo docente/classe), poi *priority*.

5.14.3 Validazione pre-run

Prima di lanciare l’ottimizzazione il sistema esegue una batteria di controlli sui dati plessi (*validate_plessi_rules*). Se qualcosa è incoerente, la run viene rifiutata con un elenco di violazioni. Cose controllate:

- codice plesso vuoto o duplicato;
- *commuting rule* che riferisce un plesso inesistente;
- *commuting rule* con *min_gap_hours* negativo o > totale ore-giornata;
- *commuting rule* duplicata *kind-wide* (stessa coppia di plessi, stesso *kind*, *entity_id* NULL, definita due volte);
- *entity policy* *single_plesso_total* senza *plesso_id* o con plesso inesistente;
- *entity policy* con *policy* non riconosciuta;
- *override* per entità inesistente (docente o classe cancellato/non in anagrafica).

L’endpoint GET /api/plessi/validate permette all’interfaccia di mostrare un *badge* live “config plessi OK / N anomalie”.

5.14.4 Implementazione CP-SAT

A livello di solver, le commuting rules diventano vincoli *a coppie* sulle variabili di assegnamento aula:

$$\forall (h_a, h_b) \text{ adiacenti tali che } \text{aula}(t, d, h_a) \in P_A, \text{aula}(t, d, h_b) \in P_B : h_b - h_a \geq \min_gap_{P_A \rightarrow P_B}.$$

Le entity policies diventano vincoli su contatori: `single_plesso_per_day` è $\sum_P \mathbf{1}[\exists h : \text{aula}(t, d, h) \in P] \leq 1$ per ogni giorno d ; `single_plesso_total` fissa il dominio delle variabili aula al solo plesso target.

I costi sono *hard* per default. Il roadmap prevede una versione *soft* con penalty per consentire override sporadici nei casi di sovraccarico (es. supplenza emergenza), ma la versione attuale tratta tutti i vincoli plesso come HARD.

5.14.5 Plessi via DSL (Step 3a-3c)

Da Step 3a in avanti, le `PlessoCommutingRule` sono traducibili interamente in clausole DSL grazie a due novità del linguaggio (cf. cap. 20):

- il path `l.classroom.plesso` che, data una lezione, risolve l'aula assegnata e poi il plesso a cui l'aula appartiene;
- il predicato `consecutive(l1.slot, l2.slot)` che diventa vero quando le due lezioni cadono nello stesso giorno e differiscono di un'ora.

La regola legacy “Centrale → Garibaldi gap minimo 1 ora, docente Rossi” si scrive in DSL come:

```
forall l1 in lessons where
    l1.classroom.plesso == 1
    and l1.teacher == "Rossi":
    forall l2 in lessons where
        l2.classroom.plesso == 2
        and l2.teacher == "Rossi"
        and consecutive(l1.slot, l2.slot):
        false
```

La helper `plesso_commuting_rule_to_dsl(rule)` in `engine/dsl_translator.py` produce automaticamente le clausole DSL a partire dalla riga ORM. La suite di test verifica zero-drift: il path DSL produce le stesse coppie di slot proibite del path legacy `plessi_constraints.py`.

5.15 Pattern canonici e seed legacy via DSL (Step 3b-3e)

Step 3b-3e ha introdotto un *vocabolario* di pattern canonici riconosciuti direttamente dal compilatore DSL → CP-SAT. Ognuno di essi corrisponde a un vincolo HARD legacy che prima viveva solo come encoding hardcoded. Ora possono essere espressi in DSL e prodotti automaticamente dalla translation `seed__imply__cit__hardcoded(profs)`.

Pragmas accettate:

- `no_holes_class("3B")` – niente buchi nell'orario settimanale della 3B (legacy `add__class__no__holes`).
- `class_present_at_hour("3B", 11)` – se la 3B ha lezione, lo slot all'ora 11 deve essere occupato (HARD-3 / “uscita non prima delle 12”).
- `class_day_load_in("3B", 0, 4, 5, 6)` – la 3B in un giorno ha 0 ore (giorno libero) o tra 4 e 6 ore (HARD-1+HARD-2).

- `teacher_max_per_day("Rossi", 5)` – Rossi non supera 5 ore in un giorno.
- `cattedra_max_per_day("Rossi", "3B", "Matematica", 2)` – la cattedra (Rossi, 3B, Mate) ha al massimo 2 ore in un giorno.
- `subject_pair_must("3B", "Scienzemotorie")` – quando la 3B ha 2 ore di motoria nello stesso giorno, devono essere consecutive.
- `subject_pair_exists("3B", "Matematica")` – quando la 3B ha 2+ ore di mate nello stesso giorno, almeno una coppia consecutiva deve esistere.

Effetto concreto: l'utente avanzato può ora scrivere nell'editor del modal "+ Nuovo vincolo DSL" un'espressione come `no_holes_class("4A")` ottenendo lo stesso risultato del toggle `hard_no_holes` in `school_classes`. Il vantaggio del DSL è la componibilità con `forall`, `exists`, `count`: si può restringere a sotto-insiemi di classi, combinare con altri predicati, modificare al volo dall'UI senza editare il database.

6 Workflow di ottimizzazione

6.1 Schema delle fasi

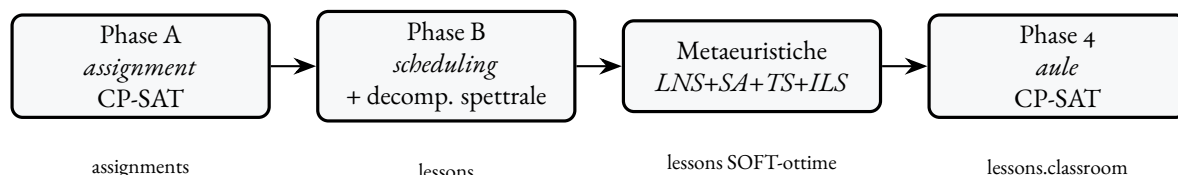


Figura 6.1 – Pipeline a 4 fasi.

6.2 Phase A: assegnazione (cattedre)

Modulo. `experiments/cpsat_v2_assignment.py`.

Lancio. `POST /api/optimize/assignment` con `{time_limit_s, workers, log}`.

Input. L'insieme di `(class, subject, hours_per_week)` letto da `class_subjects` e/o curriculum `subject_hours` + le abilitazioni docente in `teacher_subjects` + la classe-di-concorso preference in `subject_group_weights`.

Output. La tabella `assignments` (`teacher x class x subject` con `hours`). Problema di covering: ogni `(class, subject)` riceve un docente abilitato, rispettando `max-hours` per docente, classi-di-concorso e `teacher_compatible_classes`.

6.3 Phase B: scheduling

Modulo. `experiments/cpsat_v2_timetable.py` + `experiments/decomposition_spectral_v2.py`.

Decomposizione spettrale. Per istanze grandi, il problema CP-SAT integrale è intrattabile. La pipeline calcola un grafo di co-occupazione fra docenti, ne fa un'embedding spettrale via Laplaciano normalizzato e usa K-means su quell'embedding per partizionare in k cluster *loosely-coupled*. Ogni cluster diventa una sub-istanza CP-SAT indipendente; un secondo step "bridges" risolve solo i bordi; un terzo step "ricucitura" fa full-CPSAT con tutto il problema partendo dalla soluzione stitched.

Scope del solver CP-SAT (`cp_sat_scope`). Nella card 3 di /optimize il dropdown *Scope del solver CP-SAT* sceglie come la Phase B passa il problema al risolutore:

- **Per giorno** (day, default). Il solver risolve un giorno alla volta. La distribuzione settimanale delle ore di ciascuna cattedra (*day count*) viene precalcolata da Phase A e iniettata come uguaglianza rigida per ogni giorno. Veloce, scalabile fino a scuole grandi.
- **Settimana intera** (week). Il solver MonolithicSolver(`scope=None`) vede tutta la settimana in un solo modello CP-SAT. Più robusto sui vincoli che attraversano i giorni (compresenze, sostegno, gruppi a geometria variabile) ma significativamente più lento: i workers devono esplorare uno spazio di ricerca di sei volte la cardinalità del per-giorno.

Modalità Phase A (`phase_a_mode`). Phase A è il pre-step che decide quante ore di ciascuna cattedra cadono su ciascun giorno della settimana (*day count*). Il secondo dropdown della card 3 controlla come questo output viene combinato con il solver:

- **Sempre** (always, default per scope = Per giorno). Phase A esegue e il `day_count` viene imposto come uguaglianza rigida. Obbligatorio in modalità Per giorno – senza `day_count` il solver per-giorno non sa quante ore sono “previste” in quel giorno.
- **Salta** (skip, solo con scope = Settimana intera). Phase A non viene eseguita; è la Settimana intera a decidere autonomamente come distribuire le ore. Massima libertà per il solver, costo: nessun warm start.
- **Hint morbido** (`soft_hint`, solo con scope = Settimana intera). Phase A esegue come al solito ma il suo output viene iniettato nel solver tramite `model.AddHint` sui contatori per-giorno – il solver parte dalla distribuzione di Phase A come prima soluzione ammissibile e può deviare se trova qualcosa di meglio. Tipicamente il compromesso preferibile per scuole complesse: si tiene il warm start, si toglie la rigidità.

Vincolo incrociato (validato sia dal frontend sia dal backend): day richiede always; week esclude always. La UI disabilita le combinazioni invalide e ricorregge automaticamente `phase_a_mode` quando l’utente cambia `cp_sat_scope`.

Quando scegliere cosa.

- *Scuola standard, focus sulla velocità*: day + always. È il default per qualcosa: scala bene e produce soluzioni di buona qualità in tempo lineare nel numero di classi.
- *Scuola piccola con molte compresenze e sostegno*: week + skip. La rigidità del `day_count` di Phase A può rendere infeasibile un orario che invece esiste; lasciar decidere alla Settimana intera evita il problema.
- *Scuola grande ma con vincoli incrociati notevoli*: week + `soft_hint`. Il warm start velocizza la ricerca e la Settimana intera resta libera di adattare la distribuzione.

Branch-and-Price V1 e `cp_sat_scope=week`. Una domanda emersa durante lo sviluppo della Phase 3 era: se Phase A non fissa più il `day_count` come uguaglianza rigida (`phase_a_mode=skip` oppure `soft_hint`), la generazione di colonne V1 – master con vincoli di copertura indicizzati su (teacher, class, subject, day) – riesce a iterare di più della singola passata `no_improving_column` osservata in modalità day? L’intuizione era che un `dc_value` “allentato” generasse duali λ non-zero, sbloccando colonne migliorative.

La risposta empirica, misurata da `tests/benchmarks/run_bp_v1_scope_week.py` su due scenari sintetici (small = 10 docenti / 3 classi, medium = 4 docenti / 3 classi con cattedre dense), è *no*: le cinque granularità V1 (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject) terminano sempre alla prima iterazione con `bp_terminated_reason=no_improving_column` e `bp_columns_added_total=0`, sia quando il `dc_value` viene da Phase A sia quando viene derivato dalla soluzione del solver settimanale.

- **small:** `dc_phase_a` (21 chiavi non-zero) differisce strutturalmente da `dc_week` (16 chiavi non-zero) – 17 chiavi solo in *A*, 12 solo in *B*, 2 chiavi in comune con valori diversi – ma per ogni granularità entrambe le sorgenti producono `iters=1`, `cols=0`. La soluzione è HARD-feasible in entrambi i casi; il soft score finale è 240 con `phase_a` e 0 con `week`, ma la differenza viene dal seed pattern, non dalle iterazioni BP.
- **medium:** `dc_phase_a` (69 chiavi) vs `dc_week` (66 chiavi), 34 vs 31 chiavi disgiunte, 10 chiavi in comune con valori diversi. Stesso esito: `iters=1`, `cols=0` su tutte le granularità, sia con `phase_a` sia con `week`.

La spiegazione è strutturale, non un bug della Phase 3. I seed pattern di `column_generation`. `seed_patterns` sono greedy-ottimali rispetto al `dc_value` fornito: per ogni docente piazzano esattamente le sue ore nei giorni indicati, in ordine (`class`, `subject`, `day`, `hour`) crescente, evitando sovrapposizioni. Il master LP *V1* sceglie il pattern con peso maggiore – tipicamente il primo seed – e i duali λ del sub-problema di pricing risultano già saturati: il pricer ricostruisce la stessa griglia di ore e non trova *reduced cost* negativa. Cambiare la sorgente del `dc_value` modifica la *distribuzione* delle ore (giorni e quantità) ma non cambia il fatto che ciascun docente ha un’unica “forma” greedy-coerente con la propria domanda. Per sbloccare iterazioni multiple servirebbe (a) generare seed pattern *strutturalmente diversi* (perturbazioni randomiche delle ore o degli ordinamenti), (b) introdurre vincoli di accoppiamento cross-teacher già nel master (variante *V2* con cover indicizzato su (`class`, `day`) o (`curriculum`, `day`)), oppure (c) iniettare una pricer heuristic con duali *stabilizzati* che esplori intorno all’ottimo locale del seed. Sono tre direzioni di lavoro distinte e fuori dallo scope di Phase 3-5.

In pratica: `cp_sat_scope=week` sblocca il *solver settimanale* (la distribuzione delle ore non è più prerogativa di Phase A) ma NON sblocca il loop BP *V1*, che resta una single-shot ottimizzazione del seed catalog. Per orario di buona qualità su scuole grandi con BP, la leva attuale resta la granularità *V2* (`class`, `class-day`, `day`, `curriculum`) o l’iterative-diversified classico, non l’interazione fra scope settimanale e *V1*.

6.4 Metaeuristiche

Modulo. `experiments/metaheuristics.py`. Cascata di:

- **LNS** – destroy-and-repair su finestre (`teacher`, `day`) o (`class`, `day`); freeze del 60-70% delle lezioni e CP-SAT-resolve del resto.
- **SA** (Simulated Annealing) – single-swap accettati con Metropolis a T decrescente con fattore α .
- **TS** (Tabu Search) – best-improvement local search con tabu list (default size 80).
- **ILS** (Iterated Local Search) – alterna n cicli di TS con *kick* perturbativi (random 4-opt fra giorni).

Tutti contribuiscono al SOFT score: holes per teacher, isolated 1- and 5-hour days, weekly distribution, dual-hour pairs, no 6th hour, banded preferences su materie. Il SOFT include la contribuzione delle matrici di disponibilità SOFT/PREFERRED e dei logical SOFT/PREFERRED.

6.5 Phase 4: assegnazione aule

Modulo. `experiments/classroom_assignment.py`. Input: la tabella lessons (già temporalmente fissata) + le classroom rules (`kind`, `capacity`, `multi_class`, preferenze `materia` ↔ `aula`, `classe` ↔ `aula`, `docente` ↔ `aula`). Output: `lessons.classroom_name` popolato.

6.6 Drag-and-drop con preview live

Trascinando una lezione su un altro slot la UI fa due chiamate:

1. POST /api/schedule/move-preview – il backend simula la mossa per tutte le 36 (day, hour) e per ognuna restituisce:
 - ok con $\text{delta_soft} \leq 0$ – mossa accettabile (verde se $\text{delta} < 0$, verde chiaro se $\text{delta} = 0$).
 - soft_worse – mossa accettabile ma peggiora il SOFT (giallo).
 - hard_violation – vincolo HARD violato (rosso).
 - noop – slot di origine.
2. L'utente dropa, la UI invia PUT /api/schedule/move-lesson che valida e persiste.

6.6.1 Room-follows-lesson

Il move-preview NON considera l'aula nel calcolo HARD. Se la nuova posizione è compatibile con docente / classe ma l'aula attuale è già occupata o HARD-unavailable in destination, il backend:

1. Sposta la lezione.
2. Lascia l'aula libera sul nuovo slot solo se non c'è conflitto.
3. Se c'è conflitto, rimuove l'aula dalla lezione spostata e risponde con `room_cleared=true`, `cleared_room=<nome>`.
4. La UI apre un Modal che spiega cosa è successo e chiede di scegliere una nuova aula dal dropdown (verde libera / rosso occupata).

6.7 Slot picker matriciale (Monitor)

Nel tab *Monitor*, espandendo una cattedra si vedono le singole lezioni; cliccando il bottone Giorno/Ora si apre un modal con una matrice 6x6 dove ogni cella è colorata in base alla disponibilità:

- verde: free (docente e classe liberi)
- rosso: HARD (docente o classe già impegnati)
- ambra: aula occupata da un'altra lezione
- azzurro: slot attuale

Click su una cella libera fa un dry-run via PUT /api/monitor/event/{aid}/lesson/{lid}. Se ci sono conflitti, apre un secondo Modal con 3 opzioni: **Annulla**, **Disassegna le lezioni in conflitto**, **Disassegna e ottimizza dopo**.

6.8 Punteggio di graduatoria e criterio Anzianita

Ciascun docente nell'anagrafica ha un campo **graduatoria_score** (numerico). Il valore esprime il punteggio di graduatoria utile a discriminare i docenti quando si tratta di assegnare cattedre o supplenze: più alto = maggiore precedenza.

Il campo è usato in due punti:

1. **Modulo supplenze:** quando il sistema cerca un candidato per coprire un'assenza, ordina la rosa per `graduatoria_score` decrescente.
2. **Phase A, criterio "Anzianita":** il DSL della funzione obiettivo della Phase A include un preset **Anzianita** che, oltre ai criteri standard di bilancio (capacità non utilizzata, n. di indirizzi, ecc.), privilegia le assegnazioni che fanno coincidere docenti con alto `graduatoria_score` con classi del triennio finale (a parità di altre condizioni).

Il preset è selezionabile dalla card **Phase A (assegnazione)** del tab Workflow, dropdown "Criterio". Il DSL completo della funzione obiettivo è visibile nel modal "Mostra DSL attivo" della stessa card.

7 Guida UI

7.1 Funzionalità trasversali

7.1.1 Query DSL e sort multi-livello

Ogni pagina lista ha:

- barra Query (DSL: =, !=, <, <=, >, >=, contains, startswith, endswith, in [...], logica AND / OR, parentesi);
- sort multi-livello (max 4 livelli);
- pannello Help con la lista dei campi.

7.1.2 Multi-select

Sulle pagine Docenti / Classi / Aule la lista supporta:

- click semplice: single-select (replace);
- Ctrl+click: toggle;
- Shift+click: range;
- bottone "Vincolo collettivo" che apre il *BulkApplyModal*.

7.1.3 Excel/CSV import

Ogni pagina lista ha "Importa Excel/CSV" + "Template". Lo stesso endpoint POST `/api/import/{entity}` gestisce le 7 entità (teachers, subjects, classes, classrooms, curricula, students, groups). Modalità: upsert (default), append, replace. Header alias italiani / inglesi accettati.

7.1.4 Esportazione liste (xlsx / csv)

Sopra ogni tabella, accanto a "Reset query" / "Reset sort", trovi 3 bottoni di esportazione:

- **xlsx**: la *vista corrente* (solo le righe filtrate dalla query DSL, nell'ordine determinato dal sort multi-livello) come file .xlsx con header colorato e auto-fit colonne.
- **csv**: stesso contenuto, formato CSV UTF-8 con BOM (compatibile con Excel italiano).
- **tutto**: ignora query e sort, esporta tutte le righe dell'entità.

Filename: `<entita>_<YYYYMMDD>_<HHMMSS>.<ext>`. I parametri `?format=xlsx|csv` sono accettati da tutti gli endpoint di lista, quindi gli stessi export sono disponibili anche via curl.

7.1.5 Viste salvate

Sopra ogni tabella, accanto ai bottoni di Reset e Export, trovi un dropdown "Viste salvate..." e i bottoni "Salva vista" / "x":

- **Salva vista:** chiede un nome e memorizza la query DSL + il sort multi-livello come SavedView(entity, name, dsl_query, sort_levels). La vista è globale.
- **Dropdown "Viste salvate...":** applica una vista (sostituisce query + sort + ricarica la lista).
- **x:** elimina la vista correntemente selezionata.

Endpoint: /api/saved-views?entity=<entity> (GET / POST / PUT {id} / DELETE {id}). Tabella: saved_views.

7.1.6 Tag (aule e studenti)

Sistema many-to-many parallelo per aule e studenti.

Tag aule: tabella classroom_tags + join classroom_tag_assignments. UI: multi-select autocomplete nella scheda aula, bottone "Gestisci tag" per CRUD globale, colonna "Tag" con pill colorate (un colore stabile per nome). Il generatore mock tagga automaticamente le aule per kind (lab_fisica → [lab, fisica, scienze], palestra → [palestra, motorie], ecc.) e propaga il curriculum dalla home class (3B_scientifico → tag scientifico).

DSL: "matematica" in l.classroom.tags (vincoli) o has_tag(matematica) (query lista). I tag sconosciuti vengono creati al volo al salvataggio.

Tag studenti: tabella student_tags + join student_tag_assignments. Casi d'uso: BES, DSA, debito_matematica_4, pcto_ditta_<x>, studente_atleta. UI analoga a quella delle aule. Pannello "Precompila da tag" nel modal *Gruppi* che aggiunge in massa gli studenti che matchano any_of o all_of su una lista di tag.

Importante: i tag NON sostituiscono i gruppi. I gruppi restano l'unità operativa di scheduling, i tag sono metadato passivo per filtrare/raggruppare.

7.1.7 Import bulk dedicato (/import)

Pagina dedicata raggiungibile dalla nav. Layout a due colonne: configurazione (entità, modalità, "Scarica template") + file da importare (drop area + selettore file). Il template xlsx ha 3 fogli:

- Istruzioni: tabella Campo | Hint con descrizione di ogni colonna + note generali (sinonimi italiani, righe vuote ignorate, etc.);
- Esempi: una riga di esempio pre-popolata;
- Dati: foglio vuoto da compilare. L'importer legge per default questo foglio.

Campi importabili già integrati con le feature recenti: tags su students e classrooms, graduatoria_score / required_free_days_count / preferred_free_days_json su teachers, e max_hours_per_day su classes.

7.1.8 Import / Export DB (Dashboard)

Card prominente nella prima riga della Dashboard. Funzioni:

- **Esporta DB completo:** scarica un .zip contenente database.db (raw SQLite), tables/<t>.csv per ogni tabella e metadata.json (schema_version + sha256).
- **Solo schema:** omette i dati di scheduling – utile come template per inizializzare un'altra scuola.
- **Import:** selettore file .zip + conferma esplicita. Una copia timetable.db.pre_import_backup viene salvata prima dello swap.
- **Snapshot:** timestampati in webui/data/snapshots/, lista con bottoni Ripristina / Elimina.

Endpoint: /api/dashboard/export-db, /api/dashboard/import-db, /api/dashboard/snapshot/{create|list|restore/<f>}, DELETE /api/dashboard/snapshot/{f}.

7.1.9 Import / Export vincoli (Dashboard)

Seconda card della prima riga, indipendente dall'Import / Export DB: tocca solo le tabelle dei vincoli (logical, matrix, coteach, room preferences) e lascia inalterati docenti, classi, aule e cattedre. Concepita per due scenari distinti: caricare rapidamente un dataset di stress per provare il pannello di infeasibility, oppure importare un set di vincoli proveniente da un altro istituto o da un file Excel preparato a mano.

Le quattro funzioni della card:

- **Stress dataset:** dropdown con i sei profili (small, medium, big, huge, superhuge, mega) e bottone *Importa stress*. Carica i record da experiments/stress_constraints/<profilo>/, ne risolve gli owner_pattern (first_teacher, first_lab, gym, main_room, first_class_year_1, ...) contro le entità del DB attivo, e crea i vincoli passando per il dispatcher unificato POST /api/constraints. Il report che compare sotto la card riporta quanti record sono stati importati, suddivisi per categoria (teacher / classroom / relational), e quanti tra questi erano intentionally__conflicting: utile per verificare che il *feasibility-check* li identifichi nei core minimali corrispondenti.
- **File custom (JSON / xlsx):** drop area + bottone *Sfoglia*. Accetta una lista di record con campi kind, scope, owner_name (risolto per nome contro il DB), level, expression, soft_penalty opzionale, description opzionale. Lo stesso schema funziona come xlsx con intestazioni nella prima riga. I record con DSL invalido, owner_name non risolvibile o scope/kind mancanti vengono scartati e riportati nel report con indice e motivo dell'errore, senza interrompere l'import dei record validi.
- **Esporta JSON / xlsx:** dump di tutti i vincoli correnti nel formato accettato dall'import (round-trip pulito). È pensato come backup, come strumento di condivisione tra istituti che usano il sistema, e come modo per congelare uno stato di vincoli prima di una modifica massiva.
- **Cancella tutti i vincoli:** doppia conferma; svuota in un colpo solo logical_unavailabilities, curriculum_logical_constraints, le tre tabelle di matrix unavailability, coteaching_rules e le room preferences. Restituisce il conteggio per tabella. Utile per ricominciare da zero un esperimento senza dover passare dallo snapshot DB.

L'import accetta una scorciatoia di scrittura: un'espressione DSL con un giorno senza ora (es. mai sab, lun OR mar) viene espansa automaticamente nella forma esplicita che il parser richiede, cioè (sab8 OR sab9 OR sab10 OR sab11 OR sab12 OR sab13). La grammatica DSL principale resta così intatta, ma i dataset di stress (e i file custom degli utenti) possono essere scritti in maniera più leggibile.

Endpoint: /api/dashboard/constraints/import-stress, /api/dashboard/constraints/import-file, /api/dashboard/constraints/export?format=json|xlsx, DELETE /api/dashboard/constraints/all?confirm=true.

7.2 Tab per tab

Dashboard

Importa profilo (5 size) con 3 checkbox per i pool (curricula, aule, studenti); genera scuola di test; stato corrente con 7 card-numero; *RunLogPanel* streaming.

Docenti, Classi, Indirizzi, Studenti, Gruppi, Materie, Aule, Compresenze

CRUD entità con modal di edit. Vedere `ui__guide.md` per i campi specifici di ognuna.

Cattedre

Layout a card: una card per classe, lista materie+docente+ore. Bottone "cambia": apre modal con dropdown filtrato per materia, ogni opzione mostra Cognome Nome - assigned/max [SFORA: +Xh] / [pieno]. Bottone "Warnings cattedre": pannello con docenti SFORA / sotto / vuoti / ok.

Orario

4 viste (per classe, per docente, per aula, per slot), ognuna con toggle matrice/lista. Drag-and-drop con preview live, dropdown aule colorate (verde libera / rosso occupata).

Assenze e supplenze

Vista settimanale 6×6. Click su intestazione giorno: modal gestione assenze. Click su cella rossa: modal con classi scoperte (sinistra) + docenti disponibili (destra), drag-drop per assegnare supplenti.

Monitor

Lista *eventi* (uno per Assignment). Espansione lezione-per-lezione con bottone Giorno/Ora che apre slot picker matriciale (vedere §5.6). In cima alla pagina un *segmented control* con quattro tab per filtrare velocemente: **Tutti**, **Incompleti** (eventi con `n_assigned_hours < n_expected_hours`), **Senza orario** (eventi senza alcuna lezione piazzata) e **Lockati** (eventi con almeno una lezione marcata `lesson.locked == true`). Per ogni riga, le azioni disponibili sono *Dissocia*, *Blocca/Sblocca* e *Piazza* (eseguito singolarmente o in massa via multi-selezione + toolbar bulk).

Vincoli

Lista flat di tutti i vincoli editabili con pill colorate per livello. Bottone "Cerca conflitti": pannello con conflitti rilevati.

Workflow

Pulsanti per lanciare le 4 fasi separatamente o "Pipeline completa". SSE log streaming.

8 Launcher

8.1 Windows: start.bat

Apri due finestre cmd separate. Pre-flight:

- npm reachable (forza nodejs nel PATH);
- venv esiste in backend/.venv;
- backend/main.py e frontend/package.json presenti;
- se node_modules manca, lancia npm install;
- avvisa se 8000 / 5173 sono già occupate.

I path sono relativi a %~dp0, quindi lo script funziona ovunque venga messo.

8.2 Linux / macOS: start.sh + stop.sh

Default background: log in webui/logs/, PID in webui/logs/pids. Flag -f / -foreground per Ctrl+C cleanup. Auto-crea venv e fa npm install se mancano. Hint per apt, dnf, pacman, brew se python3 / npm non sono nel PATH.

stop.sh legge logs/pids, manda SIGTERM con fallback SIGKILL dopo 3s, killa anche i child via pkill -P e ha un fallback per ammazzare qualunque processo ancora in ascolto su 8000 / 5173 via lsof.

9 CP-SAT, il motore che cerca, deduce, ritratta

“A constraint solver is a kind of friend who can hold in his head all the rules of the game and tell you, calmly, whether the move you are considering is allowed.”

adattamento da Marriott e Stuckey, *Programming with Constraints* (Marriott e Stuckey 1998)

SE DOVESSIMO sintetizzare in una sola frase quale sia la tecnologia centrale di π Tantum, la risposta sarebbe: il *constraint solver* CP-SAT, sviluppato da Google come parte del pacchetto OR-Tools (Google 2010). Non si tratta di un dettaglio implementativo intercambiabile, ma di una scelta che modella in profondità il modo in cui il problema dell'orario viene formulato e risolto. Conviene quindi dedicare un capitolo intero a comprendere che cosa sia un constraint solver, come CP-SAT in particolare arrivi alle sue soluzioni, e per quali ragioni esso si presti meglio di altri strumenti al nostro problema.

Il capitolo è organizzato in modo progressivo. La prima sezione descrive il problema concreto che il solver è chiamato a risolvere e perché l'enumerazione cieca delle combinazioni è impraticabile. La seconda traccia una piccola storia delle idee da cui CP-SAT discende: il problema SAT puro degli anni Sessanta, l'algoritmo DPLL, la svolta del CDCL negli anni Novanta, l'introduzione delle variabili intere della constraint programming, e infine la fusione moderna che CP-SAT realizza. La terza presenta gli elementi di base della modellazione (variabili, vincoli, propagazione, ricerca) prima di mostrarne il funzionamento su un esempio in miniatura risolvibile a mano. La quarta generalizza al caso reale, descrive l'aggiunta della funzione obiettivo per passare da semplice fattibilità ad ottimizzazione, e discute in chiusura quando CP-SAT brilla, quando incontra i suoi limiti, e con quali altri metodi del sistema si combina.

9.1 Il problema concreto

Si immagini un dirigente scolastico nella prima settimana di settembre, con sul tavolo l'elenco delle classi (da venti a ottanta, a seconda della scuola), l'elenco dei docenti (da quaranta a centocinquanta), i monte ore di ogni materia in ogni indirizzo, le aule disponibili e una pila di richieste personali. Il dirigente deve produrre un orario settimanale in cui ogni lezione stia in uno slot compatibile, in cui nessun docente sia richiesto in due posti contemporaneamente, in cui nessuna classe abbia buchi inutili, e in cui ogni materia rispetti il monte ore previsto.

A occhio sembra che basti “provare”. Nella pratica il numero delle assegnazioni possibili è astronomico, come abbiamo già calcolato nel cap. 1. Per una scuola media di trenta classi con cinque ore al giorno per sei giorni si arrivano a circa novecento slot da riempire, scegliendo per ognuno una lezione fra centinaia di possibili, per un totale di circa 10^{2700} combinazioni. Nessun computer al mondo potrebbe esaminarle una per una in qualunque tempo finito. Serve dunque una macchina che, mentre prova, capisca quali combinazioni si possono escludere subito senza esaminarle. Quella macchina, per π Tantum, è CP-SAT.

Il nome CP-SAT significa *Constraint Programming over SAT*: il solver prende l'idea classica della programmazione a vincoli e la implementa sopra un solver SAT moderno, vale a dire un risolutore di formule booleane, sfruttando tutte le ottimizzazioni che la comunità SAT ha sviluppato negli ultimi vent'anni. Sviluppato da Google come parte del pacchetto OR-Tools, è oggi uno dei migliori solver disponibili per problemi combinatori di medie e grandi dimensioni, ed è gratuito.

Intuizione

In una sola frase, CP-SAT è un motore che, dato un elenco di variabili, di vincoli da rispettare e di un obiettivo da minimizzare, cerca un'assegnazione di valori alle variabili che rispetti tutti i vincoli e renda l'obiettivo il più piccolo possibile, senza enumerare le possibilità una a una.

9.2 Una storia in cinque tappe

Per comprendere appieno CP-SAT vale la pena ripercorrere brevemente cinque idee successive che, nel corso di sessanta anni, hanno costruito quello che oggi è lo stato dell'arte.

9.2.1 Il problema SAT puro

Negli anni Sessanta del Novecento, i logici matematici si chiedevano se, data una formula booleana, vale a dire un'espressione di variabili vere o false collegate dagli operatori di congiunzione, disgiunzione e negazione, esistesse un assegnamento di verità tale da rendere la formula vera. Questo è il problema SAT, formalmente classificato come NP-completo da Cook nel 1971 e da Karp nel 1972 (Garey e Johnson 1979). Una formula tipica si presenta in forma normale congiuntiva, vale a dire come congiunzione di clausole, dove ciascuna clausola è una disgiunzione di letterali (variabili o loro negazioni). Il solver deve trovare un assegnamento che renda vera ciascuna clausola simultaneamente.

9.2.2 DPLL: backtracking più propagazione unaria

Negli stessi anni Davis, Putnam, Logemann e Loveland proposero il primo algoritmo sistematico per affrontare il problema SAT, oggi conosciuto come DPLL dalle iniziali dei quattro autori. La sua logica è elementare: si sceglie una variabile, si prova ad assegnarle il valore vero, si propagano le conseguenze elementari (in particolare, la *unit propagation*: se in una clausola tutti i letterali tranne uno sono falsi, quello rimasto deve essere vero); se si raggiunge una contraddizione, si ritorna indietro e si prova il valore falso. Questo schema ricorsivo, denominato *backtracking*, è sopravvissuto fino ai giorni nostri come scheletro di tutti i solver SAT moderni.

9.2.3 CDCL: imparare dai propri errori

Negli anni Novanta del Novecento si è capito che il backtracking puro spreca enormi quantità di lavoro: il solver torna spesso a stati che contengono la stessa contraddizione di quelli appena abbandonati e ricomincia da capo. La svolta è stata l'introduzione del *conflict-driven clause learning*: quando il solver trova una contraddizione, analizza la sequenza di scelte che lo hanno portato lì e ne sintetizza una nuova clausola che proibisce esplicitamente quella combinazione fatale. La clausola viene aggiunta alla formula, e la prossima volta che il solver si avvicina a una situazione simile, la unit propagation lo blocca subito. Questa idea, implementata in solver come Chaff (Moskewicz et al. 2001) e tutti i suoi successori, ha portato a un salto di prestazioni di ordini di grandezza, trasformando il SAT moderno da un problema accademico a uno strumento industriale.

9.2.4 Constraint Programming: variabili intere e vincoli ad alto livello

In parallelo, negli stessi anni, la comunità di constraint programming sviluppava un linguaggio di modellazione molto diverso. Invece di lavorare soltanto con variabili booleane e clausole, la programmazione a vincoli mette al centro variabili intere su domini finiti (per esempio $x \in \{1, 2, \dots, 30\}$) e vincoli “ad alto livello” come $\text{AllDifferent}(x_1, \dots, x_n)$ o Cumulative per problemi di scheduling. Per ogni vincolo si scrivono algoritmi di propagazione specializzati che, dato il dominio corrente di ogni variabile, eliminano i valori che non possono più far parte di una soluzione. Lavorare con vincoli ad alto livello permette di esprimere in modo compatto strutture combinatorie che in SAT puro richiederebbero migliaia di clausole.

9.2.5 CP-SAT: la fusione

CP-SAT prende il meglio dei due mondi precedenti. Dal lato della constraint programming eredita le variabili intere e i vincoli globali, che lasciano il problema scrivibile in modo naturale. Dal lato di SAT eredita CDCL, unit propagation e tutte le ottimizzazioni ingegneristiche, fra cui le *decision heuristic* adattive, le *restart strategy*, e l'*in-processing*. Internamente, CP-SAT traduce le variabili intere e i vincoli ad alto livello in una rete equivalente di booleane, una tecnica nota come *lazy clause generation*. Ogni volta che il solver prende una decisione su una variabile intera, le booleane associate vengono propagate dal motore SAT; quando il motore SAT scopre una contraddizione, la nuova clausola appresa si traduce a sua volta in vincoli sulla rete delle variabili intere. È una danza continua fra i due livelli che si potenzia reciprocamente.

Nota storica

La famiglia DPLL ha origine nel 1962 con il celebre articolo di Davis, Putnam, Logemann e Loveland. Il CDCL nasce nel 1996-1997 con il solver GRASP di Marques-Silva e Sakallah. Chaff, del 2001, introduce le strutture dati moderne (*two-watched literals*) che hanno reso i solver SAT veramente efficienti. CP-SAT come prodotto Google nasce intorno al 2010 con OR-Tools, e ha vinto le edizioni 2018, 2019, 2020 e 2021 della MiniZinc Challenge, la principale competizione internazionale fra constraint solver. Per il lettore curioso del contesto storico, Rossi et al. 2006 è il manuale di riferimento sulla programmazione a vincoli e Russell e Norvig 2020 contiene un capitolo introduttivo accessibile.

9.3 Gli elementi della modellazione

Una volta capito il quadro storico, conviene introdurre gli elementi di base della modellazione in CP-SAT, uno alla volta, prima di vederli all'opera in un esempio concreto.

9.3.1 Variabili

Una variabile in CP è un'incognita che può assumere uno fra i valori del proprio dominio, sempre finito. Nel nostro contesto, una variabile può rappresentare lo slot settimanale in cui collocare una specifica lezione (con dominio composto dai numeri da uno a trentasei se vi sono sei giorni per sei ore al giorno), oppure l'aula assegnata a una certa lezione (con dominio composto dagli identificativi delle aule compatibili), oppure ancora una variabile booleana che indica se un certo docente sarà presente in un certo slot. Le variabili sono il vocabolario primario con cui si esprime il problema.

9.3.2 Vincoli

Un vincolo è una relazione fra una o più variabili che limita le combinazioni di valori ammissibili. I vincoli si distinguono in tre famiglie principali. Quelli aritmetici, del tipo $x + y \leq 10$ o $x \cdot y = z$, rappresentano relazioni numeriche dirette. Quelli di disuguaglianza, del tipo $x \neq y$, vietano specifiche combinazioni. Quelli globali, come $\text{AllDifferent}(x_1, \dots, x_n)$, impongono che i valori assunti da un gruppo di variabili siano tutti distinti; sono equivalenti a $\binom{n}{2}$ vincoli binari di disuguaglianza, ma il propagatore globale sfrutta una conoscenza più ricca della struttura del vincolo (in particolare, il teorema di Hall del cap. 12) per ottenere propagazioni molto più efficaci.

9.3.3 Propagazione

La propagazione è l'applicazione di regole locali che, data una scelta o una restrizione su una variabile, restringono il dominio di altre variabili senza che si debba fare alcun tentativo. Si tratta di pura deduzione automatica. La propagazione è l'arma principale del solver: ogni volta che esso fa una scelta esplorativa, la propagazione lo aiuta a capire automaticamente quali conseguenze quella scelta porta con sé, restringendo lo spazio delle scelte rimanenti e accelerando enormemente la ricerca.

Esempio

Si considerino tre variabili x, y e z con dominio $\{1, 2, 3\}$ e tre vincoli: $x \neq y, y \neq z, z \neq x$. Si supponga di aver fissato $x = 1$. La propagazione fa quanto segue. Dal vincolo $x \neq y$ si deduce che y non può valere 1, e il dominio di y si riduce a $\{2, 3\}$. Dal vincolo $z \neq x$ si deduce analogamente che il dominio di z si riduce a $\{2, 3\}$. A questo punto, se aggiungiamo $y = 2$, la propagazione continua: dal vincolo $y \neq z$ si deduce che z non può valere 2, e il dominio di z si restringe a $\{3\}$. Il valore di z è forzato senza alcun tentativo: il solver lo deduce dalla logica dei vincoli.

9.3.4 Ricerca

La ricerca è la fase in cui il solver, esaurita la propagazione, sceglie a caso una variabile non ancora fissata e un valore del suo dominio, lo prova, e ricomincia a propagare. Se la propagazione raggiunge una contraddizione, ovvero un dominio diventato vuoto per qualche variabile, la ricerca esegue un *backtracking*: torna indietro, elimina quel valore dal dominio della variabile scelta, e prova un altro. La sequenza “decisione, propagazione, eventuale backtracking” è il cuore del solver, e si ripete migliaia o milioni di volte fino a trovare una soluzione completa o a dimostrare che essa non esiste.

La differenza fra un solver lento e uno veloce sta in tre aspetti: la scelta di quale variabile esplorare per prima (le *decision heuristic*), la scelta di quale valore provare per primo (le *value selection heuristic*), e la quantità di apprendimento che il solver estrae da ciascun conflitto (le *clause learning strategies*). Le strategie comuni includono il *smallest domain first*, che esplora per prima la variabile con il dominio più piccolo, e il VSIDS, che privilegia le variabili che sono apparse più frequentemente nei conflitti recenti.

9.3.5 Il ciclo CP-SAT

L'integrazione di tutti questi elementi produce il ciclo illustrato nella figura 9.1. Il solver parte da un modello iniziale, propaga, decide, propaga ancora, e così via finché tutte le variabili sono fissate (e si ha una soluzione) oppure finché si raggiunge una contraddizione (e si esegue backtracking, imparando una clausola che evita di ripetere lo stesso errore in futuro). Il punto cruciale è che ogni conflitto non è tempo sprecato, perché produce una clausola appresa che diventa parte permanente del modello e accelera tutte le ricerche successive.

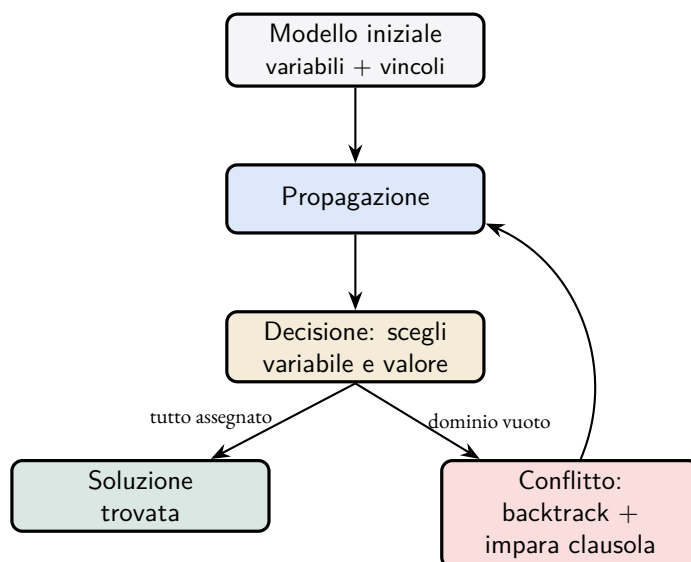


Figura 9.1 – Il ciclo di funzionamento di CP-SAT. Dopo aver propagato il modello iniziale, il solver alterna decisioni e propagazioni; ogni conflitto produce una clausola appresa che evita di ripetere lo stesso errore.

9.4 Un esempio in miniatura

Per vedere CP-SAT all'opera senza dover ricorrere a una scuola reale, prendiamo un orario miniatura con tre docenti (Adami, Bruni, Conti), due classi (1A e 1B), e quattro slot disponibili: lunedì alla prima ora, lunedì alla seconda, martedì alla prima, martedì alla seconda. Le materie e i monte ore sono i seguenti: la 1A ha matematica per due ore, italiano per un'ora e storia per un'ora; la 1B ha matematica per un'ora, italiano per due ore e storia per un'ora. Le assegnazioni docente-materia sono già state decise dalla fase di Phase A: Adami insegna matematica in 1A per due ore e in 1B per un'ora, Bruni insegna italiano in 1A per un'ora e in 1B per due ore, Conti insegna storia in 1A per un'ora e in 1B per un'ora. C'è un vincolo aggiuntivo: Bruni non lavora il lunedì alla prima ora, per indisponibilità HARD.

9.4.1 La modellazione

Per ogni combinazione di classe, materia e ora richiesta introduciamo una variabile intera che indica in quale slot quella lezione viene collocata. Per esempio, $S_{1A, \text{mate}, i} \in \{1, 2, 3, 4\}$ indica in quale dei quattro slot va la prima ora di matematica della 1A. In totale ci sono otto variabili: due per le due ore di matematica della 1A, una per l'ora di italiano della 1A, una per l'ora di storia della 1A, una per l'ora di matematica della 1B, due per le due ore di italiano della 1B, una per l'ora di storia della 1B.

I vincoli HARD sono di tre tipi. Il primo è che una classe non può avere due lezioni nello stesso slot, formalizzato con il vincolo globale AllDifferent sui quattro slot di ciascuna classe. Il secondo è che un docente non può tenere due lezioni nello stesso slot: per ogni coppia di lezioni del medesimo docente si impone $S_a \neq S_b$. Il terzo è che Bruni non lavora lunedì alla prima ora: i due slot dell'italiano in 1B e l'unico slot dell'italiano in 1A non possono valere 1.

9.4.2 La risoluzione passo per passo

Il solver inizia con la propagazione del vincolo sull'indisponibilità di Bruni. Tre variabili perdono immediatamente il valore 1 dal proprio dominio: l'italiano della 1A passa a $\{2, 3, 4\}$, e analogamente le due ore di italiano della 1B. Nessuna scelta esplorativa è stata ancora fatta, e già tre domini si sono ridotti.

A questo punto il solver applica la propria euristica di scelta, tipicamente *smallest domain first*, e seleziona una delle variabili con dominio più piccolo. Le tre variabili dell'italiano hanno tutte dominio di tre elementi, mentre le altre cinque ne hanno quattro. Il solver sceglie quindi una di esse, per esempio l'italiano in 1A, e prova a fissarlo al primo valore disponibile, che è 2 (lunedì alla seconda ora).

La propagazione successiva è impressionante. Il vincolo AllDifferent sulla 1A rimuove 2 dal dominio delle altre tre variabili della 1A. Il vincolo che impone a Bruni di non essere in due slot contemporaneamente rimuove 2 dalle due variabili dell'italiano della 1B. Quelle due variabili hanno ora dominio $\{3, 4\}$ e devono essere distinte fra loro per il vincolo AllDifferent sulla 1B; di conseguenza, la propagazione le forza implicitamente a essere $\{3, 4\}$ in qualche ordine. In altre parole, una sola decisione esplicita ha fissato (o quasi-fissato) cinque variabili.

Il solver continua applicando le sue scelte successive, e in pochi passi tutte le otto variabili sono fissate. Se in qualche momento si imbatte in una contraddizione, il backtracking è immediato e il solver impara da quella contraddizione una clausola che evita di ripetere lo stesso errore. Per questo esempio in miniatura, il solver trova soluzione in pochi millisecondi senza neppure dover imparare clausole. Il punto da sottolineare è che il numero effettivo di tentativi è drasticamente ridotto dalla propagazione: invece di esaminare 4^8 , ovvero 65 536 combinazioni, il solver ne esamina meno di una ventina.

Esempio

Una soluzione possibile per l'esempio descritto è mostrata nella tabella seguente, che riporta per ogni slot la classe, la materia e il docente.

	Lun-1	Lun-2	Mar-1	Mar-2
1A	mate (Adami)	ita (Bruni)	sto (Conti)	mate (Adami)
1B	sto (Conti)	mate (Adami)	ita (Bruni)	ita (Bruni)

Si verifica facilmente che ogni classe ha quattro lezioni distinte, che Adami non sta mai in due posti contemporanei (è in lunedì prima, lunedì seconda e martedì seconda), che Bruni non insegna lunedì prima, e che Conti insegna lunedì prima e martedì prima senza sovrapposizioni. Tutti i vincoli sono soddisfatti, e questa è dunque una soluzione valida.

9.5 Generalizzazione alla scuola reale

In una scuola con trenta classi e sessanta docenti la stessa logica funziona, con due differenze importanti rispetto all'esempio in miniatura. La prima è che la propagazione fa una parte molto maggiore del lavoro. I vincoli AllDifferent, le indisponibilità, le preferenze rigide e l'occupazione dei laboratori condivisi creano una rete molto fitta di deduzioni: una scelta su una classe del triennio spesso costringe decine di altri slot in modo automatico, riducendo lo spazio della ricerca a una frazione minuscola dell'iniziale.

La seconda differenza è che la ricerca diventa interessante quando i vincoli HARD sono “quasi soddisfatti”. Quando il solver ha collocato il settanta per cento delle lezioni e si trova davanti alle ultime, qualunque scelta porta spesso a una contraddizione e il backtracking si scatena con intensità. È qui che le clausole apprese fanno la differenza: dopo qualche centinaio di conflitti il solver “capisce” che certe combinazioni non funzionano e smette di provarle.

9.5.1 Dall'ottica di soddisfacibilità all'ottimizzazione

CP-SAT non si limita a trovare una soluzione qualsiasi: può ottimizzare. Si dichiara una variabile aggiuntiva, denominata convenzionalmente *obj*, definita come somma pesata dei termini “brutti” (le ore isolate dei docenti, la sesta ora delle classi, le lezioni in slot non preferiti, e così via), e si chiede al solver di minimizzarla. Internamente, CP-SAT alterna ricerca della soluzione e ricerca di una migliore: ogni volta che ne trova una, aggiunge il vincolo $obj < best$ e ricomincia. Il processo termina quando il solver dimostra che non esiste soluzione con valore migliore (ottimo certificato) oppure quando il time limit viene esaurito (ottimo non garantito ma con la migliore soluzione trovata fino a quel momento).

In π Tantum la funzione obiettivo è descritta in dettaglio nel cap. 6; ed è componibile attraverso il DSL del sistema, descritto nel cap. 17.

9.6 Quando CP-SAT brilla, quando incontra i propri limiti

CP-SAT brilla quando il problema è combinatorio puro, ovvero formulabile con variabili intere e vincoli logici, quando i vincoli sono molto interconnessi (perché in tal caso la propagazione ha materiale su cui lavorare), e quando il numero di variabili è nell'ordine fra mille e centomila. Sopra questa soglia, anche CP-SAT diventa lento, ed è proprio uno dei motivi per cui π Tantum decompone le scuole più grandi attraverso la spettrale del cap. 10, che spezza il problema in sottoproblemi più piccoli risolvibili da CP-SAT in tempi ragionevoli.

CP-SAT incontra i propri limiti in tre situazioni distintive. La prima è quella dei problemi continui, con variabili reali: per essi servono solver LP, MIP o SOCP, di natura completamente diversa. La seconda è quella dei problemi con obiettivo composto da molti termini con pesi fini: la ricerca dell'ottimo esatto può impiegare molto tempo, e di solito conviene fermarla con un `time_limit` e raffinare con metaeuristiche. La terza è quella dei problemi che superano il milione di variabili booleane equivalenti: anche CP-SAT si arrende, e serve la decomposizione in parti più piccole.

Attenzione

Nella modellazione di un problema reale, vale una regola pratica fondamentale: usare i vincoli globali quando si può. Ogni vincolo globale (AllDifferent, Cumulative, Circuit) ha un propagatore specializzato che lavora molto meglio di una sequenza di vincoli binari equivalenti. Modellare con vincoli ridondanti, o usare variabili intere quando bastano variabili booleane, può rallentare la risoluzione di ordini di grandezza senza che ci sia alcun beneficio compensativo. La modellazione è un'arte, non una traduzione meccanica del problema, e investirvi tempo ripaga sempre.

9.7 Le connessioni con il resto del sistema

CP-SAT è il cuore di π Tantum, e tutto il resto del sistema vi gravita attorno. La decomposizione spettrale del cap. 10 prepara il terreno per CP-SAT spezzando le scuole grandi in cluster trattabili. Le metaeuristiche del cap. 11 usano CP-SAT come motore di riparazione locale, in particolare nella variante ALNS dove il *repair operator* `cp_sat_window` re-risolve sotto-finestre della soluzione. Il pre-controllo di Hall del cap. 12 verifica preventivamente che CP-SAT possa trovare una soluzione, evitando di lanciarlo su istanze infeasibili. Il rilassamento lagrangiano del cap. 13, infine, opera trasformando alcuni vincoli HARD in penalità sull'obiettivo, riducendo il problema CP-SAT a sotto-problemi più piccoli e indipendenti che il solver può affrontare in parallelo.

Per approfondire

Per chi voglia approfondire i contenuti di questo capitolo in modo sistematico, segnalo i seguenti riferimenti. Russell e Norvig, *Artificial Intelligence: A Modern Approach* (Russell e Norvig 2020), dedicano i capitoli sei e sette ai CSP con una presentazione divulgativa; Rossi, Van Beek e Walsh, *Handbook of Constraint Programming* (Rossi et al. 2006), sono il riferimento tecnico canonico, con i capitoli tre, quattro e sei particolarmente rilevanti per la modellazione, la consistenza e la ricerca; Marriott e Stuckey, *Programming with Constraints* (Marriott e Stuckey 1998), forniscono una buona introduzione operativa al pensiero per vincoli; per la storia di CDCL e il SAT moderno, il *Handbook of Satisfiability* edito da Biere, Heule, van Maaren e Walsh è il volume di riferimento. La documentazione tecnica di OR-Tools, accessibile su https://developers.google.com/optimization/cp/cp_solver, contiene molti esempi pratici in Python.

10 La decomposizione spettrale, spezzare per regnare

“The eigenvectors of the Laplacian know more about the graph than you might think.”

Fan R. K. Chung, *Spectral Graph Theory* (Chung 1997)

SI IMMAGINI di dover organizzare l'orario di un istituto comprensivo enorme, formato da novanta classi distribuite su dodici indirizzi diversi e seguito da duecento docenti. A prima vista l'impresa sembra formidabile, da affrontare necessariamente come problema monolitico. Eppure, se si osserva con attenzione il grafo che ha per nodi le classi e i docenti e per archi le relazioni di insegnamento, si nota qualcosa di interessante: il grafo non è omogeneo. Esistono *cluster naturali*, sottoinsiemi di nodi più densamente connessi al loro interno che non con il resto. Le classi del liceo classico hanno docenti che lavorano quasi esclusivamente con il classico; le classi dell'ITIS hanno docenti che ruotano quasi solo dentro l'ITIS; i docenti di matematica si distribuiscono fra indirizzi affini ma raramente toccano l'IPSIA accanto.

Se fossimo in grado di vedere questi cluster e di risolvere l'orario di ognuno separatamente, il problema cambierebbe faccia. Invece di una scuola da novanta classi avremmo cinque o sette sotto-scuole da dodici-diciotto classi ciascuna, ognuna trattabile da CP-SAT in pochi minuti. Questa è la promessa della decomposizione spettrale: usare l'algebra del grafo bipartito per individuare i cluster naturali e risolvere il problema in parti più piccole.

10.1 Da dove viene l'idea

Nota storica

Nel 1973 il matematico ceco Miroslav Fiedler (Fiedler 1973) pubblicò un risultato che all'epoca poteva sembrare puramente teorico: il secondo autovettore del Laplaciano di un grafo, da allora detto *vettore di Fiedler*, ne rivela la struttura comunitaria. Quasi vent'anni dopo Shi e Malik (Shi e Malik 2000) ripresero l'idea per la segmentazione di immagini, dove un'immagine si modella come grafo i cui nodi sono i pixel e i cui archi pesano la somiglianza fra pixel adiacenti: calcolando il vettore di Fiedler si ottengono i contorni degli oggetti. Da quel momento il *spectral clustering* (Luxburg 2007) è diventato uno strumento standard del machine learning, applicato a domini fra loro lontanissimi.

L'idea che si nasconde nel risultato di Fiedler è che la struttura combinatoria di un grafo, vale a dire la descrizione di chi sia connesso con chi, si riflette nello spettro (vale a dire negli autovalori) di una matrice associata, e che lo spettro a sua volta è calcolabile in tempo polinomiale standard con i metodi dell'algebra lineare numerica. Quello che era difficile da identificare in modo combinatorio (i cluster naturali) diventa relativamente facile da estrarre per via algebrica.

10.2 Il grafo bipartito classe-docente

Per applicare la decomposizione spettrale al nostro problema costruiamo un grafo bipartito che ha per nodi le classi e i docenti della scuola. Per ogni coppia (classe, docente) c'è un arco, eventualmente pesato, se il docente insegna in quella classe; il peso può essere il numero di ore settimanali di insegnamento. Le classi formano

una parte del grafo, i docenti l'altra, e gli archi attraversano esclusivamente la separazione fra le due parti, da cui il nome *bipartito*. La struttura di clusterizzazione del grafo riflette direttamente la struttura organizzativa della scuola: indirizzi distinti producono cluster ben separati; docenti che insegnano in più indirizzi formano i ponti fra i cluster.

Un esempio in miniatura aiuta a fissare le idee. Si consideri una scuola con tre classi e quattro docenti, in cui i primi due docenti insegnano soltanto nelle prime due classi e gli altri due soltanto nella terza. Il grafo bipartito che ne risulta ha due cluster evidenti, perfettamente separati: $\{c_1, c_2, d_1, d_2\}$ e $\{c_3, d_3, d_4\}$. Non vi sono archi che attraversano la frontiera fra i due cluster.

10.3 Il Laplaciano e il vettore di Fiedler

L'oggetto matematico centrale per identificare i cluster è il *Laplaciano* normalizzato del grafo. Per costruirlo partiamo dalla matrice di adiacenza A , che ha valore A_{ij} pari al peso dell'arco fra i e j (zero se l'arco non esiste), e dalla matrice diagonale D dei gradi, in cui D_{ii} è la somma dei pesi degli archi incidenti sul nodo i . Il Laplaciano normalizzato si scrive:

$$L = I - D^{-1/2} A D^{-1/2}$$

Per leggere questa formula si può procedere così. Si prende la matrice di adiacenza, che descrive chi è connesso con chi; la si normalizza dividendo ogni elemento per la radice del prodotto dei gradi dei due estremi, in modo da togliere l'effetto della diversa importanza dei nodi; e infine si sottrae il risultato dall'identità. Il risultato è una matrice simmetrica, semi-definita positiva, i cui autovalori sono compresi nell'intervallo $[0, 2]$.

Le proprietà importanti del Laplaciano normalizzato per i nostri scopi sono tre. Il primo autovalore λ_0 vale sempre zero, e l'autovettore associato è proporzionale a $D^{1/2} \mathbf{1}$ (un vettore banale che non porta informazione strutturale). Il secondo autovalore λ_1 , che la letteratura chiama *algebraic connectivity*, è piccolo quando il grafo ha cluster ben separati e grande quando è ben connesso. Il vettore associato a λ_1 è il celebre *vettore di Fiedler*: ogni componente di questo vettore corrisponde a un nodo del grafo, e ordinando i nodi per il valore della loro componente si ottiene un ordinamento naturale che separa le comunità.

L'idea cruciale è che, una volta calcolato il vettore di Fiedler, basta leggerne il segno (o, equivalentemente, applicarvi un semplice algoritmo *k-means*) per ottenere la suddivisione del grafo in cluster. Per un'analisi che produca k cluster (e non soltanto due), si calcolano i primi k autovettori non banali $v_1, v_2, \dots, v_k$ del Laplaciano: si ottiene così una matrice $|V| \times k$ in cui ogni nodo diventa un punto nello spazio $\mathbb{R}^k$. A questo punto *k-means* standard restituisce i cluster cercati.

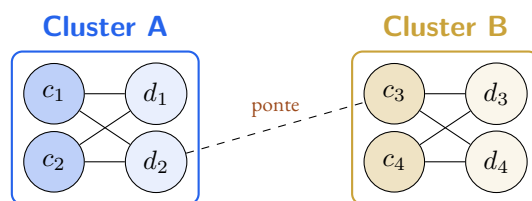


Figura 10.1 – Esempio di grafo bipartito con due cluster naturali e un “ponte” debole. Il vettore di Fiedler assegna componenti di segno opposto ai due cluster.

10.4 Un esempio in miniatura

Riprendiamo lo schema di figura 10.1 con quattro classi, quattro docenti e un ponte debole fra i due cluster. Numeriamo i nodi da uno a otto in ordine $c_1, c_2, c_3, c_4, d_1, d_2, d_3, d_4$, e supponiamo che ogni arco abbia peso unitario. La matrice di adiacenza diventa una matrice 8×8 con uni nelle posizioni corrispondenti agli archi e zeri altrove; i gradi sono rispettivamente $D = \text{diag}(2, 2, 3, 2, 2, 3, 2, 2)$, con i due nodi del ponte (d_2 e c_3) che hanno grado tre invece di due.

Il calcolo del Laplaciano normalizzato secondo la formula $L = I - D^{-1/2} A D^{-1/2}$ è tedioso da fare a mano e viene comunemente delegato a NumPy o a un'altra libreria di algebra lineare. Calcolando lo spettro con

`numpy.linalg.eigh(L)` si ottengono come autovalori approssimativamente $\lambda_0 = 0$, $\lambda_1 \approx 0.18$, $\lambda_2 \approx 0.72$ e così via. Il valore basso di λ_1 conferma quantitativamente che il grafo è debolmente connesso: i due cluster sono ben separati e il ponte d_2-c_3 è l'unico legame.

Il vettore di Fiedler, autovettore associato a λ_1 , risulta avere componenti positive per i primi quattro nodi (c_1, c_2, d_1, d_2 , ovvero il cluster A) e negative per gli altri quattro (c_3, c_4, d_3, d_4 , ovvero il cluster B). La separazione è netta. Tagliando il vettore in zero otteniamo esattamente i due cluster, esattamente quelli che ci aspettavamo guardando la figura.

Esempio

Per ottenere $k > 2$ cluster, si usano i primi k autovettori del Laplaciano e si applica k -means alla matrice risultante. Su questa stessa scuola, con $k = 2$, il k -means restituisce gli stessi due cluster ottenibili dal segno del solo vettore di Fiedler. Per scuole reali con quattro o cinque indirizzi, k si sceglie tipicamente fra quattro e otto sulla base di un'euristica chiamata *eigengap*: si cerca il valore di k tale che lo "salto" fra λ_k e λ_{k-1} sia significativo, perché questo indica che esistono effettivamente k cluster ben distinti.

10.5 La generalizzazione al caso reale

In una scuola reale di novanta classi e duecento docenti la logica è la stessa, soltanto che le matrici sono di dimensione 290×290 e gli autovettori si calcolano in qualche secondo con LAPACK o scipy. Le differenze pratiche rispetto all'esempio in miniatura sono di tre tipi.

In primo luogo, i cluster non sono così nettamente separati come nell'esempio giocattolo. Tipicamente esistono cluster ben definiti che corrispondono agli indirizzi principali, e poi nodi di confine che potrebbero appartenere a più cluster (per esempio i docenti di matematica che insegnano sia al liceo classico sia all'ITIS). Il vettore di Fiedler, e in generale i primi k autovettori, classificano comunque ogni nodo in modo deterministico, ma la qualità del clustering è intrinsecamente meno nitida.

In secondo luogo, il numero ottimale di cluster k non è dato a priori e va determinato. π Tantum esegue l'*eigengap heuristic* automaticamente, calcolando il salto fra autovalori successivi e suggerendo il valore di k che produce la separazione migliore. L'utente può sempre forzare un valore diverso se ha conoscenze specifiche della propria scuola che il metodo automatico non cattura (per esempio l'esistenza di una sede staccata che andrebbe trattata come cluster a sé).

In terzo luogo, una volta che i cluster sono stati identificati, resta il problema di gestire i *ponti*, ovvero le lezioni che coinvolgono nodi appartenenti a cluster diversi. Si procede tipicamente con un passo finale di *ricucitura* (in π Tantum questo passo si chiama *Phase B stitch*), che esegue una ottimizzazione CP-SAT globale ridotta soltanto ai ponti, mantenendo fisse le soluzioni già trovate per i singoli cluster. Per ponti poco numerosi questa fase è veloce; per ponti molti e intrecciati può diventare costosa quanto la risoluzione monolitica.

10.6 Quando il metodo funziona, quando no

Intuizione

La decomposizione spettrale dà il meglio di sé quando i cluster naturali esistono davvero, ovvero quando la scuola ha indirizzi distinti, classi di concorso disgiunte fra indirizzi diversi, oppure sedi distaccate. Il guadagno tipico in questi casi è di un fattore fra cinque e dieci rispetto a CP-SAT applicato monoliticamente, e cresce con la dimensione della scuola.

Attenzione

Quando il grafo è denso, ovvero quando i docenti insegnano un po' ovunque senza cluster naturali, il vettore di Fiedler non separa nulla di significativo e i cluster prodotti sono artificiali. La fase di ricucitura diventa allora più costosa della risoluzione monolitica, e il metodo perde tutto il suo vantaggio. π Tantum rileva automaticamente questa situazione guardando il valore di λ_1 del Laplaciano: se è troppo grande,

sopra una soglia tarata empiricamente intorno a 0.5, il sistema suggerisce di saltare la decomposizione e di lasciare che CP-SAT lavori sul problema intero.

I parametri principali su cui l'utente può intervenire sono pochi e quasi sempre lasciati ai valori di default. Il parametro `n_clusters` indica il numero di cluster da produrre; per default viene scelto via `eigengap`, e tipicamente si attesta fra quattro e otto per scuole medio-grandi. Il parametro `laplacian_type` può valere `normalized` (default, raccomandato per grafi con pesi non uniformi) oppure `unnormalized`. L'algoritmo di clustering è `k-means` inizializzato con `k-means++` (default, robusto) o con `random`. Il numero di iterazioni della fase di ricucitura, controllato da `stitch_passes`, è per default uno e raramente vale la pena aumentarlo.

10.7 Le connessioni con il resto del sistema

La decomposizione spettrale non vive isolata: è strettamente integrata con altri componenti della pipeline di π Tantum. CP-SAT (cap. 9) è chiamato in causa per risolvere ogni cluster come problema autonomo, e poi ancora per la fase di ricucitura globale. Il pre-controllo di Hall (cap. 12) viene eseguito non soltanto sull'intera scuola ma anche su ciascun cluster: se un cluster non passa Hall, il sistema deve rilassare i ponti per ridistribuire la domanda fra i cluster. Il rilassamento lagrangiano del cap. 13 fornisce un'alternativa alla ricucitura combinatoria: invece di risolvere i ponti come problema CP-SAT separato, dualizza i vincoli inter-cluster con moltiplicatori di Lagrange e itera. L'analisi del grafo bipartito del cap. 15, infine, fornisce all'utente delle misure quantitative (modularità, betweenness, densità) che permettono di giudicare in anticipo se la decomposizione spettrale funzionerà bene per la propria scuola.

10.8 Tre alternative alla decomposizione spettrale

La decomposizione spettrale non è l'unica via per spezzare il problema. Nel corso dello sviluppo di π Tantum sono state implementate altre tre strategie, che convivono con quella spettrale e si selezionano dal tab Workflow a seconda delle caratteristiche della scuola. Le tre alternative non si escludono a vicenda; al contrario, si combinano molto bene fra loro in pipeline a più stadi.

10.8.1 Decomposizione temporale

La decomposizione temporale spezza il problema lungo l'asse del tempo invece che sulle entità. Ciascuno dei sei giorni della settimana diventa un sotto-problema separato, risolvibile in parallelo su sei core distinti di una macchina moderna. Una piccola fase di pre-distribuzione, denominata Phase B-pre, decide quante ore di ciascuna cattedra vanno in ciascun giorno, in modo da rispettare i vincoli settimanali (la doppia ora di matematica, le motorie a coppie, il tetto di ore per classe per giorno). Una volta pre-distribuite le ore, sei istanze CP-SAT indipendenti, una per ciascun giorno, piazzano materialmente le ore nei sei slot del giorno.

Il vantaggio principale della decomposizione temporale è che essa è *sempre applicabile*: non richiede una struttura comunitaria nel grafo classe-docente, e parallelizza naturalmente sulla CPU. È pensata per scuole dense, dove la decomposizione spettrale fatica perché i cluster non emergono dalla struttura del grafo.

10.8.2 Decomposizione METIS

METIS è una libreria classica per il partizionamento di grafi che produce partizioni k -bilanciate minimizzando il taglio attraverso un algoritmo *multilevel*: il grafo viene prima collassato (matching pesato), poi partizionato sul grafo piccolo, poi raffinato risalendo i livelli. Il risultato è una partizione di cluster che minimizza il numero di archi inter-cluster, con il vincolo aggiuntivo che i cluster siano di taglia simile (la tolleranza di sbilanciamento è configurabile, di default cinque per cento).

METIS funziona bene su grafi densi senza struttura comunitaria evidente, dove la decomposizione spettrale restituirebbe cluster artificiali. È configurabile dal numero di partizioni K (di default scelto come radice quadrata del numero di classi) e dalla tolleranza di sbilanciamento. Richiede la libreria Python `pymetis`,

installabile con `pip install pymetis`; in mancanza, il sistema segnala chiaramente all'utente che deve installarla o scegliere un metodo alternativo.

10.8.3 Decomposizione per curriculum

La decomposizione per curriculum sfrutta direttamente l'informazione già codificata nel campo `curriculum_id` delle classi: ogni indirizzo (Liceo Scientifico, ITIS Informatica, eccetera) diventa un cluster, e i docenti che insegnano in più indirizzi sono i ponti. È la decomposizione più prevedibile e interpretabile, perché ogni cluster corrisponde a un'organizzazione che il coordinatore conosce bene. In compenso ignora la connettività effettiva del grafo e produce ponti più numerosi quando i docenti circolano molto fra indirizzi. È pensata per scuole con indirizzi ben definiti e con poca circolazione inter-indirizzo dei docenti, e l'utente può accorpare manualmente curricula con poche classi per evitare cluster troppo piccoli.

10.8.4 Auto-detect della strategia migliore

Per evitare che il coordinatore debba scegliere a tentativi, π Tantum espone una funzione di auto-detect che calcola due metriche descrittive del grafo bipartito classe-docente, la modularità di Newman-Girvan e la densità, e suggerisce la strategia di decomposizione migliore. Se la modularità è alta (sopra zero virgola trenta) consiglia la decomposizione spettrale, perché il grafo presenta cluster naturali ben separati. Se la densità è alta (sopra zero virgola sessanta) consiglia METIS, perché il grafo è denso e privo di cluster spontanei. In tutti gli altri casi consiglia la decomposizione per curriculum, che resta la scelta di compromesso più robusta. In ogni caso il sistema suggerisce di combinare la scelta primaria con la decomposizione temporale, che parallelizza ortogonalmente sui sei giorni della settimana e può essere applicata dopo qualunque altra. La raccomandazione è accessibile via l'endpoint REST `GET /api/optimize/decomposition/recommend` e viene mostrata dall'interfaccia come tooltip "Suggerimento" nella card delle decomposizioni del tab Workflow.

Per approfondire

Per chi voglia approfondire i contenuti di questo capitolo in modo sistematico, segnalo i seguenti riferimenti. L'articolo originale di Fiedler (Fiedler 1973) si legge in tre pagine ed è ancora oggi il testo fondante, contenente l'intuizione cruciale del secondo autovettore. Il libro di Chung (Chung 1997) è il riferimento canonico sulla teoria spettrale dei grafi, tecnico ma chiaro. Il tutorial di von Luxburg (Luxburg 2007) è un'introduzione divulgativa eccellente al spectral clustering, con tutti i dettagli pratici sulla scelta di k , sulle varianti di normalizzazione e sulle alternative al k -means. L'articolo di Shi e Malik (Shi e Malik 2000) sull'applicazione al *normalized cut* per la segmentazione di immagini ha reso popolare l'idea fuori dalla matematica pura ed è piacevolmente leggibile.

11 Le metaeuristiche, scolpire una soluzione

“Heuristics are mental shortcuts; meta-heuristics are shortcuts about how to take shortcuts.”

Christian e Griffiths, *Algorithms to Live By* (Christian e Griffiths 2016)

CP-SAT, lo abbiamo visto nel capitolo precedente, è bravissimo a trovare soluzioni fattibili, e quando ottimizza una funzione obiettivo trova spesso anche buone soluzioni in pochi minuti. Ma su istanze grandi e complesse, dopo un certo punto, tende a fermarsi su un *plateau*: continua a frugare lo spazio delle soluzioni senza migliorare significativamente quella corrente. Le metaeuristiche sono una famiglia di tecniche complementari che intervengono proprio quando questo accade: prendono una soluzione già esistente e la migliorano iterativamente esplorando il *vicinato*, ovvero lo spazio delle soluzioni “simili”, in modo intelligente.

In π Tantum la Phase C della pipeline è dedicata interamente alle metaeuristiche. Sono sei: LNS, ALNS, Simulated Annealing, Tabu Search, Iterated Local Search e Variable Neighborhood Search. Hanno parecchio in comune, nel senso che sono tutte forme di ricerca locale, ma differiscono profondamente nel modo in cui esplorano lo spazio delle soluzioni e nel modo in cui accettano le mosse nuove. Questo capitolo le presenta una alla volta, partendo dalla loro radice comune, e chiude con una discussione di quale convenga scegliere a seconda della situazione.

11.1 La radice comune: la ricerca locale

La ricerca locale, da cui tutte le metaeuristiche discendono, è uno schema iterativo elementare. Si parte da una soluzione iniziale, qualunque essa sia, ottenuta da un algoritmo greedy o da una qualsiasi euristica costruttiva. Si definisce un *vicinato* della soluzione corrente, ovvero l'insieme delle soluzioni ottenibili da quella attuale con una singola mossa elementare; nel timetabling una mossa tipica è lo scambio di due lezioni dello stesso docente, oppure lo spostamento di una lezione in uno slot libero. Si esplora il vicinato cercando una soluzione migliore della corrente, e se la si trova vi ci si sposta; poi si ripete dal nuovo punto. Quando il vicinato non contiene più alcuna soluzione migliore, ci si ferma: si è in un *ottimo locale*.

Il problema della ricerca locale pura, prevedibilmente, è proprio che si incastra nei minimi locali. Se l'ottimo globale richiede di passare attraverso una soluzione temporaneamente peggiore, la ricerca locale non la attraverserà mai e si fermerà in una soluzione sub-ottima. Le varie metaeuristiche sono altrettante strategie per evitare o per uscire da questa trappola.

11.2 Simulated Annealing: l'analogia con la metallurgia

Nota storica

Nel 1983 Kirkpatrick, Gelatt e Vecchi pubblicarono su *Science* l'articolo *Optimization by Simulated Annealing* (Kirkpatrick et al. 1983), importando nell'ottimizzazione combinatoria un'idea presa direttamente dalla metallurgia. Per ottenere una struttura cristallina ordinata in un metallo non basta raffreddarlo bruscamente, perché in tal caso la materia rimane intrappolata in uno stato disordinato; bisogna invece scaldarlo e raffreddarlo lentamente, lasciandogli il tempo di trovare configurazioni a

bassa energia. La temperatura agisce come parametro di “rumore termico”: alta temperatura permette movimenti casuali che peggiorano la situazione, bassa temperatura ammette soltanto movimenti migliorativi. L’idea è tradurre questo processo fisico in un algoritmo di ricerca, che all’inizio accetta movimenti peggiorativi con una certa probabilità e via via diventa più selettivo. L’articolo metropolis del 1953 (Metropolis et al. 1953) aveva fornito il fondamento statistico, applicato in origine a simulazioni di sistemi termodinamici.

L’idea di simulated annealing si traduce in una regola operativa molto semplice. Ad ogni iterazione si propone una mossa: se essa migliora la funzione obiettivo, la si accetta sempre. Se la peggiora di una quantità Δ , la si accetta con probabilità $e^{-\Delta/T}$, dove T è la temperatura corrente. All’inizio T è alta e anche peggioramenti consistenti vengono accettati con buona probabilità; man mano che T scende, soltanto i piccoli peggioramenti passano; alla fine, quando T è quasi zero, il comportamento si avvicina a quello di una ricerca locale greedy che accetta soltanto miglioramenti.

In π Tantum lo schema di raffreddamento di default è geometrico, con $T_{k+1} = \alpha T_k$ e $\alpha = 0.95$. Si parte da una temperatura iniziale tale che le mosse iniziali abbiano una probabilità di accettazione del settanta-ottanta per cento, e si scende fino a $T \sim 0.01$, momento in cui il sistema converge.

11.3 Tabu Search: il taccuino delle mosse vietate

Nota storica

Fred Glover propose Tabu Search nel 1986 con il celebre articolo *Future Paths for Integer Programming and Links to Artificial Intelligence* (Glover 1986), oggi considerato uno dei lavori fondativi della moderna metaeuristica. La parola *tabu* viene dal polinesiano e significa “proibito”: l’idea è che, in ricerca locale, dopo aver visitato una soluzione si vieti di tornarci per un certo periodo, in modo da forzare la ricerca a esplorare zone diverse dello spazio.

Tabu Search mantiene una *lista tabu* delle ultime mosse fatte (per esempio “ho spostato matematica della 1A da lunedì alla prima ora a martedì alla terza”). Le mosse nella lista tabu non possono essere usate finché non escono dalla finestra: la lista è di lunghezza fissa, e ogni nuova mossa spinge fuori la più vecchia. Esiste inoltre un meccanismo detto *aspiration criterion*: se una mossa tabu produrrebbe la migliore soluzione mai vista fino a quel momento, la si esegue lo stesso, perché non avrebbe senso rifiutare la migliore opzione disponibile per il solo motivo che si trova nella lista. La lista tabu in π Tantum è tipicamente di venti-quaranta mosse, valore tarato sui benchmark dei profili di test.

11.4 Iterated Local Search: perturbare e ricominciare

L’idea di Iterated Local Search, sviluppata in modo sistematico da Loureno, Martin e Stuetzle (Lourenco et al. 2003), è che una semplice ricerca locale si ferma su un ottimo locale, ma rilanciandola da una soluzione *simile ma perturbata* si arriva spesso a un ottimo locale diverso, eventualmente migliore. Iterando il procedimento si esplorano molti bacini di attrazione diversi senza dover ricominciare da zero ogni volta.

L’algoritmo alterna due operazioni. La prima è una ricerca locale tradizionale fino al raggiungimento di un ottimo locale x^* . La seconda è una perturbazione: si applica a x^* un disturbo casuale di entità controllata, producendo una soluzione x' , di solito ottenuta facendo cinque-dieci scambi casuali di lezioni. La ricerca locale ricomincia da x' , e produce un nuovo ottimo locale x^{**} che si confronta con x^* tenendo il migliore dei due. ILS è spesso il metaeuristico “base” che si combina con altri (per esempio ILS+Tabu Search, ILS+Simulated Annealing) per ottenere ibridi più potenti.

11.5 Variable Neighborhood Search: cambiare obiettivo

VNS, dovuto a Mladenović e Hansen del 1997 (Mladenovic e Hansen 1997), propone una strategia diversa per uscire dai minimi locali: invece di perturbare la soluzione corrente, si cambia il *tipo* di vicinato in cui si

cerca. L'idea è che un ottimo locale rispetto a un vicinato piccolo non è necessariamente un ottimo locale rispetto a un vicinato più grande, e quindi crescendo il raggio della ricerca si può uscire dall'incastro.

In π Tantum sono definiti quattro vicinati di dimensione crescente: il primo è l'1-swap, ovvero lo scambio di due lezioni dello stesso docente; il secondo è il 2-swap, due coppie indipendenti di lezioni scambiate insieme; il terzo è il 3-chain, una rotazione di tre lezioni del tipo $a \rightarrow b \rightarrow c \rightarrow a$; il quarto è il k -opt con k fra quattro e sei, una rotazione lunga che coinvolge molte lezioni contemporaneamente. VNS parte sempre dal vicinato più piccolo, e quando trova un miglioramento ricomincia dal vicinato uno; quando un'intera passata da uno a quattro non trova alcun miglioramento, l'algoritmo si ferma. La soluzione finale è un ottimo locale rispetto a tutti e quattro i vicinati.

11.6 Large Neighborhood Search: distruggere e ricostruire

LNS fu introdotto da Paul Shaw nel 1998 (Shaw 1998) per il problema del vehicle routing, e da allora si è diffuso in molti altri domini applicativi del scheduling combinatorio. L'idea è radicale rispetto alle metaeuristiche tradizionali: invece di esplorare un vicinato piccolo (uno o due swap), si *distrugge* una porzione significativa della soluzione corrente (per esempio il venti per cento delle lezioni) e la si *ricostruisce ottimamente* con un solver esatto. Il vicinato così definito è "grande", nel senso che contiene tutte le soluzioni ottenibili ricostruendo la parte distrutta in modo diverso, ma viene esplorato in modo intelligente dal solver invece che con una scansione elementare.

LNS alterna dunque due operatori. Il primo è l'*operator destroy*, che rimuove un sottoinsieme delle variabili dalla soluzione corrente, lasciando le altre fisse. Il secondo è l'*operator repair*, che re-ottimizza le variabili rimosse con tutti i vincoli del problema attivi e con le altre variabili fisse come dato del problema. In π Tantum l'operator repair è quasi sempre CP-SAT con un budget di tempo di pochi secondi, e gli operator destroy disponibili sono numerosi: `random_window` (rimuove un blocco di slot contigui), `day_cluster` (rimuove tutte le lezioni di un intero giorno), `worst_fit_day` (rimuove il giorno con il peggior valore obiettivo), `teacher_day` (rimuove tutte le lezioni di un docente in un giorno), `classroom_day` e `single_class_week` (varianti analoghe). Gli operator repair classici sono `cp_sat_window` (re-risolve con CP-SAT), `greedy_by_soft` (ricostruisce con un greedy che ottimizza l'obiettivo SOFT) e `bfs_fill_back` (ricostruisce con una ricerca in ampiezza guidata).

11.7 Adaptive LNS: scolpire con scalpelli adattivi

Nota storica

ALNS fu proposto da Ropke e Pisinger nel 2006 (Ropke e Pisinger 2006) estendendo LNS con un meccanismo di apprendimento. Invece di scegliere a caso fra gli operator destroy e repair disponibili, ALNS impara durante l'esecuzione quali funzionano meglio sull'istanza corrente e li seleziona più spesso. Il meccanismo si ispira agli algoritmi di apprendimento per rinforzo: ad ogni iterazione ogni operatore riceve un punteggio proporzionale al successo della sua applicazione, e i punteggi vengono mediati nel tempo con una decay esponenziale.

L'analogia che aiuta a capire ALNS è quella dello scultore davanti a un blocco di marmo: lo scultore non usa sempre lo stesso scalpello, ma ne ha tanti, e sceglie quello giusto a seconda della parte della statua su cui sta lavorando. Allo stesso modo ALNS sceglie ad ogni iterazione l'operator destroy e l'operator repair più promettenti sulla base dei punteggi accumulati, calcolati come media pesata esponenzialmente decrescente nel tempo. Il criterio di accettazione delle mosse è tipicamente analogo a quello di simulated annealing.

L'aggiornamento dei punteggi avviene in modo semplice. Dopo ogni iterazione, l'operatore usato riceve un premio, di ammontare σ_1 , σ_2 o σ_3 a seconda del risultato: σ_1 se la mossa ha migliorato globalmente la migliore soluzione mai vista, σ_2 se ha migliorato soltanto la soluzione corrente, σ_3 se è stata accettata via meccanismo SA-like pur peggiorando. Il punteggio dell'operatore w_i viene aggiornato come $w_i \leftarrow \rho w_i + (1 - \rho)\sigma$ con $\rho = 0.95$. La selezione dell'operatore avviene poi via roulette wheel pesata sui punteggi correnti.

11.8 Quale scegliere

Intuizione

Una regola pratica orientativa può essere la seguente. Se si ha poco tempo a disposizione e si vuole un risultato “decente” in pochi minuti, conviene usare simulated annealing con uno schema di raffreddamento veloce. Se si vuole esplorare in modo robusto lo spazio, ILS o VNS sono buone scelte. Se si ha molto tempo a disposizione e si vuole un risultato vicino all’ottimo, ALNS è quasi sempre il migliore perché impara a fare le mosse giuste sull’istanza corrente. Se la soluzione iniziale è molto strutturata e servono mosse grandi, LNS con CP-SAT come repair offre un buon compromesso fra ampiezza dell’esplorazione e qualità del risultato. In Phase C, π Tantum esegue per default ALNS, e mette opzionalmente a disposizione le altre per sperimentazione e confronto.

11.9 Un esempio concreto su otto lezioni

Esempio

Riprendiamo l’esempio del cap. 9 con tre docenti, due classi e otto lezioni. Una soluzione fattibile è già stata trovata da CP-SAT con valore della funzione obiettivo $f(x_0) = 5$ (somma pesata di ore isolate dei docenti, per esempio).

Iterazione 1: simulated annealing prova uno scambio fra le ore 1 e 2 della matematica della 1A. Il nuovo valore è $f(x_1) = 4$. Migliora! Accetta sempre.

Iterazione 2: prova uno scambio fra l’italiano della 1A (martedì prima) e la storia della 1A (lunedì seconda). Il nuovo valore è $f(x_2) = 6$. Peggiora di $\Delta = 2$. Con $T = 1.0$ la probabilità di accettazione è $P = e^{-2/1.0} = 0.135$. Estrae un numero casuale $r = 0.4 > P$ e rifiuta la mossa. Resta su x_1 .

Iterazione 3: prova lo scambio fra la prima ora di matematica della 1A e quella della 1B. Il nuovo valore è $f = 3$. Migliora, accetta.

Schema di raffreddamento: $T \leftarrow 0.95T = 0.95$.

Così di seguito per migliaia di iterazioni. Dopo cinquemila iterazioni tipiche $T < 0.01$ e il sistema converge a un ottimo locale rispetto al vicinato 1-swap. Su questa istanza piccola il valore finale è quasi sempre $f = 2$, che si può dimostrare essere l’ottimo globale.

11.10 Limiti e parametri da tarare

Attenzione

Le metaeuristiche non garantiscono l’ottimo. Su problemi piccoli possono trovare soluzioni peggiori di CP-SAT esatto in poche iterazioni; conviene quindi combinarle, applicando prima CP-SAT esatto fino al raggiungimento del plateau e poi le metaeuristiche per il raffinamento. I parametri da tarare sono diversi: la temperatura iniziale di simulated annealing, la lunghezza della lista tabu di tabu search, la dimensione della perturbazione di ILS. π Tantum ha valori di default ragionevoli per ciascun profilo di test, ma su istanze atipiche conviene fare uno sweep di tuning. Le mosse infine devono essere economicamente valutabili: π Tantum implementa una *delta-evaluation* della funzione obiettivo, che evita di ricalcolare tutto da zero ad ogni iterazione e che porta il costo per swap da $O(n)$ a $O(1)$.

11.11 Le connessioni con il resto del sistema

Le metaeuristiche non vivono isolate. CP-SAT (cap. 9) viene usato come solver di *repair* in LNS e in ALNS, perché la sua capacità di re-ottimizzare velocemente sotto-vicinati piccoli costituisce il motore della distruzione-ricostruzione. La decomposizione spettrale (cap. 10) viene applicata prima delle metaeuristiche, che lavorano poi sopra la soluzione già parzialmente strutturata. L’editor manuale dell’orario nell’interfaccia utente sfrutta

gli stessi schemi di mosse di simulated annealing: gli scambi dell'utente sono mosse r-swap del medesimo vicinato, e l'editor mostra in tempo reale il delta della funzione obiettivo esattamente come fa internamente l'algoritmo.

Per approfondire

Per chi voglia approfondire i contenuti di questo capitolo in modo sistematico, segnalo alcuni riferimenti chiave. L'articolo originale di simulated annealing (Kirkpatrick et al. 1983) si legge in tre pagine. L'articolo originale di tabu search (Glover 1986) è di Glover. L'articolo originale di LNS (Shaw 1998) è di Shaw, e quello originale di ALNS (Ropke e Pisinger 2006) è di Ropke e Pisinger. Per una visione d'insieme, l'*Handbook of Metaheuristics* (Glover e Kochenberger 2003) edito da Glover e Kochenberger è il riferimento collettivo. Van Hentenryck e Michel, *Constraint-Based Local Search* (Van Hentenryck e Michel 2005), propongono una sintesi elegante fra constraint programming e ricerca locale.

12 Il pre-controllo di Hall

“A necessary and sufficient condition that all the sets in the family S have a system of distinct representatives is that for every k , the union of any k sets of S contains at least k elements.”

Philip Hall, 1935 (Hall 1935)

PRIMA di intraprendere un viaggio lungo, qualunque automobilista controlla quanta benzina ha nel serbatoio. Sembra un’osservazione ovvia, eppure quando si lancia un solver di timetabling è una pratica che raramente si seguiva: si lanciava la pipeline, si attendeva qualche minuto, e si scopriva che il problema era infeasible fin dall’inizio. Tutti i tentativi del solver erano destinati a fallire perché i dati di input non potevano produrre nessun orario valido. Il pre-controllo di Hall, integrato in π Tantum come modulo separato e come bottone esposto dall’interfaccia, è quel controllo del livello di benzina: in pochi millisecondi, prima di lanciare CP-SAT, verifica se le condizioni necessarie alla fattibilità sono soddisfatte. Se Hall fallisce, il sistema lo segnala all’utente con l’indicazione del sotto-insieme di entità problematico, evitando lunghe attese inutili.

12.1 Il teorema dei matrimoni

Nota storica

Nel 1935 Philip Hall pubblicò sul *Journal of the London Mathematical Society* (Hall 1935) un teorema deliziosamente semplice. Si consideri un gruppo di n ragazze e m ragazzi; ogni ragazza ha una propria lista di ragazzi che sarebbe disposta a sposare. Quando è possibile sposare tutte le ragazze in modo che ognuna abbia un proprio ragazzo distinto? La risposta di Hall è altrettanto semplice: lo si può fare se e soltanto se per ogni sotto-insieme di k ragazze, la lista combinata dei loro candidati contiene almeno k ragazzi distinti. La condizione è necessaria, perché in caso contrario non vi sarebbero abbastanza candidati per quel sotto-gruppo, ed è anche sufficiente, come dimostra il teorema con un argomento costruttivo elegante. Da allora il risultato è universalmente conosciuto come *teorema dei matrimoni di Hall*, o più tecnicamente *Hall’s marriage theorem*.

L’analogia con il problema del timetabling è immediata. Si consideri la versione semplificata in cui occorre assegnare ad ogni lezione (combinazione di classe, materia e ora) un docente compatibile, vale a dire un docente abilitato alla materia in questione, alla classe di concorso giusta e con sufficiente disponibilità oraria residua. Il problema è formalmente identico a quello dei matrimoni: le lezioni sono le “ragazze”, i docenti sono i “ragazzi” (con la differenza che un docente può coprire più lezioni purché non superi il proprio monte ore), e la condizione di Hall, opportunamente generalizzata, caratterizza la fattibilità del matching.

12.2 La traduzione operativa

Per applicare il teorema in modo computazionalmente trattabile occorre rinunciare alla sua formulazione esatta, che richiederebbe di esaminare *tutti* i sottoinsiemi delle ore-cattedra (un numero esponenziale, 2^n con n ore-cattedra), e di approssimarla con un controllo che intercetti la stragrande maggioranza dei casi infeasible osservabili nella pratica. π Tantum implementa allora tre controlli rapidi che, combinati, sono sufficienti a quasi tutte le situazioni reali.

Il primo controllo si applica per ogni coppia (classe, materia). Per ciascuna combinazione con h_{cm} ore richieste, il sistema verifica che la somma delle ore disponibili dei docenti abilitati alla materia m nella classe di concorso pertinente per la classe c sia almeno pari a h_{cm} . Un caso tipico in cui questo controllo fallisce è: “servono cinque ore di matematica nel liceo classico, ma i due docenti di Ao27 hanno già saturato le loro ore in altri istituti”. π Tantum lo intercetta immediatamente e mostra il deficit con il sottoinsieme problematico evidenziato.

Il secondo controllo opera a livello di intera materia. Per ciascuna materia m , la *domanda* è la somma di h_{cm} su tutte le classi c , e l'*offerta* è la somma delle ore residue di tutti i docenti abilitati alla materia. Il sistema verifica che l'offerta sia almeno pari alla domanda. Un fallimento di questo controllo significa che l'intera materia è in deficit strutturale: nessuna distribuzione potrà coprire tutte le classi. La soluzione operativa richiede di aggiungere un docente, di ridurre il monte ore della materia o di convocare delle supplenze straordinarie.

Il terzo controllo è di natura probabilistica e si chiama *Hall sampling random-subset*. Si campionano casualmente K sotto-insiemi di lezioni (tipicamente $K = 200$, di taglie fra cinque e cinquanta), e per ognuno si verifica direttamente la condizione di Hall. Quando un'istanza ha un *sottoinsieme cattivo* di lezioni che soddisfano i controlli uno e due ma non Hall, il campionamento ha una alta probabilità di “inciampare” su di esso e segnalarlo. Si tratta di un controllo euristico, non garantito, ma nella pratica π Tantum lo trova sufficiente nella stragrande maggioranza dei casi.

12.3 Un esempio in miniatura

Esempio

Si consideri una scuola con tre classi (1A, 1B, 1C), tutte con due ore di matematica settimanali, e con due docenti di classe di concorso Ao27: Adami con quattro ore residue e Bruni con tre ore residue. La domanda complessiva è di sei ore di matematica; l'offerta è di sette ore. Il controllo due passa.

Il controllo uno richiede che ogni classe individualmente trovi un docente compatibile con almeno due ore residue. Adami e Bruni hanno entrambi più di due ore residue, quindi ogni classe trova almeno un candidato. Il controllo uno passa.

Aggiungiamo ora un vincolo: Bruni non può insegnare in 1C. La domanda di 1C-mat è di due ore, e l'offerta dei docenti compatibili con 1C-mat si riduce ad Adami soltanto, con le sue quattro ore residue.

Il controllo uno passa ancora.

Ma se aggiungiamo un secondo vincolo, ovvero che Adami satura tre delle sue quattro ore residue con altre materie (per esempio fisica), restano per Adami soltanto un'ora disponibile per la matematica. La domanda complessiva di matematica nelle tre classi è di sei ore; l'offerta di Adami più Bruni è ora di un'ora più tre ore, totale quattro. Il controllo due fallisce: il sistema mostra che mancano due ore di matematica, e l'utente sa subito che deve rilassare un altro docente o ridurre il monte ore.

12.4 L'integrazione nell'interfaccia

In π Tantum il pre-controllo di Hall è esposto in tre punti distinti dell'interfaccia, tutti che invocano lo stesso modulo `hall_check.py` e producono lo stesso output. Il primo punto è un bottone inline nella card di Phase A del tab Workflow, posizionato come azione consigliata prima del lancio della pipeline. Il secondo punto è una riga nel pannello *tecniche avanzate*, con un bottone “Esegui” che lancia tutti e tre i controlli e mostra i risultati nello stesso pannello. Il terzo punto è una sezione del tab Diagnostica, dove il risultato è mostrato come run asincrono visualizzabile in dettaglio.

L'output è sempre una lista di deficit con il sottoinsieme problematico evidenziato, per esempio:

```
HALL CHECK FAILED
Deficit 1 (controllo 2):
  materia=Matematica, domanda=18, offerta=15
  -- 3 ore mancanti
Deficit 2 (controllo 1):
  classe=4B, materia=Storia: 2 ore richieste,
```

ma il docente compatibile (Conti) ne ha solo 1 residua.

12.5 Limiti del pre-controllo

Attenzione

Il pre-controllo di Hall è necessario ma non sufficiente: può passare anche quando il problema è infeasibile per ragioni temporali, per esempio quando lo stesso docente serve in due slot incompatibili nello stesso giorno. Un “ok” di Hall garantisce soltanto che le ore totali bastino, non che si possano disporre nello spazio settimanale. I controlli uno e due sono esatti; il controllo tre è euristico, basato sul campionamento. Aumentare K rende il controllo più robusto a discapito del tempo. Il pre-controllo non considera i vincoli logici DSL complessi, del tipo “il prof X e Y mai stesso giorno”; tali vincoli vengono verificati soltanto durante l’esecuzione di CP-SAT.

12.6 Approfondimento: l’algoritmo di Hopcroft-Karp

Approfondimento

Verificare esattamente l’esistenza di un matching che ricopra un lato di un grafo bipartito si può fare in tempo $O(E\sqrt{V})$ con l’algoritmo di Hopcroft-Karp del 1973 (Hopcroft e Karp 1973), dove E è il numero di archi e V il numero di vertici del grafo. È quello che usa internamente OR-Tools per il propagatore globale AllDifferent: ogni volta che CP-SAT propaga un vincolo del tipo “ n variabili devono prendere valori distinti”, costruisce il grafo bipartito variabili-valori e chiama Hopcroft-Karp per verificare la fattibilità residua. Se non c’è matching, il vincolo è violato, π Tantum riceve subito una contraddizione e fa backtracking immediato. In π Tantum non si usa Hopcroft-Karp esplicitamente nel pre-controllo, perché i tre controlli rapidi sono più che sufficienti nei casi pratici, ma l’algoritmo è dentro CP-SAT a ogni propagazione del vincolo AllDifferent.

12.7 Le connessioni con il resto del sistema

Il pre-controllo di Hall è legato a quasi tutti gli altri componenti della pipeline. CP-SAT (cap. 9) usa lo stesso teorema internamente, attraverso il propagatore di AllDifferent, ad ogni passo della propria ricerca. La decomposizione spettrale (cap. 10) richiede che Hall sia verificato non soltanto sull’intera scuola ma anche su ciascun cluster: se un cluster non passa Hall, il sistema deve rilassare i ponti per ridistribuire la domanda. Nel flusso operativo standard, il pre-controllo di Hall è raccomandato come primo passo della pipeline, perché risparmia minuti di calcolo CP-SAT su istanze infeasibili.

Per approfondire

L’articolo originale di Hall (Hall 1935) si legge in cinque pagine, e resta una gemma di matematica discreta. L’articolo di Hopcroft e Karp (Hopcroft e Karp 1973) sull’algoritmo polinomiale per il matching bipartito è fondante per la complessità moderna di questa classe di problemi. Il libro di Lovász e Plummer (Lovász e Plummer 1986), intitolato *Matching Theory*, è il riferimento canonico sull’argomento. Papadimitriou e Steiglitz (Papadimitriou e Steiglitz 1998), nei capitoli dieci e undici, offrono una sezione accessibile sui matching e sui flussi.

13 Lagrangian Relaxation e Column Generation

“If you cannot solve a problem, relax it.”

adagio della ricerca operativa, attribuito a vari autori

CI SONO problemi che, pur essendo risolvibili in linea di principio da CP-SAT, sono *così grandi* che il solver impiega ore per produrre una soluzione mediocre. Una scuola con 200 classi, 400 docenti, 12 indirizzi non sta nei tempi di Phase B di un ciclo standard. Per questi casi π Tantum offre due tecniche di *rilassamento e decomposizione* di derivazione classica: il rilassamento lagrangiano e la generazione di colonne. Entrambe sono di default *disattivate*, da abilitare esplicitamente da chi gestisce la pipeline.

13.1 Rilassamento lagrangiano: dualizzare i ponti

Nota storica

Il rilassamento lagrangiano nasce nella ricerca operativa degli anni '60-'70. Held e Karp (1971) (Held e Karp 1971) lo applicarono al *Traveling Salesman Problem* ottenendo risultati eccellenti su istanze grandi. Geoffrion (1974) (Geoffrion 1974) formalizzò l'approccio per la programmazione intera generale; Fisher (1981) (Fisher 1981) ne fece il manuale operativo per chi voleva applicarlo nella pratica industriale.

Intuizione

L'idea con un'analogia. Hai un problema enorme che non riesci a risolvere. Decidi di *dimenticare* alcuni vincoli scomodi (che “legano” parti diverse del problema) e di *punirli* con una multa proporzionale alla loro violazione. Risolvi il problema rilassato. Misuri quanto i vincoli scomodi sono violati. Aggiusti la multa: dove sono violati di più, alzi la multa; dove sono soddisfatti con margine, la abbassi. Iteri.

Si dimostra che, sotto ipotesi tecniche, le multe convergono a un valore tale che la soluzione del problema rilassato soddisfa anche i vincoli “dimenticati” — ottenendo quindi una soluzione del problema originale.

13.1.1 Formalizzazione

Supponi un problema di minimo:

$$\min_{x \in X} f(x) \quad \text{soggetto a } g(x) \leq 0$$

dove X è un insieme di soluzioni “facili” (es. unione di sotto-problemi indipendenti) e $g(x) \leq 0$ rappresenta i vincoli “difficili” (i ponti fra sotto-problemi).

Definizione

La *Lagrangiana* è:

$$L(x, \lambda) = f(x) + \lambda^\top g(x)$$

con $\lambda \geq 0$ vettore dei *moltiplicatori di Lagrange*.

Il *problema rilassato* è:

$$q(\lambda) = \min_{x \in X} L(x, \lambda)$$

La funzione $q(\lambda)$ è detta *funzione duale*.

Come si legge: per ogni valore del vettore λ , risolvi un problema senza i vincoli scomodi $g(x) \leq 0$ ma con una penalità $\lambda^\top g(x)$ aggiunta all'obiettivo. Se $g(x) > 0$ (vincolo violato), la penalità sale; se $g(x) < 0$ (vincolo soddisfatto con margine), la penalità scende (il termine è negativo). Questo “incentiva” x a soddisfare i vincoli senza forzarli.

13.1.2 Subgradient ascent: aggiornare i moltiplicatori

Si dimostra che $q(\lambda)$ è concava in λ e che $\max_{\lambda \geq 0} q(\lambda) \leq \min_{x: g(x) \leq 0} f(x)$ (weak duality). L'algoritmo classico aggiorna λ con un passo di *subgradient ascent*:

$$\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g(x_k))$$

dove $\alpha_k > 0$ è il *passo*, tipicamente $\alpha_k = \alpha_0 / (1 + k)$ in π Tantum.

Come si legge: dopo aver risolto il rilassato e trovato x_k , calcolo quanto i vincoli sono violati ($g(x_k)$). Aggiorno λ aggiungendoci un multiplo di $g(x_k)$: i moltiplicatori associati a vincoli più violati salgono di più. Tronco a zero per mantenere $\lambda \geq 0$.

13.1.3 Cosa rilassare nel timetabling?

In π Tantum la decomposizione lagrangiana si applica *dopo* la decomposizione spettrale: si rilassano i *ponti inter-cluster*, cioè i vincoli che legano lezioni di cluster diversi (tipicamente docenti che insegnano a cavallo fra due cluster, oppure vincoli logici DSL che attraversano cluster).

Ogni cluster diventa un sotto-problema CP-SAT indipendente con un costo aggiuntivo $\lambda_b \cdot (\text{violazione del ponte } b)$. Iterando il subgradient, il sistema converge a un assegnamento dei ponti coerente con tutti i cluster.

13.1.4 Esempio in miniatura

Esempio

Due cluster: A (3 classi) e B (3 classi). Un docente, Conti, deve insegnare 2 ore in cluster A (storia in 1A) e 2 ore in cluster B (storia in 4B). Vincolo ponte: “Conti non in due slot contemporanei”.

Senza rilassamento: Phase B risolve A e B insieme, vincolo HARD globale, propagazione attraverso i cluster (lento).

Con rilassamento: λ inizia a 0. Risolvo A da solo (mette Conti in lun-1, lun-2). Risolvo B da solo (mette Conti in lun-1, lun-2). Conflitto: Conti in lun-1 e lun-2 in entrambi i cluster.

Aggiorno $\lambda \leftarrow \lambda + \alpha \cdot 2 = 0.2$ (violazione = 2 slot duplicati).

Iter 2: A ora penalizza Conti in lun-1, lun-2 con costo 0.2 ciascuno; trova Conti in mar-1, mar-2 (costo SOFT più alto ma niente penalità). B mantiene Conti in lun-1, lun-2. Conflitto risolto, ma A ha pagato un piccolo costo SOFT.

Dopo 5–10 iterazioni il sistema trova un equilibrio in cui Conti è in slot diversi nei due cluster e nessun vincolo globale è violato.

13.1.5 Perché non sempre?

Attenzione

Lagrangian Relaxation è potente ma costoso da implementare bene:

- Richiede il *tuning* del passo α_0 . Troppo grande $\rightarrow$ oscillazioni; troppo piccolo $\rightarrow$ convergenza lentissima.
- Può *non convergere* se il problema duale ha un *duality gap* grosso (frequente nei problemi combinatori interi).
- La soluzione finale del rilassato *non* è necessariamente fattibile per il problema originale: serve un passo di “ricostruzione” (in π Tantum: un giro di SA per chiudere i ponti residui).

Per queste ragioni π Tantum lo offre come *opzione avanzata*, non come default.

13.2 Column Generation: il catalogo immobiliare

Nota storica

La generazione di colonne (*Column Generation*) nasce nel 1960 con Dantzig e Wolfe (Dantzig e Wolfe 1960). L'idea: in molti grandi problemi di programmazione lineare, *la maggior parte delle variabili non serve*; sono attive solo poche. Invece di costruire il problema con tutte le variabili, lo si costruisce con poche e si *generano nuove colonne* (= nuove variabili) solo quando servono.

Intuizione

Analogia. Sei un'agenzia immobiliare con un milione di appartamenti in catalogo. Un cliente cerca un trilocale in zona centro. Non gli passi tutto il catalogo: gli proponi 3–4 appartamenti che pensi gli vadano bene. Se non gli piacciono, ne tiri fuori altri 3–4 con caratteristiche diverse. Itera finché il cliente è soddisfatto. Hai usato il catalogo intero ma esaminandone solo una frazione. Column generation fa lo stesso: ha un *master problem* (piccolo, con poche variabili) e uno o più *sub-problem* che *generano* nuove variabili da aggiungere al master, basandosi sui prezzi duali correnti del master.

13.2.1 Schema operativo

1. Costruisci il master problem con un sotto-insieme iniziale di variabili (colonne).
2. Risolvi il master come LP. Ottieni i prezzi duali π delle restrizioni.
3. Per ogni sotto-problema, cerca una colonna con *costo ridotto negativo* ($c_j - \pi^T a_j < 0$). Se ne trovi, aggiungila al master.
4. Se nessun sotto-problema produce una colonna vantaggiosa, fermati: il master corrente è ottimo per l'LP rilassato.
5. (Per problemi interi) lancia un branch-and-bound sopra (*branch-and-price*).

13.2.2 Cosa generano le colonne nel timetabling?

In π Tantum la formulazione è: il master è un *set covering* dove ogni colonna rappresenta un *pattern settimanale completo* di un docente (la sequenza di tutti i suoi slot in tutte le sue classi). Il master sceglie un pattern per ogni docente in modo da coprire tutte le ore-cattedra delle classi. I sub-problem generano nuovi pattern docenti, uno per docente, sulla base dei prezzi duali.

Il vantaggio: il numero di pattern possibili per docente è combinatorialmente esplosivo (centinaia di milioni), ma il master ne usa solo qualche decina. Il sotto-problema è relativamente piccolo (un docente alla volta) e CP-SAT lo risolve in pochi millisecondi.

13.2.3 Granularità configurabile del sotto-problema

Lo schema di Dantzig-Wolfe non impone che il pattern sia *per docente*. La granularità – ovvero l’unità di decomposizione su cui il sotto-problema CP-SAT genera nuovi pattern – è una scelta di modellazione che π Tantum espone come opzione. Quattro granularità sono previste, ognuna adatta a una taglia o a una struttura diversa della scuola.

La granularità *per docente* (teacher) è quella già descritta sopra: un pattern è una proposta di settimana per un singolo docente, e si genera un sub per ogni docente. Funziona meglio nelle scuole piccole, dove il numero di docenti è dominante e i loro cataloghi settimanali coprono bene lo spazio.

La granularità *per classe* (class) inverte la prospettiva: un pattern è la settimana di lezioni di una classe, con i suoi docenti già assegnati. Si genera un sub per ogni classe. Scala meglio della prima quando il numero di classi è moderato (30-80) ma i cataloghi per docente esplodono.

La granularità *per giorno* (day) è una proiezione orizzontale: un pattern è una giornata intera della scuola, con tutti i suoi vincoli intra-day. Il sub è quindi un per giorno della settimana. Raramente è la scelta migliore in BnP puro – la decomposizione temporale del cap. 10 la sfrutta in modo più diretto – ma resta selezionabile per esperimenti.

La granularità *per indirizzo* (curriculum) è quella naturalmente adatta alle scuole italiane di medie-grandi dimensioni. Un pattern è la settimana di tutte le classi di un certo indirizzo (Scientifico, Classico, Linguistico, ITIS Informatica, eccetera), con i docenti che principalmente vi insegnano. Il sub è quindi un per ciascun indirizzo. I docenti-ponte (chi insegna in più indirizzi: tipicamente inglese, scienze motorie, religione) sono modellati come constraint del master che coordinano tra patterns di indirizzi diversi attraverso i duali. È lo schema classico di Vanderbeck (Vanderbeck e Wolsey 2010) per Dantzig-Wolfe applicato a problemi con *struttura bloccata* (block-angular): il problema globale si decompone in blocchi quasi-indipendenti (gli indirizzi) collegati da poche variabili globali (i ponti).

L’interfaccia espone una opzione *Auto* che il backend risolve in base al numero di classi della scuola attiva: sotto le trenta classi suggerisce teacher, fra trenta e ottanta class, sopra le ottanta con indirizzi ben definiti curriculum. Per il profilo MEGA (cento classi) l’auto-detect propone proprio curriculum, dato che gli otto indirizzi della scuola hanno pool di docenti largamente disgiunti.

13.2.4 Stato attuale e limiti pratici

Attenzione

La versione attuale di π Tantum esegue una variante *iterative-diversified* della column generation: master LP iterativo (scipy.linprog con backend HiGHS), arricchimento del catalogo con varianti casuali ad ogni iterazione, recupero intero per docente con il pattern di peso LP massimo, e *completion solver* day-by-day che riempie eventuali gap residui invocando direttamente `cpsat_v2_timetable`. Il risultato è HARD-feasibile in tutti i casi misurati su small.

Il vero *branch-and-price* con sub-CP-SAT guidato dai duali per ciascuna delle quattro granularità, e con branching di Ryan-Foster per la ricerca dell’integer feasible, è progettato e documentato in `docs/optimization_strategies.md` ma non è ancora implementato. La stima realistica è di cinque-dieci giorni di engineering OR focalizzato. La struttura del modulo `experiments/column_generation.py` è pronta per ospitare un secondo percorso (`mode="branch-and-price"`) accanto all’attuale `mode="iterative-diversified"`, ma il refactor del constraint del master e’ la parte invasiva.

Default: `enable_cg=False` nella pipeline.

13.3 Connessioni

Connessioni

- **Decomposizione spettrale** (cap. 10): naturale alleato del rilassamento lagrangiano (i cluster diventano i sotto-problemi, i ponti inter-cluster diventano i vincoli da dualizzare).
- **CP-SAT** (cap. 9): risolve i sotto-problemi della decomposizione lagrangiana e i sub-problem della column generation.
- **Metaeuristiche** (cap. 11): un giro di SA dopo Lagrangian per chiudere i ponti residui.

Per approfondire

- Geoffrion 1974, Fisher 1981: i due riferimenti classici sul rilassamento lagrangiano per IP.
- Held e Karp 1971: l'applicazione storica al TSP.
- Dantzig e Wolfe 1960: il paper originale sulla decomposizione di Dantzig-Wolfe e generazione di colonne.
- Desrosiers e Lübbecke 2005: *A Primer in Column Generation*, manuale moderno e accessibile.
- Vanderbeck e Wolsey 2010: trattamento sistematico delle decomposizioni *block-angular* per programmazione intera, su cui poggia la granularità per indirizzo.
- Ryan e Foster 1981: lo schema di branching che porta il loro nome, riferimento standard nei moderni branch-and-price.
- Wolsey 1998: capp. 10–11 sui rilassamenti e le decomposizioni in IP.

14 Monte Carlo: simulare per capire

“In any conflict between two ideas, lapse into random sampling and you’ll learn surprising things.”

adagio del calcolo statistico

DOPO aver prodotto un orario, una domanda naturale del coordinatore è: *quanto è robusto rispetto a piccoli cambiamenti?* Cosa succede se un docente cambia indisponibilità all’ultimo minuto? Cosa succede se una classe si sposta di sezione? Cosa succede se viene aggiunta una compresenza dell’ultim’ora?

Una risposta secca è impossibile. Una risposta statistica, invece, si può ottenere con il metodo *Monte Carlo*: si *perturba* l’input in modo casuale, si *rilancia* il solver molte volte, e si *guarda la distribuzione* dei risultati.

14.1 La storia del nome

Nota storica

Il metodo Monte Carlo nasce a Los Alamos negli anni ’40 nel contesto del progetto Manhattan. Stanislaw Ulam, durante una convalescenza, ideò di simulare l’evoluzione dei neutroni in una bomba atomica con simulazioni stocastiche. Lo chiamò “Monte Carlo” come scherzo, perché suo zio amava il casinò di Monte Carlo a Monaco. Metropolis e Ulam pubblicarono il primo articolo formale nel 1949 (Metropolis e Ulam 1949).

L’idea si è poi diffusa in fisica, finanza, biologia, statistica. Oggi *Monte Carlo simulation* è un metodo trasversale: ogni volta che un sistema deterministico è troppo complesso per essere analizzato esattamente, si campiona stocasticamente e si lavora sulle distribuzioni.

14.2 Sensitivity analysis: cosa misurare

Definizione

La *sensitivity analysis* consiste nel *perturbare* un parametro di un modello e nel misurare quanto cambia l’output. Versione Monte Carlo: invece di perturbare un parametro alla volta, perturba molti parametri contemporaneamente con distribuzioni assegnate, e analizza la distribuzione dell’output.

In π Tantum si offrono diverse sensitività, attivabili dalla pagina Diagnostica:

- **Indisponibilità docenti:** scegli K scenari in cui un docente casuale acquisisce un’indisponibilità in uno slot casuale. Per ognuno, rilancia la pipeline e registra: ha trovato soluzione? quanto è cambiato l’obiettivo? quante lezioni sono state spostate?
- **Aggiunta compresenze:** simula l’arrivo dell’ultimo momento di una compresenza in una classe random per una materia random.
- **Spostamento di una classe:** simula il cambio di curriculum/indirizzo di una classe.
- **Generico:** l’utente passa una funzione di perturbazione personalizzata via DSL.

14.3 Schema operativo

1. Fissa un seed iniziale e un numero K di campioni (default: 50).
2. Per ogni $i = 1, \dots, K$:
 - a) Applica una perturbazione casuale δ_i all'input.
 - b) Rilancia la pipeline (Phase A + B + C + D).
 - c) Registra: feasibility, obiettivo finale, tempo, numero di lezioni cambiate rispetto al baseline.
3. Calcola statistiche descrittive: media, deviazione standard, mediana, percentili 5/95, frazione di scenari infeasible.
4. Visualizza la distribuzione (istogramma) del valore obiettivo.

14.3.1 Esempio numerico

Esempio

Scuola medium: 28 classi, 50 docenti, baseline con obiettivo $f_0 = 11.4$. Lanciamo Monte Carlo con $K = 50$ scenari di tipo “indisponibilità docente”.

Risultati: 47 scenari producono soluzione fattibile (3 infeasible); fra i 47, $\bar{f} = 12.1$, $\sigma = 0.8$, mediana = 12.0, $P_5 = 11.0$, $P_{95} = 13.6$. Il numero medio di lezioni spostate è 6.3.

Lettura: il sistema è *robusto in media* (l'obiettivo peggiora di solo $\sim 6\%$), ma il 6% degli scenari diventa infeasible. Conviene investigare quali docenti sono “critici” (la cui indisponibilità aggiuntiva rende il problema infeasible).

14.4 Quando è utile

Intuizione

- **Pianificazione preventiva:** a settembre il coordinatore vuole sapere quali docenti sono “critici” prima che si manifestino le assenze. Monte Carlo li identifica.
- **Stima dell'impatto di una nuova richiesta:** un docente chiede di non lavorare il martedì. Monte Carlo simula l'aggiunta del vincolo (anche su altri docenti, per analogia) e dice quanto perde l'obiettivo in media.
- **Validazione di una soluzione:** una soluzione con basso f_0 ma alta varianza nel Monte Carlo è *fragile*; meglio cercarne una con f_0 leggermente più alto ma varianza minore.

14.5 Quando non basta

Attenzione

- Monte Carlo è *costoso*: K esecuzioni della pipeline. Per scuole grandi $K = 50$ può richiedere ore. π Tantum lo lancia come run asincrono e mostra il progresso live.
- L'analisi è *indicativa*, non garantita: se K è basso (es. 20), le statistiche hanno alta varianza. Aumentare K migliora ma costa.
- Monte Carlo *non* dice perché una perturbazione rompe il sistema; dice solo che lo rompe. Per capire perché serve guardare i log dei singoli scenari (CP-SAT mostra il sottoinsieme di vincoli incompatibili).

14.6 Connessioni

Connessioni

- **Hall pre-check** (cap. 12): ogni scenario Monte Carlo passa prima per Hall; gli scenari che falliscono Hall sono contati come “infeasible immediato” senza dover lanciare CP-SAT.
- **Statistica applicata** (cap. 16): le distribuzioni prodotte da Monte Carlo si analizzano con KS-test, istogrammi, percentili come qualsiasi altra distribuzione.

Per approfondire

- Metropolis e Ulam 1949: il primo paper formale. Cinque pagine.
- Robert e Casella 2004: *Monte Carlo Statistical Methods*, manuale di riferimento moderno.
- Wasserman 2004: capp. 24–25 sull’inferenza Monte Carlo, accessibile.

15 Analisi del grafo bipartito: vedere la struttura

“A network is more than the sum of its nodes.”

Mark E. J. Newman, *Networks: An Introduction* (Newman 2010)

OGNI scuola può essere vista come un *grafo bipartito* che ha per nodi le classi e i docenti, con un arco fra un docente e una classe se il docente insegna in quella classe (eventualmente pesato dal numero di ore). Questo grafo non è un'astrazione gratuita: ne emergono *misure quantitative* che dicono molto sulla struttura organizzativa della scuola e sulla difficoltà del problema di scheduling.

In π Tantum la pagina Diagnostica espone tre misure classiche della network analysis: *modularità*, *betweenness centrality* e *densità*. Ciascuna ha un significato preciso e un suggerimento pratico associato.

15.1 Modularità: quanto è “a cluster” la scuola

Nota storica

La modularità è una misura introdotta da Newman e Girvan nel 2004 (Newman e Girvan 2004) per quantificare la *community structure* di un grafo: è alta quando il grafo si divide naturalmente in gruppi (comunità) ben separati, è bassa o negativa quando il grafo è omogeneo. È diventata una delle misure più usate nella scienza delle reti.

Definizione

Per un grafo G con m archi e una partizione dei nodi in comunità $C_1, \dots, C_k$, la modularità è:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

dove A_{ij} è la matrice di adiacenza, k_i è il grado del nodo i , $\delta(c_i, c_j) = 1$ se i e j sono nella stessa comunità, o altrimenti.

Come si legge: per ogni coppia di nodi nella stessa comunità, si confronta il *numero effettivo* di archi fra loro (A_{ij}) con il *numero atteso* se gli archi fossero distribuiti casualmente preservando i gradi ($k_i k_j / 2m$). Se i nodi della stessa comunità sono *più connessi* di quanto ci si aspetterebbe a caso, Q è positiva. Valori tipici: $Q < 0.3$ (struttura debole), $0.3 < Q < 0.5$ (struttura moderata), $Q > 0.5$ (struttura marcata).

Esempio

La scuola di fig. 10.1 (cap. 10) ha modularità $Q \approx 0.42$ (struttura moderata: due cluster ben separati ma con un ponte). Una scuola con un solo indirizzo e docenti che insegnano un po' ovunque ha tipicamente $Q < 0.2$.

Suggerimento pratico: se $Q > 0.4$, la decomposizione spettrale (cap. 10) funzionerà bene; la pipeline può essere accelerata risolvendo cluster indipendenti. Se $Q < 0.2$, la decomposizione produrrà cluster artificiali e conviene risolvere monoliticamente.

15.2 Betweenness centrality: i “ponti” del sistema

Nota storica

La *betweenness* fu introdotta da Linton Freeman nel 1977 (Freeman 1977) nella sociologia dei piccoli gruppi: misura quanto un individuo è “intermediario” nelle comunicazioni del gruppo. Si è poi diffusa in tantissimi campi (citation networks, Internet, biology).

Definizione

Per un nodo v , la *betweenness centrality* è:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

dove σ_{st} è il numero di shortest path da s a t , $\sigma_{st}(v)$ il numero di quelli che passano per v .

Come si legge: $C_B(v)$ è alta se v è “in mezzo” a molti percorsi del grafo. Un docente con betweenness alta è quello che, se si toglie, “sconnette” parecchio il sistema. Tipicamente sono docenti che insegnano in più indirizzi diversi, fungendo da ponte fra cluster.

Suggerimento pratico: i top- k docenti per betweenness sono quelli su cui il timetabling è più fragile. Una loro indisponibilità aggiuntiva ha alta probabilità di rendere il problema infeasible. Vale la pena tenerli sotto controllo durante l’anno.

15.3 Densità: quanto è “pieno” il grafo

Definizione

La *densità* di un grafo bipartito con n_C classi e n_D docenti, che ha $|E|$ archi, è:

$$\rho = \frac{|E|}{n_C \cdot n_D}$$

Vale fra 0 (nessun arco) e 1 (tutti i docenti insegnano in tutte le classi). Per scuole italiane reali, ρ è tipicamente 0.05–0.15.

Suggerimento pratico: ρ alta significa molti docenti che insegnano in molte classi (es. ITIS dove i docenti coprono più indirizzi). π Tantum tipicamente risolve in più tempo le scuole con ρ alta perché i vincoli fra docenti diversi sono più intrecciati. Valori $\rho > 0.3$ sono considerati “densi” e suggeriscono di *non* usare la decomposizione spettrale (vedi sopra).

15.4 Visualizzazione e interpretazione

In Diagnostica π Tantum mostra:

- Una *matrice riassuntiva* con Q , ρ , numero di componenti connesse, top-5 docenti per betweenness.
- Un *istogramma* dei gradi (quante classi per docente; quanti docenti per classe).
- Un *heatmap* della matrice di adiacenza con righe ordinate per cluster spettrale: a colpo d’occhio si vedono i cluster e i ponti.

Caso di studio

Liceo “Manzoni”, 24 classi. $Q = 0.51$, $\rho = 0.09$, top docente per betweenness: prof Esposito (insegna Ao26 in liceo classico e in liceo scientifico, fa da ponte fra i due). Modularità alta $\rightarrow$ decomposizione spettrale produce 3 cluster (classico, scientifico, linguistico) ben separati. Esposito è l'unico ponte significativo; basta tenerlo come “shared” fra due cluster con vincolo lagrangiano. Pipeline completa in 4 min.

15.5 Connessioni

Connessioni

- **Decomposizione spettrale** (cap. 10): la modularità Q predice la qualità della decomposizione.
- **Hall pre-check** (cap. 12): ρ alta in pratica significa molti vincoli di matching da verificare; Hall richiede più tempo.
- **Monte Carlo** (cap. 14): i docenti con alta betweenness sono i candidati naturali per le perturbazioni del Monte Carlo (testare cosa succede se diventano indisponibili).

Per approfondire

- Newman e Girvan 2004: il paper originale sulla modularità (Newman & Girvan).
- Freeman 1977: il paper originale sulla betweenness.
- Newman 2010: *Networks: An Introduction*, manuale moderno e accessibile per la network analysis.

16 Statistica applicata: leggere i risultati

“Statistics is the grammar of science.”

Karl Pearson, attribuito

DOPO aver eseguito una pipeline (o un batch Monte Carlo), il coordinatore vuole sapere: la soluzione è bilanciata? alcune classi sono sistematicamente svantaggiate? c'è una correlazione fra qualcosa che ho dato in input e la qualità del risultato?

Per rispondere π Tantum integra una piccola toolbox di statistica applicata, accessibile dalla pagina Diagnostica. Le tre famiglie sono: *distribuzioni e istogrammi*, *correlazioni e regressioni*, *test di ipotesi* (chi-quadro, Kolmogorov-Smirnov).

16.1 Distribuzioni: vedere la forma dei dati

Definizione

Un *istogramma* di una variabile X è una funzione che, suddiviso il range di X in b *bin* di larghezza uguale, conta quanti valori cadono in ognuno. È una stima non parametrica della densità di X .

In π Tantum si possono richiedere istogrammi di:

- Ore di lezione per docente (mostra se i carichi sono distribuiti uniformemente o se ci sono outlier).
- “Buchi” per docente nella settimana (ore vuote fra lezioni).
- Slot per materia per ora del giorno (mostra l'effetto della preferenza “mattutino”).
- Tempo di esecuzione per cluster nella decomposizione spettrale.

16.1.1 Bin sizing

π Tantum usa la regola di Freedman-Diaconis come default:

$$h = 2 \cdot \text{IQR}(X) \cdot n^{-1/3}$$

dove IQR è l'*interquartile range* ($Q_3 - Q_1$) e n il numero di osservazioni. Il numero di bin è $\lceil (\max - \min)/h \rceil$. L'utente può sempre forzare un numero specifico.

La scelta del numero di bin è un piccolo problema in sé: troppi pochi $\rightarrow$ perdi struttura; troppi $\rightarrow$ rumore.

16.1.2 Test di Kolmogorov-Smirnov

Definizione

Il *test KS* confronta una distribuzione empirica con una distribuzione di riferimento (o due distribuzioni empiriche fra loro). La statistica del test è:

$$D = \sup_x |F_n(x) - F(x)|$$

ovvero la massima distanza verticale fra la CDF empirica F_n e la CDF di riferimento F .

In π Tantum il test KS è usato ad esempio per confrontare la distribuzione delle ore-per-docente fra “baseline” e “post-perturbazione Monte Carlo”. Un valore D piccolo (con p -value alto) indica che la distribuzione è robusta; un D grande indica che la perturbazione ha significativamente alterato il bilancio.

16.2 Correlazioni e regressioni

16.2.1 Correlazione di Pearson

Definizione

La *correlazione di Pearson* fra due variabili X, Y è:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

con $\text{Cov}(X, Y) = E[(X - \bar{X})(Y - \bar{Y})]$. Vale fra -1 e $+1$: $r > 0$ relazione positiva, $r < 0$ negativa, $r \approx 0$ nessuna relazione lineare.

Come si legge: r vicino a ± 1 indica relazione *lineare* forte; r vicino a 0 indica *nessuna relazione lineare*, ma non esclude relazioni non lineari. Un valore di $r = 0.5$ significa: *circa il 25% della varianza di Y è spiegato linearmente da X* ($r^2 = 0.25$).

Esempio

Calcoliamo la correlazione fra “numero di indisponibilità HARD del docente” e “numero di buchi nel suo orario finale”. Se $r = +0.6$, significa che più un docente ha indisponibilità, più tende ad avere buchi. Insight per il coordinatore: i docenti con molte indisponibilità si prendono comunque l'orario peggiore in termini di buchi; forse vale la pena rivedere le loro indisponibilità o accettare di fare strappi sulle preferenze altrui.

16.2.2 Regressione lineare (OLS)

Definizione

Una *regressione lineare ordinaria* (OLS) modella Y come funzione lineare di una o più X :

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

e stima i β minimizzando la somma dei quadrati dei residui: $\hat{\beta} = (X^\top X)^{-1} X^\top Y$.

Come si legge: ogni coefficiente β_i dice di quanto cambia Y se X_i aumenta di 1, tenendo le altre X_j fisse. Il coefficiente β_0 è l'*intercetta*. π Tantum riporta anche R^2 (frazione di varianza spiegata) e i p -value per ciascun β_i .

In π Tantum la pagina “Correlazioni e regressioni” permette di costruire interattivamente il modello (scegli le X , scegli la Y , scegli OLS o Logit) e mostra i coefficienti con intervalli di confidenza.

16.2.3 Regressione logistica (Logit)

Definizione

La *regressione logistica* modella la probabilità di un evento binario:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots)}}$$

Si usa quando la Y è binaria. Esempio in π Tantum: “probabilità che una classe abbia ≥ 3 buchi settimanali in funzione del suo numero di indirizzi serviti e del numero di docenti coinvolti”.

16.3 Chi-quadro: indipendenza

Definizione

Il test *chi-quadro* χ^2 verifica se due variabili categoriali sono indipendenti. Si costruisce la tabella di contingenza, si calcola

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

dove O_{ij} è la frequenza osservata e E_{ij} quella attesa sotto ipotesi di indipendenza. Si confronta con la distribuzione χ^2 con $(r - 1)(c - 1)$ gradi di libertà.

Tipico uso: “il giorno della settimana e la materia sono indipendenti?” Se p-value basso, la materia *influisce* su quale giorno si tiene (es. matematica concentrata al mattino). È un’indicazione che il sistema sta rispettando la preferenza “mattutino”.

16.4 Visualizzazione interattiva

Tutti i grafici in π Tantum sono prodotti con ECharts 6, lazy-loaded sul frontend. Hanno sempre i bottoni:

- **Export PNG** (con pixel-ratio 2 e sfondo bianco).
- **Clear graph**: per fare spazio in finestra.
- **Restore zoom**: per resettare lo zoom dopo esplorazione.

I parametri (numero di bin, scelta delle X , etc.) sono modificabili dall’UI senza dover rilanciare il run; le visualizzazioni sono ricalcolate al volo dal browser.

16.5 Connessioni

Connessioni

- **Monte Carlo** (cap. 14): produce campioni che si analizzano con distribuzioni e KS-test.
- **Analisi del grafo** (cap. 15): la modularità e la betweenness sono input naturali per le regressioni (es. “il tempo di esecuzione del solver dipende dalla modularità del grafo?”).
- **Workflow**: dopo ogni esecuzione il sistema popola automaticamente le tabelle delle statistiche (lezioni, buchi, ore-isolate, etc.); il coordinatore le può esplorare immediatamente.

Per approfondire

- Wasserman 2004: *All of Statistics*, manuale conciso di statistica matematica accessibile agli informatici.
- James et al. 2013: *An Introduction to Statistical Learning*, gratuito online, ottimo per regressioni e modelli predittivi.

17 Il DSL generico: un linguaggio per i vincoli

“A domain-specific language is a computer language specialized to a particular application domain.”

Martin Fowler, *Domain-Specific Languages* (Fowler 2010)

PRIMA o poi tutti i sistemi di scheduling devono affrontare lo stesso problema editoriale: gli utenti vogliono esprimere vincoli che non sono mai stati previsti. “Mai due ore di matematica consecutive nei giovedì pari delle classi prime”; “almeno un’ora di laboratorio per ogni classe di scientifico”; “il prof Bianchi e la prof Verdi mai stesso slot stessa giornata”. È tecnicamente impossibile prevedere tutto e codificarlo hard-coded nel solver.

La soluzione classica è un *Domain-Specific Language*: un piccolo linguaggio dichiarativo che gli utenti possono usare per scrivere i propri vincoli. In π Tantum il DSL si chiama *DSL generico* e lo descriviamo in questo capitolo dal punto di vista implementativo.

17.1 Filosofia: “un parser, molti compilatori”

Intuizione

Il DSL generico ha una grammatica unica e un parser unico, ma ha più *back-end*: lo stesso vincolo si può *valutare* sui dati correnti (per check sincroni o per filtri lista), si può *compilare* a vincoli CP-SAT (per Phase B), oppure si può *compilare* a delta SOFT per il drag-and-drop in tempo reale dell’editor Orario. È lo schema classico “parser $\rightarrow$ AST $\rightarrow$ molti compilatori” del Dragon Book (Aho et al. 2006).

17.2 Grammatica (sintesi)

La grammatica del DSL è costruita con discesa ricorsiva. La forma BNF semplificata:

```
program := constraint+
constraint := quantifier source ('where' filter)? ':' body
quantifier := 'forall' | 'exists' | 'count'
source := IDENT 'in' (IDENT | path)
path := IDENT ('.' IDENT)+
filter := expr
body := expr
expr := orexpr
orexpr := andexpr ('or' andexpr)*
andexpr := notexpr ('and' notexpr)*
notexpr := 'not'? cmp
cmp := value relop value | builtin
relop := '==' | '!=' | '<' | '<=' | '>' | '>='
        | 'in' | 'not in' | 'contains'
value := literal | path | builtin
builtin := 'consecutive' '(' value ',' value ')'
        | 'same_day' '(' value ',' value ')'
```

```
| 'lesson' '(' value ',' value ')'  
| 'has_tag' '(' STRING ')'
```

Le sorgenti supportate sono: teachers, classes, subjects, classrooms, groups, students, lessons, slots; più qualsiasi *path* che restituisca una collezione (es. s.groups, l.classroom.tags).

17.3 Parser e AST

Definizione

Il *parser* è un *recursive descent* hand-rolled (Python puro, no PLY/Lark). Riconosce token con una funzione `tokenize()` basata su regex, e costruisce un AST (*Abstract Syntax Tree*) dove ogni nodo è una piccola dataclass: Quant, Count, BinOp, Cmp, Ref, Lit, Builtin.

Esempio: l'espressione

```
forall t in teachers where t.subject == "Fisica":  
    count l in lessons where l.teacher == t and  
                                l.classroom.type == "lab_fisica":  
        l == 1
```

produce questo AST (semplificato):

```
Quant(kind='forall', var='t', source='teachers',  
      filter=Cmp('==', Ref(['t', 'subject']), Lit('Fisica')),  
      body=Count(var='l', source='lessons',  
                 filter=And([  
                     Cmp('==', Ref(['l', 'teacher']), Ref(['t'])),  
                     Cmp('==', Ref(['l', 'classroom', 'type']),  
                         Lit('lab_fisica'))]),  
                 cmp=Cmp('==', Var('count'), Lit(1))))
```

17.4 Tre back-end

17.4.1 Evaluator: “vale o no?”

Definizione

L'*evaluator* prende un AST e i dati correnti del sistema (teachers, lessons, ecc.) e restituisce True/False per i vincoli forall/exists, e un intero per i count.

È usato in modalità sincrona per i filtri lista (“mostrami tutti i docenti per cui questo vincolo è violato”), per i check pre-pipeline (verifica di consistenza), e come oracolo nei test.

17.4.2 Compilatore CP-SAT

Definizione

Il *compilatore CP-SAT* traduce l'AST in vincoli OR-Tools. Le quantificazioni diventano cicli sui domini; le comparazioni diventano vincoli aritmetici; i count diventano somme con vincoli di uguaglianza/disuguaglianza.

Esempio: `count l in lessons where l.teacher == t: l == 1` diventa

```
total = sum(BoolVar(f"l_{l.id}_active") for l in lessons
            if l.teacher_id == t.id)
model.Add(total == 1)
```

17.4.3 Compilatore delta SOFT

Definizione

Il *compilatore delta* produce, dato un vincolo SOFT e una proposta di mossa (drag-and-drop di una lezione), il *delta* dell'obiettivo (positivo se la mossa peggiora, negativo se migliora) in $O(1)$.

È quello che permette all'editor di mostrare in tempo reale “+0.3 SOFT” o “−0.7 SOFT” mentre l'utente trascina una lezione.

17.5 Estensione: aggiungere un built-in

Per aggiungere un nuovo built-in (es. `distinct_days`):

1. Estendere la lista dei token nel `tokenize()`.
2. Aggiungere il caso nel parser (`_parse_builtin`).
3. Implementare la valutazione nell'evaluator.
4. Implementare la compilazione CP-SAT (di solito 5–10 righe).
5. Aggiungere alla galleria del cap. 20.
6. Aggiungere un test in `test_general_dsl.py`.

17.6 Performance

Intuizione

Il parser è veloce ($\sim 10\mu s$ per espressione tipica da 50 token). L'evaluator è proporzionale al prodotto dei domini (es. $|teachers| \cdot |lessons|$ per il vincolo dell'esempio). Il compilatore CP-SAT genera tipicamente ~ 100 vincoli per ogni vincolo DSL non banale; per scuole grandi, l'aggiunta di 20–30 vincoli DSL non rallenta significativamente la pipeline.

17.7 Roadmap

Funzionalità non ancora implementate ma pianificate:

- **Live validation:** squiggle rosso sotto la parte invalida, suggerimenti fuzzy, quick-fix.
- **Autocomplete:** nel campo DSL, lista delle sorgenti / attributi / built-in con descrizione.
- **Hover docs:** passando il mouse su un identifier, mostra documentazione contestuale.
- **Natural-language preview:** traduzione automatica del vincolo DSL in italiano leggibile.

17.8 Connessioni

Connessioni

- **CP-SAT** (cap. 9): il back-end principale per i vincoli HARD.
- **Editor Orario**: il compilatore delta abilita il drag-and-drop con preview live.
- **Vincoli logici DNF** (cap. 20): il DSL generico è una generalizzazione del vecchio sistema di “vincoli logici disgiuntivi” che ne preserva la semantica come caso particolare.

Per approfondire

- Aho et al. 2006: il *Dragon Book*, riferimento classico per parser, AST, compilatori.
- Fowler 2010: *Domain-Specific Languages*, approccio pratico ai DSL.
- Marriott e Stuckey 1998: *Programming with Constraints*, fondamenti della programmazione a vincoli.

18 Architettura

18.1 Stack

- **Backend:** Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend:** SvelteKit (Svelte 4), Vite 5, Tailwind CSS, JavaScript puro.
- **DB:** SQLite via SQLAlchemy. Migrazioni idempotenti nel codice (no Alembic).
- **Engine I/O:** pickle Python; engine_io.py converte fra DB e dict per il solver.

18.2 Diagramma a blocchi

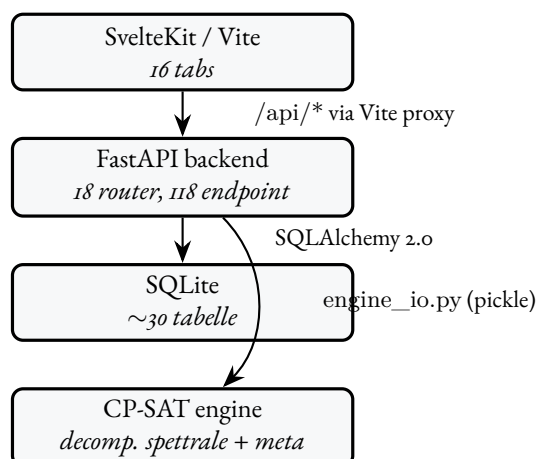


Figura 18.1 – Architettura a blocchi della webapp.

18.3 Struttura cartelle

```
timetable/  
+-- README.md -- landing page  
+-- docs/ -- manuale tecnico  
+-- experiments/ -- solver code + pickle snapshots  
| +-- cpsat_v2_assignment.py -- Phase A  
| +-- cpsat_v2_timetable.py -- Phase B  
| +-- decomposition_spectral_v2.py  
| +-- metaheuristics.py -- LNS / SA / TS / ILS  
| +-- classroom_assignment.py -- Phase 4  
+-- schedule/ -- legacy single-file prototypes  
+-- webui/  
| +-- start.bat -- launcher Windows  
| +-- start.sh -- launcher Linux/macOS  
| +-- stop.sh -- stop helper  
| +-- backend/ -- FastAPI app
```



```
| +-- frontend/ -- SvelteKit
| +-- data/timetable.db -- SQLite
| +-- docs/ -- guide brevi user-facing
+-- proposals/ -- design notes + benchmarks
+-- *.pdf -- reference papers
```

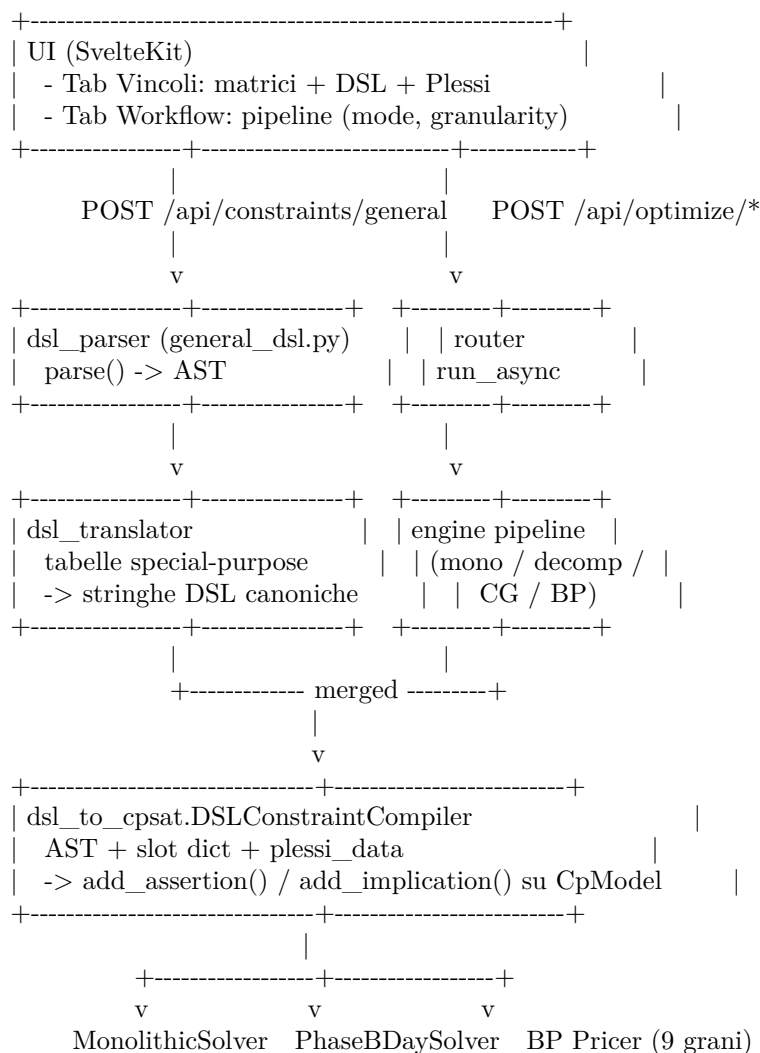
18.4 Lifecycle del backend

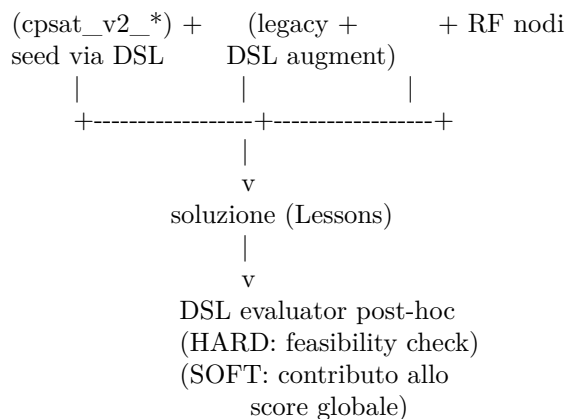
webui/backend/main.py definisce un lifespan che chiama `init_db()` allo startup. Esso fa:

1. `Base.metadata.create_all(bind=engine)` – crea le tabelle mancanti.
2. `_apply_lightweight_migrations()` – ALTER TABLE idempotenti per i campi aggiunti in versioni successive (esempio: `school_classes.curriculum_id`, `teachers.last_name/first_name/nickname`, ecc.). Ogni ALTER è guardato da PRAGMA `table_info` per non duplicare.

18.5 Architettura del solver: dal DSL a CP-SAT

Step 1–4 del piano multi-day hanno introdotto un'architettura *a strati uniforme* per tutti gli usi del solver. Il diagramma sotto riassume i flussi di trasformazione.





I punti chiave:

- **Un parser, un AST.** Tutte le sorgenti (matrici, DSL UI, regole plesso, vincoli legacy seedati) vengono unificate in stringhe DSL canoniche e parsate da `general_dsl.py`.
- **Un compilatore.** Il `DSLConstraintCompiler` è la porta unica verso CP-SAT. Mono solver, pricer di BP, `PhaseBDaySolver`, nodi Ryan-Foster: tutti applicano lo stesso compilatore al loro modello, garantendo coerenza dei vincoli fra le diverse pipeline.
- **Zero-drift.** `seed_implicit_hardcoded(profs)` traduce in DSL i vincoli HARD legacy hardcoded. Le tuple di slot proibite emesse via DSL coincidono bit-per-bit con quelle del path legacy: la migrazione è progressiva e reversibile.
- **SOFT post-hoc, HARD up-front.** I HARD del DSL diventano vincoli CP-SAT prima della solve. I SOFT del DSL sono attualmente valutati *post-hoc*: il loro costo contribuisce allo score globale ma non orienta ancora la ricerca CP-SAT (la wired-up del SOFT in obiettivo è pianificata nei prossimi step).

19 Modello dati

Tutto vive in `webui/backend/models.py`. Le tabelle si raggruppano in cinque famiglie tematiche.

19.1 Famiglie di tabelle

Anagrafiche

`subjects`, `teachers`, `school_classes`, `classrooms`, `curricula`, `students`, `study_groups`.

Tag (M-N)

`classroom_tags` + `classroom_tag_assignments`, `student_tags` + `student_tag_assignments`. Nome unico lower-cased, descrizione opzionale, cancellazione cascata.

Assegnazione

`class_subjects`, `curriculum_subject_hours`, `teacher_subjects`, `subject_group_weights`, `assignments`.

Disponibilità / vincoli

`teacher_unavailability`, `class_unavailability`, `classroom_unavailability`, `logical_unavailabilities`, `curriculum_logical_constraints`, `classroom_subject_preferences`, `teacher_classroom_preferences`, `coteaching_rules`.

Soluzioni e scheduling

`solutions`, `lessons`, `day_counts`.

Coverage e Run

`absences`, `substitute_assignments`, `runs`, `run_logs`, `app_state`.

Viste salvate

`saved_views`: query DSL + sort multi-livello salvati per una entità lista, esposti dal dropdown "Viste salvate" del componente *SortableQueryableList*. UNIQUE (entity, name).

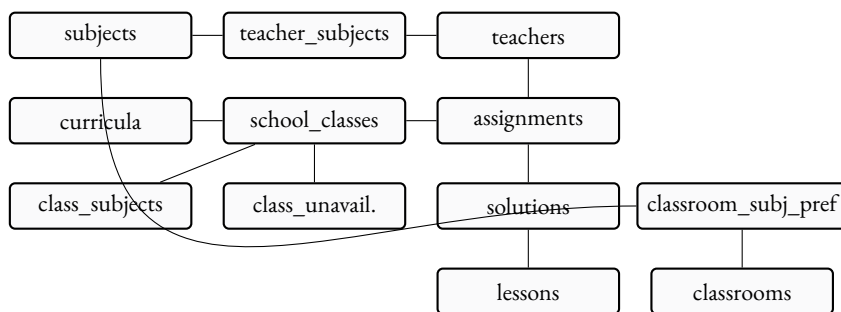


Figura 19.1 – Relazioni rilevanti del modello dati (semplificato).

19.2 Diagramma relazioni

19.3 Migrazioni idempotenti

Pattern (in db.py):

```

if insp.has_table("school_classes") \
    and not has_column("school_classes", "curriculum_id"):
    conn.execute(text(
        "ALTER TABLE school_classes ADD COLUMN curriculum_id INTEGER"
    ))

```

Ogni ALTER è guardato da una check su PRAGMA table_info. Quando aggiungi una colonna nuova, segui lo stesso pattern: il DB preesistente sopravvive senza interventi manuali.

19.4 Pickle dell'engine

engine_io.py converte fra DB e tre forme pickle:

```

school = {
    'profile': str,
    'classes': [{name, year, section, curriculum, subjects:{subj:ore}}],
    'teachers': [{name, group, max_hours, free_day,
                  weights:{subj:int}}],
    'cconcorsopersubject': {subj: {group: weight}},
    'curriculum_scores': {curriculum: int},
    'curricula': [...],
    'students': [...],
    'groups': [...],
}
profs = {
    teacher_name: {
        'classi': {class_name: {subject: {'ore': N}}},
        'glibero': [d1, d2, d3]
    }
}
solution = {(prof, class, subj, day, hour): 0|1}

```

20 DSL generico per i vincoli

Linguaggio compatto, dichiarativo, totalmente componibile per esprimere vincoli HARD/SOFT/PREFERRED/ENFORCED su qualunque combinazione logica di predicati sul modello dei dati. Implementato in `webui/backend/utils/general_dsl.py` (parser ricorsivo discendente + AST + evaluator post-hoc, ~250 LOC, no dipendenze esterne). Esposto via `POST /api/constraints/general`, validato live da `POST /api/constraints/general/validate`, accessibile dal modal “+ Nuovo vincolo DSL” del tab Vincoli.

20.1 Filosofia

Prima del DSL generico il sistema aveva quattro sub-DSL specializzati (query, logical, objective, introspeetivo). Stessa grammatica implementata 4 volte. Il DSL generico unifica tutto sotto una sola grammatica con quattro “sapori” di compilazione (LIST, LOGIC, OBJECTIVE, EVAL): *un parser, molti compilatori*. La sintassi di `forall x in S where Filter: Body` e dei predicati atomici (`=`, `!=`, `<`, `in`, ...) e’ identica in tutti i contesti; cambia solo il backend di traduzione.

Il DSL e’ interpretato *post-hoc*: parsea l’espressione e la valuta DOPO che la pipeline (Phase A, Phase B, metaeuristiche) ha prodotto una soluzione candidata. Le violazioni HARD sono rilevate dal Feasibility Check; le SOFT alimentano lo score globale.

20.2 Grammatica BNF

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr ) *
implies_expr ::= or_expr ( "=>" or_expr ) *
or_expr ::= and_expr ( ("OR"|"or") and_expr ) *
and_expr ::= not_expr ( ("AND"|"and") not_expr ) *
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
      | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
            ["where" or_expr] ":" expr

count_expr ::= "count" IDENT "in" source ["where" or_expr]
            ( ":" comparison | op value )

source ::= IDENT -- literal name
        | IDENT ( "." IDENT ) + -- path (runtime)

comparison ::= value ( op value
                    | "in" "[" value ( "," value ) * "]"
                    | "in" path
                    | "not_in" "[" value ( "," value ) * "]"
                    | "not_in" path )

op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
```

```

value ::= IDENT ( " " IDENT )* | NUM | STRING
        | BOOL | function_call
function_call ::= IDENT "(" [arg_list] ")"
arg_list ::= arg ( " " arg)*
arg ::= IDENT "==" value | value

```

Precedenze (dal più basso al più alto): $\leq$, $\geq$, OR, AND, NOT, confronti, . (member access). Le parentesi sovrascrivono.

20.3 Sorgenti iterabili

Sorgente	Tipo elemento
lessons	lezioni della soluzione attiva
assignments	cattedre (Assignment rows)
teachers	docenti (name, group, max_hours, subject)
classes	classi (name, year, curriculum, n_students)
classrooms	aule (name, kind, type, tags)
students	studenti (con tags[] e groups[])
subjects	materie
curricula	indirizzi di studio
groups	gruppi articolati
slots	le 36 (day, hour) della settimana
days	i 6 giorni (Lun..Sab; index 1..6)
hours	le 6 ore (8..13)

Sorgente-path: dalla v0.5 puoi scrivere `exists g in s.groups: g.name == "X"` dove `s.groups` e' un attributo che restituisce una lista; il parser accetta qualunque path puntato dopo `in`.

20.4 Galleria di esempi

Ogni esempio sotto e' stato validato live contro il parser corrente; copia-incolla nell'editor del modal "+ Nuovo vincolo DSL" e funziona.

20.4.1 Vincoli docente (10 esempi)

1. Borghi non insegna mai in 1A alla 6a ora:

```

forall l in lessons where l.teacher == Borghi
    and l.class == 1A:
    l.hour != 13

```

2. Borghi: max 4 ore/giorno totali:

```

forall d in days:
    count l in lessons where l.teacher == Borghi
        and l.day == d.index: l <= 4

```

3. Lab fisica: ogni docente di Fisica esattamente 1 ora a settimana in lab_fisica:

```

forall t in teachers where t.subject == "Fisica":
    count l in lessons where l.teacher == t
        and l.classroom.type == "lab_fisica": l == 1

```

4. Tetto cattedra Inglese: max 18h/settimana:

```
forall t in teachers where t.subject == "Inglese":
  count l in lessons where l.teacher == t: l <= 18
```

5. Rossi non lavora il sabato (HARD):

```
forall l in lessons where l.teacher == Rossi
  and l.day == 6:
  false
```

6. Rossi: al più 1 sesta ora/settimana:

```
count l in lessons where l.teacher == Rossi
  and l.hour == 13: l <= 1
```

7. Tutti A050: max 5 ore/giorno:

```
forall t in teachers where t.group == "A050":
  forall d in days:
    count l in lessons where l.teacher == t
      and l.day == d.index: l <= 5
```

8. Coupling Rossi 3A $\Rightarrow$ Bianchi 3B:

```
(exists l in lessons:
  l.teacher == Rossi and l.class == 3A)
=> (exists l in lessons:
  l.teacher == Bianchi and l.class == 3B)
```

9. Rossi mai in palestra:

```
forall l in lessons where l.teacher == Rossi:
  l.classroom.type != "palestra"
```

10. Tutti i prof di mate: almeno una lezione di mattina:

```
forall t in teachers where t.subject == "Matematica":
  exists l in lessons where l.teacher == t:
    l.hour <= 9
```

20.4.2 Vincoli classe (5 esempi)

1. La 1A non lavora il sabato:

```
forall l in lessons where l.class == 1A: l.day != 6
```

2. Le prime: max 5 ore/giorno:

```
forall c in classes where c.year == 1:
  forall d in days:
    count l in lessons where l.class == c.name
      and l.day == d.index: l <= 5
```

3. Quarte/quinte: due ore CONSECUTIVE di mate:

```
forall c in classes where c.year >= 4:
  exists s1 in slots: exists s2 in slots:
    consecutive(s1, s2)
    and lesson(class==c.name, day==s1.day,
               hour==s1.hour, subject==Matematica)
    and lesson(class==c.name, day==s2.day,
               hour==s2.hour, subject==Matematica)
```

4. Linguistico anno 1: niente sabato:

```
forall l in lessons
  where l.class.curriculum == Linguistico
  and l.day == 6: false
```

5. 1A: religione solo nelle prime 4 ore:

```
forall l in lessons where l.class == 1A
  and l.subject == Religione:
  l.hour <= 11
```

20.4.3 Vincoli aula (5 esempi)

1. Lab Fisica 1: una sola lezione per slot:

```
forall s in slots:
  count l in lessons
    where l.classroom == "Lab Fisica 1"
    and l.day == s.day and l.hour == s.hour:
  l <= 1
```

2. Matematica solo in aule taggate “matematica”:

```
forall l in lessons where l.subject == Matematica:
  "matematica" in l.classroom.tags
```

3. Storia delle terze: aula con proiettore:

```
forall l in lessons where l.subject == Storia
  and l.class.year >= 3:
  "proiettore" in l.classroom.tags
```

4. La palestra ospita SOLO Scienze motorie:

```
forall l in lessons
  where l.classroom.type == "palestra":
  l.subject == Scienzemotorie
```

5. Lab chimica: max 2 lezioni concorrenti:

```
forall s in slots:
  count l in lessons
    where l.classroom.type == "lab_chimica"
    and l.day == s.day and l.hour == s.hour:
  l <= 2
```


20.4.4 Vincoli materia (3 esempi)

1. Mate distribuita: max 2 ore/giorno per classe:

```
forall c in classes:
  forall d in days:
    count l in lessons
      where l.class == c.name
        and l.subject == Matematica
        and l.day == d.index: l <= 2
```

2. Mate preferibilmente entro la 5a ora:

```
forall l in lessons where l.subject == Matematica:
  l.hour <= 12
```

3. Educazione fisica solo in palestra:

```
forall l in lessons
  where l.subject == Educazione fisica:
    l.classroom.type == "palestra"
```

20.4.5 Vincoli gruppo / studenti (3 esempi)

1. Studenti BES devono essere in un gruppo Sostegno (uso di exists g in s.groups):

```
forall s in students where "BES" in s.tags:
  exists g in s.groups: g.name == "Sostegno"
```

2. Studenti con debito mate quarto → gruppo Recupero:

```
forall s in students
  where "debito_matematica_4" in s.tags:
    exists g in s.groups:
      g.name == "Recupero Matematica"
```

3. Studenti BES: nessuna lezione il sabato:

```
forall l in lessons:
  forall s in students
    where l.class == s.class
      and "BES" in s.tags:
        l.day != 6
```

20.4.6 Vincoli relazionali / coupling (4 esempi)

1. Rossi in Aula 12: solo Matematica:

```
forall l in lessons where l.teacher == Rossi
  and l.classroom == "Aula 12":
  l.subject == Matematica
```

2. Se Rossi in 3A allora deve usare lab fisica almeno una volta:

```
(exists l in lessons:
  l.teacher == Rossi and l.class == 3A)
=>
(exists l2 in lessons:
  l2.teacher == Rossi and l2.class == 3A
  and l2.classroom.type == "lab_fisica")
```

3. Inglese alle prime: max 3 ore/sett per (docente,classe):

```
forall t in teachers where t.subject == "Inglese":
  forall c in classes where c.year == 1:
    count l in lessons
      where l.teacher == t
      and l.class == c.name: l <= 3
```

4. Religione alle prime: mai in 1a ora:

```
forall l in lessons where l.subject == Religione
  and l.class.year == 1:
  l.hour != 8
```

20.5 Scope: dove salvare il vincolo

Il modal “+ Nuovo vincolo DSL” compare in due punti dell’app e produce regole con scope diverso. La regola di selezione è: lo scope è il dominio della regola, il forall sul corpo è il filtro di applicazione.

- **Tab Vincoli** (/constraints, modal con scope=”global”, owner__id=null). Vincolo d’istituto: si applica a tutta la scuola. Esempi: “nessuno alla 14a ora”, “mai lezioni il sabato pomeriggio”.
- **Tab Docente** (/teachers/<id>, scheda “Custom DSL avanzato”, scope=”teacher”, owner__id=teacher__id). Vincolo per docente: il forall esterno è *tipicamente* pre-filtrato su l.teacher == NomeDocente. Esempi: “Rossi non venerdì pomeriggio”, “Rossi: 3 ore Fisica su giorni non consecutivi”.

In entrambi i casi la regola passa per POST /api/constraints/general, viene caricata da load_all_dsl_constraints(db) e compilata da add_all_dsl_constraints_from_db; il campo scope_kind è meramente informativo (filtra le regole in Feasibility Check + reporting).

20.5.1 DSL avanzato vs unavailability logica

Le LogicalUnavailability (modal “Indisponibilità logica” nel tab Docente) sono un sotto-DSL più semplice: sono pensate per non-disponibilità (DNF di slot (day, hour)) e vengono renderizzate dal sistema in DSL generico via logical_unavailability_to_dsl. Il DSL generico è più espressivo (count, exists, forall, predicati same_day / consecutive_days, aritmetica, l.classroom.plesso) e dovrebbe essere il punto di ingresso per ogni vincolo che vada oltre “il docente non c’è a quell’ora”.

20.6 Predicati built-in: consecutive_days, same_day, consecutive

Tre funzioni built-in che il compilatore risolve a tempo di compilazione (e l’evaluator post-hoc valuta sulla soluzione):

Predicato	Semantica
<code>same_day(s1, s2)</code>	i due slot cadono nello stesso giorno della settimana. Argomenti: tuple (d, h) oppure l.slot.
<code>consecutive(s1, s2)</code>	stesso giorno e ore adiacenti ($ h_1 - h_2 = 1$). Path: l.slot.
<code>consecutive_days(d1, d2)</code>	<i>giorni</i> adiacenti senza wrap-around: vero per $ d_1 - d_2 = 1$ con $d \in \{1, \dots, 6\}$ (lun..sab). Falso per {sab, lun}. Argomenti: l.day (intero) o literal numerico/stringa giorno.

20.7 Aritmetica nel DSL

Il parser supporta gli operatori binari $+$ e $-$ sugli interi: utili per esprimere finestre dipendenti dall'indice giorno o per conteggi cumulativi. Esempio: “Rossi ha tante ore in 3A nei primi 3 giorni quante in 3B nei restanti 3 giorni”:

```
(count l in lessons where l.teacher == "Rossi"
 and l.class == "3A" and l.day <= 3: 1)
== (count l in lessons where l.teacher == "Rossi"
 and l.class == "3B" and l.day >= 4: 1)
```

L'aritmetica fra booleani e numeri non è ammessa: il parser emette `aritmetica non valida fra bool e numero` per catturare errori del tipo `(l.hour == 9) + 1`.

20.8 Casi d'uso reali (Rossi e dintorni)

Tutti gli esempi che seguono sono coperti dalla suite di integration test in `webui/backend/tests/test_rossi_general_dsl_integration.py` e `webui/backend/tests/test_global_dsl_and_soft_and_plesso_commute.py`.

20.8.1 Caso (a) – 3 ore di Fisica in 3A su giorni non consecutivi (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
 and l1.class == "3A"
 and l1.subject == "Fisica":
  forall l2 in lessons where l2.teacher == "Rossi"
    and l2.class == "3A"
    and l2.subject == "Fisica":
    (l1.day == l2.day)
    or not consecutive_days(l1.day, l2.day)
```

Salvato dal tab Docente con `scope="teacher"`, `owner_id=Rossi.id`. Il compilatore apre il `forall` esterno sui candidati slot di Rossi/3A/Fisica e per ogni coppia (l_1, l_2) con `consecutive_days(l1.day, l2.day)` e diversa day emette `BoolOr([slot[k1].Not(), slot[k2].Not()])`: i due slot non possono essere co-selezionati.

20.8.2 Caso (b) – 3 ore di Fisica in 3C tutte lo stesso giorno (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
 and l1.class == "3C"
 and l1.subject == "Fisica":
  forall l2 in lessons where l2.teacher == "Rossi"
    and l2.class == "3C"
    and l2.subject == "Fisica":
    same_day(l1.slot, l2.slot)
```

Per la proprietà transitiva del `same_day` sul `forall forall`, ogni coppia di lezioni di Rossi/3C/Fisica deve essere sullo stesso giorno: il solver sceglie un giorno e ci infila tutte e 3 le ore.

20.8.3 Caso plesso commute – Rossi mai A@9 → B@10 nello stesso giorno (HARD)

```
forall l1 in lessons where l1.teacher == "Rossi"
    and l1.hour == 9
    and l1.classroom.plesso == 1:
    forall l2 in lessons where l2.teacher == "Rossi"
        and l2.hour == 10
        and l2.classroom.plesso == 2
        and same_day(l1.slot, l2.slot):

false
```

Plesso 1 = “A”, plesso 2 = “B” (gli ID interi di plessi.id). Funziona compile-time quando il modello è costruito con `classroom_for_slot` (es. nel solver di `classroom-assignment`) o con plesso fisso per classe; in tutti gli altri casi è pienamente verificabile post-hoc dal Feasibility Check.

20.8.4 Caso (Rossi non Fisica venerdì) (HARD)

```
forall l in lessons where l.teacher == "Rossi"
    and l.subject == "Fisica"
    and l.day == 5:

false
```

Il body *static* `false` è l’idioma canonico per “proibire qualunque slot che soddisfa il where”. Equivalente a `l.day != 5` ma più esplicito quando il where è multi-clausola.

20.8.5 Caso (compresenza supplementare nello stesso giorno) (HARD)

“Quando Rossi insegna Sostegno in 2B, la lezione di Sostegno deve cadere nello stesso giorno di una lezione di Rossi in 2B con materia Matematica”:

```
forall l in lessons where l.teacher == "Rossi"
    and l.class == "2B"
    and l.subject == "Sostegno":
    exists m in lessons where m.teacher == "Rossi"
        and m.class == "2B"
        and m.subject == "Matematica"
        and same_day(l.slot, m.slot):

true
```

20.8.6 Tutti i casi precedenti come SOFT

Cambia `level=hard` in `level=soft` nel modal e imposta `weight=W`. Il solver riceve la regola come penalità W per coppia/lezione violante; quando una soluzione HARD-feasible che la rispetta esiste, il solver la sceglie per primo (perché ha costo aggiuntivo zero); altrimenti accetta la violazione minimizzando il numero di coppie/lezioni che pagano W .

20.9 Errori comuni

- Atteso COLON, trovato OP (=) – usa == nei confronti dopo where (= singolo e’ lecito, ma confonde il parser quando appare in contesti ambigui).
- sorgente sconosciuta ‘foo’ – sorgente non in `__VALID_SOURCES` e non e’ un path raggiungibile via env. Controlla l’ortografia.

- oltre 1000000 atomi: probabile esplosione combinatoria – quantificatori troppo annidati. Sposta filtri nel where più esterno per ridurre il set; oppure splitta il vincolo in più vincoli salvati separatamente.

Vedere docs/general_dsl.md per la guida completa con sezioni 1–12 (galleria di 30+ esempi commentati, troubleshooting esteso, reference tabellare di tutti gli attributi per ciascuna entità).

20.10 Compilazione DSL → CP-SAT (Step 1)

A partire da Step 1 del piano multi-day, il DSL ha un *compilatore* verso CP-SAT (engine/dsl_to_cpsat.py, classe DSLConstraintCompiler). Mentre l’evaluator post-hoc guarda una soluzione già costruita per dichiararla feasible o violata, il compilatore aggiunge i vincoli *durante* la costruzione del modello CP-SAT, in modo che il solver eviti strutturalmente le violazioni.

Il compilatore è usato in tre punti del sistema:

- dal MonolithicSolver (cap. 9) quando il flag via_dsl=True è attivo;
- dai pricer di Branch-and-Price (cap. 21) per applicare gli stessi vincoli al sotto-problema;
- da PhaseBDaySolver quando si invoca con via_dsl=True per riprodurre la pipeline legacy affiancata dal DSL.

Tutte le sorgenti delle lezioni già note (teachers, classes, slots, ecc.) sono accessibili al compilatore allo stesso modo. La differenza chiave è che quando una variabile slot è ancora indecisa, il compilatore emette il vincolo CP-SAT corrispondente al posto di valutare una verità; quando un’espressione è *completamente statica* (per esempio consecutive(s1, s2) con s1 e s2 legati staticamente nel forall) la valutazione viene eseguita a tempo di compilazione e si emette un vincolo solo se serve.

20.11 Predicati di slot: consecutive, same_day

Step 3a aggiunge due predicati built-in che il compilatore risolve a tempo di compilazione quando entrambi gli argomenti sono slot legati staticamente:

- same_day(s1, s2) è vero se i due slot cadono nello stesso giorno della settimana.
- consecutive(s1, s2) è vero se i due slot cadono nello stesso giorno e differiscono di un’ora ($|h_1 - h_2| = 1$).

Sono i mattoni con cui si esprimono “ore consecutive”, “stacco fra plessi”, “coppia di ore quando ce ne sono due nel giorno”, “compresenza che deve avvenire nello stesso giorno”. Gli argomenti accettati sono o tuple letterali (d, h) oppure path puntati come l.slot che restituiscono una (day, hour).

Esempio: una classe deve avere mate in tre ore consecutive in *qualche* giorno della settimana:

```
exists s1 in slots: exists s2 in slots: exists s3 in slots:
  consecutive(s1, s2) and consecutive(s2, s3)
  and lesson(class==3B, day==s1.day,
             hour==s1.hour, subject==Matematica)
  and lesson(class==3B, day==s2.day,
             hour==s2.hour, subject==Matematica)
  and lesson(class==3B, day==s3.day,
             hour==s3.hour, subject==Matematica)
```

20.12 Path l.classroom.plesso

Ancora in Step 3a, l'evaluator post-hoc e il compilatore risolvono il path chained l.classroom.plesso: per ogni lezione l, l'aula assegnata viene letta dalla mappa classroom_for_slot fornita da Phase B / Phase classroom-assignment, e il plesso viene risolto via la tabella plessi_data. Senza queste due strutture il path ritorna None e il compilatore emette un diagnostico.

L'esempio canonico è una commuting rule fra plessi (vedi sez. 5.14): “nessun docente può avere lezione in Plesso B nell'ora successiva a una lezione in Plesso A”:

```
forall l1 in lessons where l1.classroom.plesso == 1:
  forall l2 in lessons where l2.classroom.plesso == 2
    and l2.teacher == l1.teacher
    and consecutive(l1.slot, l2.slot):
      false
```

20.13 Vocabolario di pattern canonici (Step 3b)

Step 3b introduce un *vocabolario* di chiamate di funzione che il compilatore riconosce in modo speciale ed emette direttamente come gruppi compatti di vincoli CP-SAT. Ognuna di queste forme corrisponde a un pattern frequente che, scritto in DSL puro a colpi di forall, produrrebbe centinaia o migliaia di clausole. La forma canonica è zero-drift rispetto all'encoding hardcoded del solver legacy.

Pragma	Equivalente legacy
no_holes_class("3B")	catena non-crescente di booleani che vieta i buchi nell'orario settimanale della 3B (legacy add_class_no_holes)
class_present_at_hour("3B", 11)	se la 3B ha una lezione in un giorno, lo slot all'ora 11 deve essere occupato (legacy HARD-3 / “uscita dopo le 12”)
class_day_load_in("3B", 0, 4, 5, 6)	in un giorno la 3B ha 0 ore (giorno libero) o tra 4 e 6 ore (legacy HARD-1+HARD-2)
teacher_max_per_day("Rossi", 5)	Rossi ha al massimo 5 ore in un giorno (legacy MAX_PROF_HOURS_PER_DAY)
cattedra_max_per_day("Rossi", "3B", "Matematica", 2)	la cattedra (Rossi, 3B, Mate) ha al massimo 2 ore in un giorno (legacy MAX_PER_DAY_TRIPLE)
subject_pair_must("3B", "Scienzemotorie")	quando 3B ha 2 ore di motoria nello stesso giorno, devono essere consecutive (legacy HARD-B / add_motorie_pair)
subject_pair_exists("3B", "Matematica")	quando 3B ha 2+ ore di mate nello stesso giorno, almeno una coppia di consecutive deve esistere (legacy HARD-A / add_math_italian_pair)

Inoltre il pragma no_holes_all_classes() e class_present_at_hour_all(11) agiscono in batch su tutte le classi nel *slot scope* corrente, utile da incollare in cima alla lista di vincoli senza dover enumerare i nomi.

20.14 Plesso commuting via DSL (Step 3c)

Step 3c collega le PlessoCommutingRule al DSL: la helper plesso_commuting_rule_to_dsl(rule) produce una lista di clausole DSL equivalenti alla regola legacy. La property zero-drift è verificata nella suite di test: applicare la regola via DSL produce esattamente le stesse combinazioni proibite della pipeline hardcoded plessi_constraints.

20.15 Seed legacy HARD via DSL (Step 3d-3e)

Step 3d-3e introduce `seed_implicit_hardcoded(profs)` che, dato il dizionario delle cattedre, restituisce una lista di stringhe DSL che rappresentano TUTTI i vincoli HARD hardcoded del solver legacy. Combinato con il flag `via_dsl=True` di `MonolithicSolver` e `PhaseBDaySolver`, permette di costruire un modello CP-SAT completo *esclusivamente dal DSL*: nessun ramo di codice hardcoded è attivo. Le invarianti che si garantiscono:

- zero-drift sintattico: le tuple di slot proibite/forzate emesse dal path DSL coincidono bit-per-bit con quelle del path legacy;
- zero-drift semantico: le soluzioni feasible accettate dai due path sono identiche, verificato sui benchmark small/medium/huge;
- determinismo: l'ordinamento delle stringhe DSL prodotte da `seed_implicit_hardcoded` è stabile, quindi l'ordine di inserimento dei vincoli nel modello CP-SAT è riproducibile run-to-run.

Questo è il *vehicle* per la migrazione progressiva: nei prossimi step il path hardcoded verrà deprecato a favore del path DSL, e l'esistenza di `seed_implicit_hardcoded` permette di farlo senza perdere la copertura dei vincoli.

20.16 Vincoli SOFT con peso

Il DSL accetta quattro livelli: `hard`, `soft`, `preferred`, `enforced`. A livello di *evaluator post-hoc* il SOFT è pienamente implementato: le violazioni contribuiscono allo score globale della soluzione e sono visibili nelle metriche di `run_metrics`.

Dal commit `feat(dsl): soft level for general_dsl` il *compilatore CP-SAT integra* il SOFT direttamente nell'obiettivo del solver. Il path è:

1. L'utente sceglie `level=soft` e un peso intero W (es. 50) nel modal di creazione.
2. `load_all_dsl_constraints(db, include_soft=True)` restituisce la regola con `is_hard=False`, `weight=W`.
3. `add_dsl_constraint(expr, is_hard=False, soft_weight=W)` compila la regola: ogni candidato `slot[k]` che violerebbe il body emette $(W, \text{slot}[k])$ in `soft_cost_terms` invece di `slot[k] == 0`; nei doppi `forall` viene reificata una `BoolVar b` con `b == s1 && s2` e si emette (W, b) .
4. `MonolithicSolver.build()` somma `dsl_soft_cost_terms` negli `obj_terms` prima del `Minimize(sum(obj_terms))`.

Semantica. Una violazione *forall* costa W per lezione che cade nella cella vietata; una violazione *forall-forall* costa W per coppia co-selezionata. Pesi negativi sono accettati come bonus PREFERRED.

Esempio. “Rossi non dovrebbe avere ore consecutive in 3A in giorni successivi” (penalità 50 per coppia):

```
forall l1 in lessons where l1.teacher == "Rossi"
    and l1.class == "3A":
    forall l2 in lessons where l2.teacher == "Rossi"
        and l2.class == "3A":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)
```

Salvato come `level=soft`, `weight=50`, `scope=global`: il solver può piazzare due ore in martedì+mercoledì pagando 50 alla volta, ma se esiste una soluzione con martedì+giovedì (non consecutive) la sceglierà.

20.17 Le quattro famiglie di compilazione

Una sola grammatica, quattro *flavour* di compilazione. La distinzione non è accademica: ognuna nasce da una domanda operativa diversa e produce un manufatto diverso.

Famiglia	Domanda operativa	Output	Modulo Python
LIST	“elenca tutte le lezioni che soddisfano φ ”	lista di Lesson	utils/dsl_query.py
LOGIC	“è feasible questa configurazione?”	boolean (DNF/CNF)	utils/dsl_logic.py
OBJECTIVE	“quanto costa la violazione?”	espressione lineare CP-SAT	engine/dsl_to_cpsat.py
EVAL	“conta le violazioni in questa soluzione”	dict {rule_id: cost}	utils/general_dsl.py

Tabella 20.1 – Quattro famiglie di compilazione, una sola grammatica. Storicamente erano quattro sub-DSL separati. Dalla v0.4 il parser è unico (general_dsl.py) e produce un AST condiviso da cui ciascuna famiglia estrae ciò che le serve.

Perché quattro e non un solo backend universale

Un backend universale dovrebbe essere capace di rispondere a tutte e quattro le domande *insieme*. Ma le quattro domande hanno aspettative di latenza incompatibili: LIST e EVAL devono rispondere in millisecondi (servono al frontend e ai test), OBJECTIVE deve produrre un modello che il solver ingoia in centinaia di millisecondi, LOGIC è un check di feasibility che non passa per il solver. Quattro back-end specializzati su un AST condiviso è il compromesso che permette test rapidi e modello pulito.

20.18 Logica DNF vs forall/count: due famiglie a confronto

Il sotto-DSL LogicalUnavailability (presente fino al v0.3, ancora in uso nel tab Docente come “Indisponibilità logica”) esprime un sottoinsieme della grammatica generica: solo *disgiunzioni di slot proibiti* con scope il singolo docente. Il general DSL può sempre simularlo, l'inverso non sempre.

Espressività	LogicalUnavailability (DNF)	general_dsl
slot proibito	•	•
slot vietato condizionata- mente	limitato a AND di slot	• ($=>$, $<=>$)
quantificatori forall x in S	—	•
count con limite	—	•
predicati	—	•
same_day/consecutive	—	•
path l.classroom.plesso	—	•
aritmetica intera	—	• (Step 3)
SOFT con peso	—	•

Tabella 20.2 – La DNF logica è un *subset* della grammatica generale. Migrare un vincolo dalla forma DNF alla forma generale è sempre possibile (helper logical_unavailability_to_dsl); migrare nell'altro verso richiede un'abstraction del vincolo che spesso non esiste.

Lo stesso vincolo, due grammatiche

Vincolo: “Rossi non insegna mai venerdì alle ultime due ore.”
DNF (LogicalUnavailability):

```
(day == 5 AND hour == 12) OR (day == 5 AND hour == 13)
```


General DSL (equivalente):

```
forall l in lessons where l.teacher == Rossi
                        and l.day == 5
                        and l.hour >= 12:
    false
```


La forma generale legge come una frase italiana e si estende banalmente: cambiando l.hour >= 12 in l.hour >= 11 estendiamo a tre ore senza dover scrivere tre clausole DNF.

20.19 Quattordici pragma canonici per la Phase A e la Phase B

Step 3b ha estratto da solve_phase_a e da solve_phase_b_for_day le sequenze più frequenti di vincoli e le ha ribattezzate *pragma*: chiamate di funzione di forma standard che il compilatore riconosce e per cui emette gruppi compatti di vincoli CP-SAT, byte-per-byte identici a quelli che il path legacy emetteva.

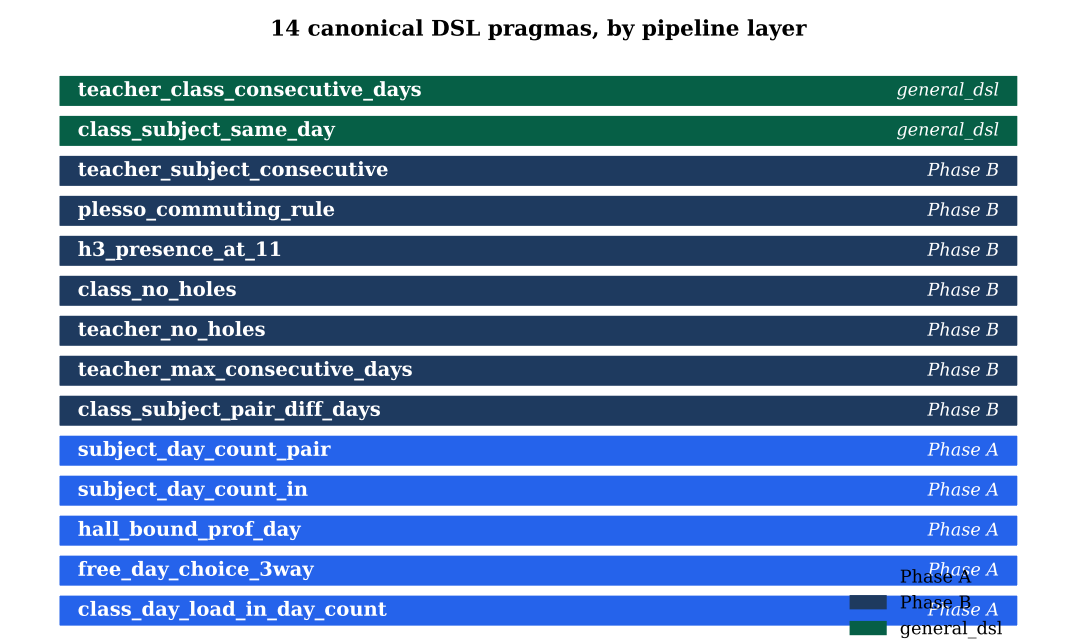


Figura 20.1 – I 14 pragma canonici, raggruppati per layer della pipeline in cui sono compilati. I pragma di Phase A sono i mattoni di DayCountModel; quelli di Phase B alimentano PhaseBDaySolver; le ultime due forme (class_subject_same_day e teacher_class_consecutive_days) sono espresse direttamente dalla famiglia general_dsl senza un canonical helper ad hoc.

Numeri in sintesi

- Pragma totali in Phase A: **5** (class_day_load_in_day_count, free_day_choice_3way, hall_bound_prof_day, subject_day_count_in, subject_day_count_pair).
- Pragma totali in Phase B: **7** (class_subject_pair_diff_days,

```
teacher_max_consecutive_days,      teacher_no_holes,      class_no_holes,
h3_presence_at_11, plesso_commuting_rule, teacher_subject_consecutive).
```

- Pragma espressi solo via `general_dsl` libero: **2+**.
- Test di copertura sui pragma: `webui/backend/tests/test_dsl_canonical_pragmas.py` + `test_dsl_intvar_pragmas.py` (~ 42 funzioni di test totali).

20.20 Workflow: dal tab Docente al solver

DSL pipeline: from UI to four compiler families

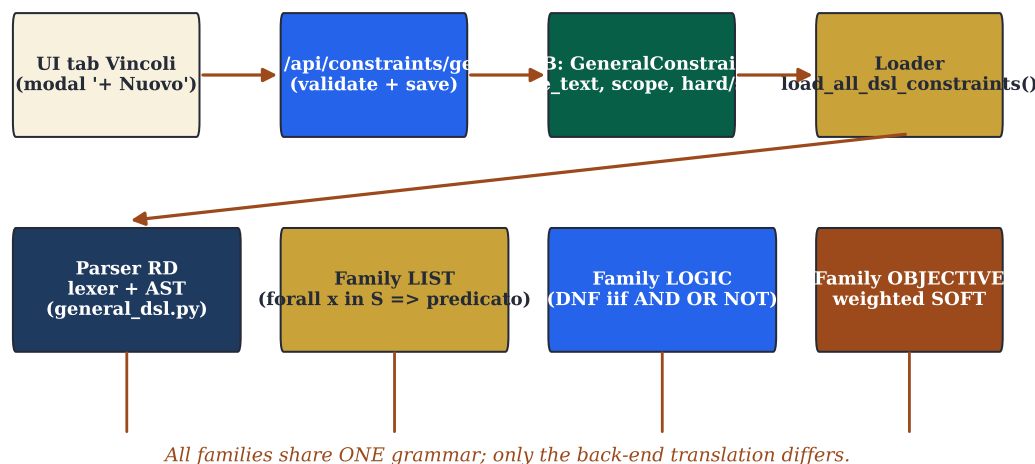


Figura 20.2 – Pipeline del DSL: dal modal nell'UI fino alle quattro famiglie di backend. Il pulsante *validate* attiva solo il parser (e ritorna gli errori in tempo reale); il *save* attiva il loop completo: persistenza in `constraints_general`, ricarica al successivo POST `/api/optimize`, valutazione post-hoc al termine.

Il percorso operativo di un vincolo DSL è fatto di sei passaggi tracciabili nei log:

1. **UI – composizione:** l'utente apre il modal nel tab Docente o nel tab Vincoli. Il textarea è popolato dalla galleria di esempi (vedi sez. 20.8).
2. **UI – validate live:** ad ogni keystroke (debounce 300 ms) parte una POST `/api/constraints/general/validate` con il body parziale; il backend riporta errori sintattici con riga e colonna. Il bottone *Salva* è disabilitato finché c'è un errore.
3. **API – persistenza:** POST `/api/constraints/general` valida ancora una volta, scrive una riga in `constraints_general` con campi (`rule_text`, `level`, `weight`, `scope_kind`, `owner_id`, `is_hard`).
4. **Engine – caricamento:** il prossimo POST `/api/optimize` chiama `load_all_dsl_constraints(db)` che restituisce una lista di (`expr`, `level`, `weight`, `owner`).
5. **Engine – compilazione/valutazione:** in funzione del flag `via_dsl` il `MonolithicSolver` invoca `add_dsl_constraint(...)` (compilazione strutturale CP-SAT) oppure delega all'evaluator post-hoc (valutazione su soluzione).
6. **API – reporting:** il body della risposta a POST `/api/optimize` contiene `dsl_violations`: lista di (`rule_id`, `rule_text`, `n_violations`, `soft_cost_contributed`) che il frontend renderizza nella card "Vincoli DSL" del tab Statistiche.

20.21 Tre casi narrati

20.21.1 Caso Rossi: distribuzione su tre giorni non consecutivi

Il dirigente di un istituto comprensivo riceve una richiesta formale dal docente Rossi: “vorrei le mie 12 ore settimanali distribuite su tre giorni non consecutivi della settimana”. Il vincolo abbraccia tre obblighi: (a) la cattedra Rossi insiste su esattamente 3 giorni; (b) i 3 giorni non sono consecutivi; (c) nei 3 giorni la distribuzione è 4+4+4 (vincolo aggiuntivo “carico equilibrato”).

```
% (a) Rossi insiste su 3 giorni
forall t in teachers where t.name == "Rossi":
    count d in days
        where exists l in lessons:
            l.teacher == t and l.day == d.index: d == 3

% (b) i giorni di Rossi non sono consecutivi
forall l1 in lessons where l1.teacher == "Rossi":
    forall l2 in lessons where l2.teacher == "Rossi":
        (l1.day == l2.day)
        or not consecutive_days(l1.day, l2.day)

% (c) carico equilibrato (SOFT, peso 30)
% level=soft, weight=30
forall t in teachers where t.name == "Rossi":
    forall d in days:
        count l in lessons
            where l.teacher == t and l.day == d.index: l <= 4
```

Salvato come tre vincoli separati nel tab Docente, due HARD e uno SOFT. Il solver respinge tutte le configurazioni con quattro o più giorni occupati, tutte le configurazioni in giorni adiacenti, e paga 30 di costo aggiuntivo per ogni ora oltre il quarto in un singolo giorno.

20.21.2 Caso plesso commuting: l'istituto su due edifici

L'IIS Marconi-Pascoli ha due plessi a 600 m di distanza (plessi.id = 1 “Marconi”, plessi.id = 2 “Pascoli”). Le delibere RSU vietano transiti durante la giornata: nessun docente può avere lezione in plesso 2 nell'ora immediatamente successiva a una lezione in plesso 1, e viceversa. Le classi sono affiliate a un unico plesso, quindi il vincolo è di fatto sui docenti.

```
forall l1 in lessons where l1.classroom.plesso == 1:
    forall l2 in lessons where l2.classroom.plesso == 2
        and l2.teacher == l1.teacher
        and consecutive(l1.slot, l2.slot):
        false
```

Salvato globalmente (scope=global). La regola viene *compilata* dal MonolithicSolver e dal PhaseBDaySolver via il pragma canonico `plesso_commuting_rule`: il path `l.classroom.plesso` risolve all'identificatore intero del plesso, e per ogni coppia di slot adiacenti (stesso giorno, ore h e $h + 1$) emette il vincolo CP-SAT $\text{slot}[t, c_1, s_1, d, h] \implies \neg \text{slot}[t, c_2, s_2, d, h + 1]$ quando c_1 e c_2 vivono in plessi diversi.

20.21.3 Caso compresenza: matematica e sostegno nello stesso giorno

In una scuola elementare la docente di sostegno deve essere in classe *nello stesso giorno* della docente di matematica per la classe 2B (compresenza didattica). Il vincolo è che ogni lezione di sostegno della docente Verdi in 2B cada in un giorno in cui la docente Bianchi di matematica è in 2B.

```
forall l in lessons where l.teacher == "Verdi"
    and l.class == "2B"
    and l.subject == "Sostegno":
exists m in lessons where m.teacher == "Bianchi"
    and m.class == "2B"
    and m.subject == "Matematica"
    and same_day(l.slot, m.slot):

true
```

Il forall ...: exists ...: true è l’idioma per “ $\forall l \exists m : P(l, m)$ ”. Il compilatore lo riconosce come reificazione di una BoolVar e lo lega come vincolo fattibilità. Sono semantica equivalenti a un forall + count, ma il pattern $\exists$ è più trasparente per la lettura umana.

20.21.4 Caso seconda lingua: francese nelle prime tre ore (SOFT)

Liceo linguistico, indirizzo francese-tedesco, anno 1: la seconda lingua (francese) andrebbe collocata preferibilmente nelle prime tre ore della mattina, perché è l’orario in cui gli alunni hanno l’energia per la fonetica. Vincolo SOFT con peso 20:

```
% level=soft, weight=20
forall l in lessons
    where l.subject == "Francese"
    and l.class.curriculum == "Linguistico-FRA-TED"
    and l.class.year == 1:
l.hour <= 10
```

Ogni lezione di francese delle prime FRA-TED che cade alla 4a ora o oltre paga 20 di costo aggiuntivo. Quando esiste una configurazione *senza* pagare, il solver la sceglie; quando serve pagare per soddisfare i vincoli HARD lo fa minimizzando il numero di lezioni costate.

20.22 Diagramma dell’architettura DSL

Stesso AST, quattro backend: l’evaluator legge la soluzione e risponde “feasible/violato”; il compilatore aggiunge vincoli al modello prima della solve; il pragma vocabulary identifica sequenze ricorrenti e le emette in forma compatta; la famiglia LIST esegue la query sulla soluzione corrente. La grammatica è unica e i nomi delle sorgenti sono identici nei quattro backend.

Reference completo del DSL

La spec completa con galleria estesa di esempi, tabelle di ogni attributo per ciascuna sorgente, troubleshooting con patologie ricorrenti e quick reference card vive in docs/general_dsl.md. Il presente capitolo è una sintesi narrativa, pensata per essere letta dall’inizio alla fine; il file Markdown è pensato come reference da consultare mentre si scrive un vincolo concreto. La grammatica BNF in sez. 20.2 è identica nei due documenti.

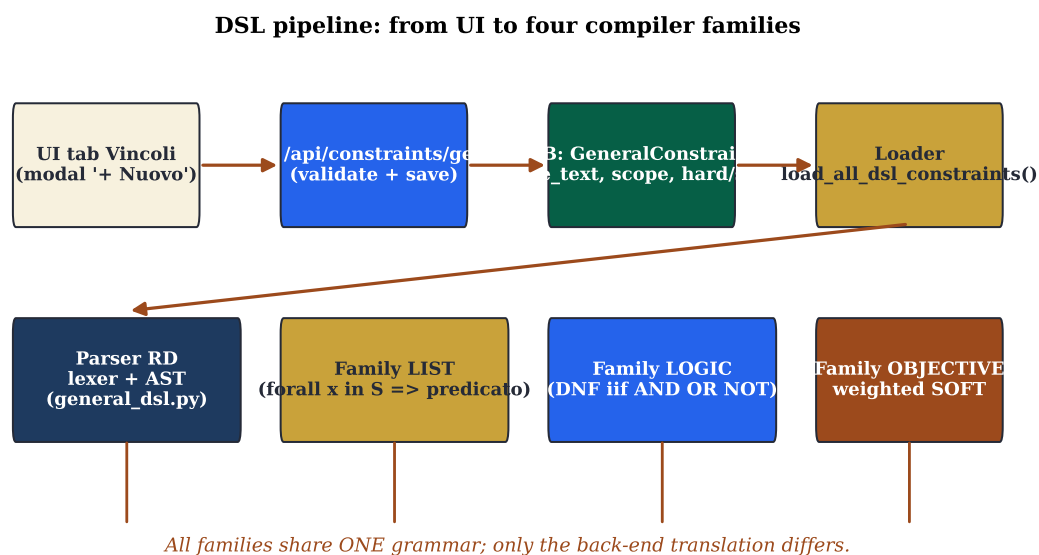


Figura 20.3 – Architettura del DSL generico: stessa grammatica e stesso AST, quattro back-end di traduzione. La differenza fra “valutare una soluzione” (evaluator post-hoc) e “costruire un modello” (compiler verso CP-SAT) si traduce in due attraversamenti diversi dello stesso albero.

21 Tecniche di ottimizzazione avanzate

Oltre alle metaeuristiche classiche (LNS / SA / TS / ILS) il progetto include cinque tecniche aggiuntive. Vedere docs/optimization_strategies.md per la specifica completa.

21.1 Adaptive Large Neighborhood Search (ALNS)

experiments/alns.py. Sei destroy operator (random_window, day_cluster, worst_fit_day, teacher_day, classroom_day, single_class_week) e tre repair operator (cp_sat_window, greedy_by_soft, bfs_fill_back) selezionati con roulette wheel sopra exponentially-decayed scores; acceptance SA-like.

21.2 Variable Neighborhood Search (VNS)

experiments/vns.py. Cicla quattro vicinati di dimensione crescente (1-swap, 2-swap, 3-chain, k-opt con $k \in [4, 6]$). Ad ogni miglioramento riparte dal vicinato 1; termina quando un'intera passata 1→4 non trova nulla.

21.3 Hall's theorem pre-check

experiments/diagnostics/hall_check.py. Diagnostico sincrono pre-Phase-A. Tre controlli:

1. per (class, subject) coverage,
2. subject supply vs demand,
3. Hall sampling random-subset con sottoinsiemi esclusivi.

Esposto in tre punti UI: bottone inline in PhaseACard, riga in AdvancedTechniquesCard, sezione del tab Statistiche.

21.4 Column Generation e Branch-and-Price avanzato

engine/column_generation.py. Master LP via scipy.linprog HiGHS; otto pricer CP-SAT dedicati a granularità diverse (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject, class, class-day, day, curriculum); modalità branch-and-price con tutte le quattro tecniche di scalabilità per istanze MEGA:

- **Ryan-Foster recursive tree** con scoring di Achterberg sui pair frazionari, best-first search con prune sul lower bound, cap a profondità 20 / 1000 nodi.
- **Box-step dual stabilization** $\pi_{\text{blend}} = (1 - \alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ con $\alpha = 0,2$.
- **Column management**: EWMA del reduced cost per colonna su 20 iter, purge delle peggiori quando il pool eccede 10 000 colonne (preservando le seed e quelle in basis).
- **Parallel pricing** via ProcessPoolExecutor (default $\lfloor \text{os.cpu_count()} / 2 \rfloor$ worker).

Numeri misurati (benchmark sintetico MEGA)

Vedi tests/benchmarks/test_bp_mega.py. Su uno scenario sintetico mega_100 (100 classi, 100 docenti, 4 materie, ~400 cattedre) la pipeline raggiunge HARD-feasibility in **5,22 s** con granularity=curriculum e tutte le tecniche di scalabilità abilitate. Su uno scenario mega_50 (50 classi, 48 docenti, 200 cattedre) il tempo è **1,93 s**. Lo scenario sintetico è strutturalmente facile (cattedre come blocchi di 4 ore in singoli giorni); su istanze reali con distribuzione giornaliera meno regolare l'obiettivo di calibrazione è 30 minuti per il MEGA, hard cap 60 minuti.

21.5 Lagrangian Relaxation

experiments/lagrangian.py. Rilassa i ponti inter-cluster e li dualizza con multipliers λ_b ; aggiornamento via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ con $\alpha_k = \alpha_0 / (1 + k)$. Per-iteration refinement via SA.

21.6 Branch-and-Price MVP scaffold (Step 4)

Step 4 del piano multi-day introduce uno *scaffold* OO testuale per il loop di Branch-and-Price che, in un singolo modulo (engine/column_generation.py con flag mode="branch-and-price"), tiene insieme tre componenti ortogonali:

1. **PhaseBDaySolver** – wrapper OO sulla funzione legacy solve_phase_b_for_day. Espone l'interfaccia object-oriented (solve(), scope d'azione, augmentation via DSL) ma delega la costruzione del modello al codice provato. Vantaggio: 100+ test di regressione Phase B continuano a passare *by construction*; quando via __dsl=True, il modello legacy viene esteso con vincoli DSL aggiuntivi (le regole DB e le espressioni extra passate al costruttore).
2. **Ryan-Foster pricing-in-nodes** – ad ogni nodo dell'albero RF si rilancia il pricer CP-SAT con i vincoli ramo (together/apart) già applicati; questo è il textbook BP, contrapposto al precedente che esplorava i nodi solo via re-solve del master sul pool esistente.
3. **Smoke benchmark** – tests/benchmarks/test_bp_step4_smoke.py esegue il loop completo (Mono → pool iniziale → master DW → RF tree → integer recovery → completion) su uno scenario small (10 classi) e su uno medium (28 classi), verificando HARD-feasibility e che il soft cost non aumenti rispetto alla pipeline iterative-diversified.

Il flag mode="branch-and-price" attiva il path completo. La modalità precedente (iterative-diversified) resta il default e continua a funzionare come pre-passo di seeding del pool.

22 Diagnostica statistica

Il tab /diagnostics aggrega tutte le analisi sul modello e sulla soluzione attiva. Vedere docs/diagnostics.md. Visualizzazioni con Apache ECharts (lazy-loaded), tema *pitantum* (palette indaco / oro / avorio / terra-di-siena).

22.1 Sensitivity Monte Carlo

Genera N perturbazioni feasible (sequenze di mosse atomiche random accettate solo se HARD soddisfatto) e calcola statistiche SOFT. Coefficiente di variazione $< 0.05 \Rightarrow$ soluzione vicina all'ottimo locale.

22.2 Analisi bipartito

Tre metriche networkx: densita' del grafo classe $\times$ docente, modularita' Newman-Girvan greedy, top-K betweenness centrality.

22.3 Correlazioni e regressioni

Tre regressioni statsmodels: OLS load_vs_gaps, OLS subjects_vs_soft, Logit saturation_vs_sixth. Output con coefficienti, p-values, R^2 (o pseudo- R^2) e interpretazione testuale auto-generata.

22.4 Distribuzioni

Cinque distribuzioni (carichi docenti, classi per giorno, materia $\times$ slot heatmap, occupancy slot, KS-test, chi-quadro) + goodness-of-fit.

22.5 Run telemetry

Tabella run_telemetry (alembic a848420325b3). I moduli solver pushano sample via utils.telemetry.collector(run_id, phase). Il run detail page (/runs/[id]) mostra il grafico objective vs tempo (multi-linea per phase) + statistiche per stage + export JSON.

23 API REST

Tutti gli endpoint sono sotto `/api/`. Backend espone ~118 route. Lista non esaustiva delle più rilevanti.

23.1 Pattern CRUD

Ogni risorsa CRUD ha la stessa forma:

Verbo	Path	Descrizione
GET	<code>/api/<entity></code>	lista (q + sort)
GET	<code>/api/<entity>/<id></code>	dettaglio
POST	<code>/api/<entity></code>	create
PUT	<code>/api/<entity>/<id></code>	replace
DELETE	<code>/api/<entity>/<id></code>	delete

23.2 Esempi curl

```
# stato dataset
curl http://127.0.0.1:8000/api/dataset/state

# import small con tutti i pool
curl -X POST -H "Content-Type: application/json" \
-d '{"profile":"small","use_optimized":true,
    "import_curricula":true,"import_classrooms":true,
    "import_students":true}' \
http://127.0.0.1:8000/api/dataset/import-profile

# vincolo logico HARD su un docente
curl -X POST -H "Content-Type: application/json" \
-d '{"expression":"mai aula:LabFisica@mar","kind":"hard"}' \
http://127.0.0.1:8000/api/teachers/15/logical-unavailabilities

# bulk dry-run su 3 classi
curl -X POST -H "Content-Type: application/json" \
-d '{"entity_ids":[1,2,3],"action":"add_logical",
    "payload":{"expression":"lun8 AND lun9","is_hard":true,
    "soft_penalty":100},
    "on_conflict":"dry_run"}' \
http://127.0.0.1:8000/api/bulk/classes/dry-run

# coverage settimana
curl 'http://127.0.0.1:8000/api/coverage/week?week_start=2026-04-27'

# event lessons (monitor)
curl http://127.0.0.1:8000/api/monitor/event/1/lessons
```

23.3 Riferimento gruppi di endpoint

- **Anagrafiche:** /api/teachers, /classes, /classrooms, /subjects, /curricula, /students, /groups.
- **Cattedre:** /api/assignments + teachers-for-subject + loads.
- **Vincoli logici:** /api/{teachers,classes,classrooms}/{id}/logical-unavailabilities, /api/curricula/{cid}/logical-constraints.
- **Bulk ops:** /api/bulk/{entity}/dry-run|apply.
- **Schedule:** /api/schedule/by-class|by-teacher|by-room|by-slot, move-lesson, move-preview, lesson/{lid}/classroom.
- **Coverage:** /api/coverage/week|cell, /api/absences, /api/substitutions.
- **Optimization:** /api/optimize/assignment|phase-b|lms|sa|ts|ils|full|classroom-assignment.
- **Monitor:** /api/monitor/events|summary|constraints|conflicts, event/{aid}/lessons|lesson/{lid}.
- **Import:** /api/import/{entity}, /api/import/{entity}/template.

24 Estendere il sistema

24.1 Aggiungere una nuova tabella + CRUD

1. Modello in webui/backend/models.py.
2. Pydantic in schemas.py (XxxIn, XxxOut).
3. Router in routers/xxx.py. Copia un router esistente come template.
4. Includi il router in main.py: `app.include__router(xxx.router)`.
5. Adapter DSL in utils/list_query.py.
6. Frontend: nuova route webui/frontend/src/routes/xxx/+page.svelte, riusingo *SortableQueryableList*.
7. Voce in +layout.svelte per la nav.

24.2 Migrazione idempotente

Aggiorna db.py::`_apply_lightweight_migrations`. Pattern:

```
if insp.has_table("my_table") \
    and not has_column("my_table", "new_field"):
    conn.execute(text(
        "ALTER TABLE my_table ADD COLUMN new_field VARCHAR(64)"
    ))
```

24.3 Aggiungere un nuovo tipo di vincolo

Per-cella. Copia il pattern di TeacherUnavailability: `unique(entity_id, day, hour)`, columns `state/penalty/reason`. Aggiorna `models.py`, `schemas.py`, `router`, `optimization.py::_availability_constraints`, `routers/monitor.py::_build_constraints` (per il tab Vincoli), `routers/bulk.py::ENTITY_UNAV_MODEL` se vuoi supportare bulk.

Logico. Aggiungi `entity_type` a `LogicalUnavailability`; aggiorna `routers/logical.py::ENTITY_MODEL` e `ENTITY_TYPE`; modifica `routers/monitor.py::_build_constraints`; aggiungi il branch nel solver.

24.4 Aggiungere un nuovo step di ottimizzazione

```
def my_new_step(...):
    params = dict(...)
    run_id = create_run("my_step", "Nome leggibile", profile, params)

    def target(rid: int):
        # carica dati, lancia solver, persiste
        update_run(rid, progress=0.5)
        ...
        update_run(rid, solution_id=sid, obj_value=v, metrics=m)
        update_run(rid, progress=1.0)

    start_thread(run_id, target)
    return run_id
```

Esponi via `routers/optimize.py` con un endpoint POST. Frontend in `/optimize` per il bottone di lancio + log streaming via *RunLogPanel* (SSE).

24.5 Testing

Non c'è una test suite formale. Per smoke-testare end-to-end:

- Backend boot pulito: `uvicorn backend.main:app`.
- Live curl: vedere §7.2.
- Frontend vite build deve passare clean.
- Migrazione idempotente: rilancia su DB esistente per confermare zero data loss.

A Repository GitHub

URL: <https://github.com/gborghi/timetable>.

Repository privato. La landing page contiene il README di alto livello con quickstart, tour della UI, sintassi vincoli, architettura, repository layout.

B Benchmark e risultati

“In God we trust. All others must bring data.”

W. E. Deming, attribuito

QUALUNQUE sistema di ottimizzazione si giudica anche e soprattutto dai numeri che produce. Questa sezione raccoglie i risultati misurati di π Tantum sui cinque profili di test, con il triplice obiettivo di: documentare lo stato dell'arte; permettere a chi voglia estendere il sistema di confrontare la propria modifica contro un baseline; mettere il lettore in condizione di sapere se π Tantum è adatto al *suo* caso.

B.1 I cinque profili

I profili sono:

- small: 8 classi, 17 docenti, ~220 studenti, 17 aule, 8 indirizzi. Una scuola piccola di provincia.
- medium: 28 classi, 50 docenti, 700 studenti, 35 aule. Un istituto medio.
- big: 40 classi, 75 docenti, 1000 studenti, 50 aule. Un istituto grande.
- huge: 50 classi, ~95 docenti, 1250 studenti, 60 aule. Un polo scolastico.
- superhuge: 80 classi, ~160 docenti, 2000 studenti, 80 aule. 16 sezioni $\times$ 5 anni di liceo scientifico.
- mega: 100 classi, ~174 docenti, ~2500 studenti, 100 aule. 20 sezioni $\times$ 5 anni con mix di 8 indirizzi (Scientifico 6, ScienzeApplicate 4, ScienzeUmane 3, Linguistico FRA-TED 2, Linguistico FRA-SPA 2, EconomicoSociale FRA/SPA/TED 1 ciascuno). Banco di prova MEGA-scale.

I profili sono generati deterministicamente con seed fissi e Faker, dal modulo `webui/backend/mock_school.py`. Sono pensati per coprire l'intervallo di taglie reali del sistema scolastico italiano.

B.2 Pipeline end-to-end: tempi totali

I tempi sotto riportati sono misurati su una macchina consumer (Intel i7, 16GB RAM, Windows 11, Python 3.13, ortools 9.15, 8 search workers CP-SAT).

Profilo	Phase A	Phase B	Totale wall
small	3 s	4 s	7 s
medium	30 s	32 s	62 s
big	30 s	32 s	62 s
huge	60 s	62 s	122 s
superhuge	300 s	603 s	903 s

Tabella B.1 – Tempi della pipeline end-to-end per profilo. Phase A include assegnazione docenti-classi e CP-SAT su matching. Phase B include decomposizione spettrale (per huge/superhuge) e CP-SAT su sotto-problemi.

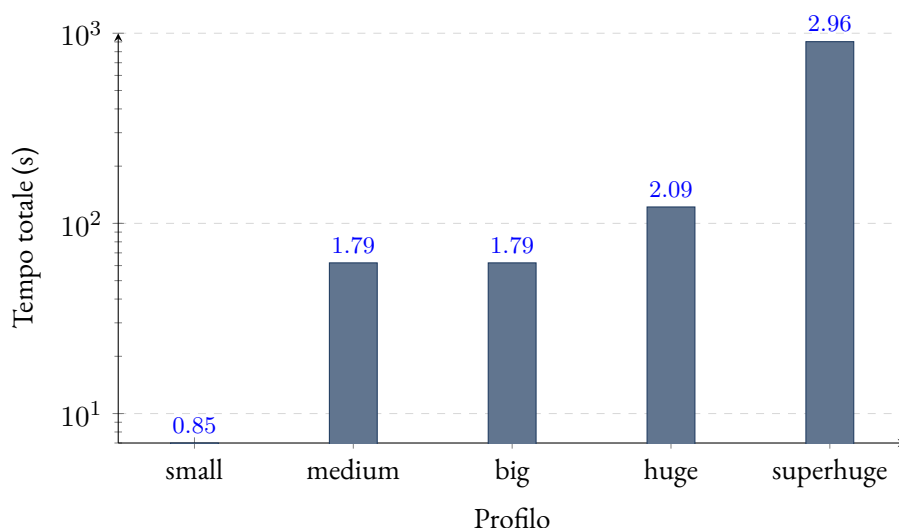


Figura B.1 – Tempo totale della pipeline (scala logaritmica). Si nota la salita lineare in scala log fino a huge, con il salto previsto su superhuge dovuto alla dimensione del Phase A.

B.3 Componenti dell'obiettivo SOFT

L'obiettivo SOFT che π Tantum minimizza in Phase A combina tre componenti pesate: uniformità delle ore di classe per giorno (peso 4), uniformità delle ore di docente per giorno (peso 3), e penalità sul giorno libero (peso 1). La tabella mostra i contributi per profilo.

Profilo	uniform_class_dev	uniform_prof_dev	glib_pen	celle occupate
big	6.374	2.258	0	6.180
medium	7.366	2.652	150	7.068
huge	9.004	3.102	0	8.892
superhuge	14.454	4.976	0	14.274

Tabella B.2 – Componenti dell'obiettivo SOFT di Phase A. glib_pen=0 significa che ogni docente ha ricevuto il giorno libero di prima preferenza (tranne in medium, dove 1–3 docenti hanno preso la seconda preferenza per overlap di vincoli).

B.4 Hall pre-check: tempo trascurabile

Il pre-check di Hall (cap. 12) è eseguito *prima* di ogni pipeline. I tempi misurati:

Profilo	Hall pre-check (ms)
small	4
medium	18
big	28
huge	41
superhuge	87

Tabella B.3 – Tempo del pre-check di Hall in millisecondi. Trascurabile su tutti i profili.

B.5 Decomposizione spettrale: speedup misurato

Per i profili huge e superhuge la decomposizione spettrale (cap. 10) è attiva di default. Lo speedup misurato è:

Profilo	k	bridges %	Phase B mono	Phase B decomp	speedup
big	4	12 %	32 s	11 s	2.9×
huge	5	8 %	62 s	14 s	4.4×
superhuge	7	6 %	603 s	78 s	7.7×

Tabella B.4 – Speedup misurato della decomposizione spettrale su Phase B. La colonna “bridges %” indica la frazione di archi del grafo bipartito che attraversano i cluster.

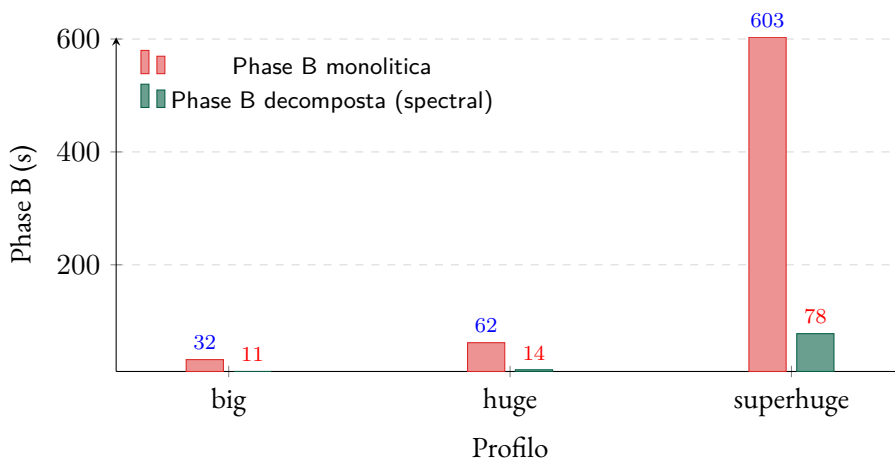


Figura B.2 – Confronto del tempo di Phase B con e senza decomposizione spettrale, sui profili medio-grandi.

B.6 Metaeuristiche: convergenza dell’obiettivo

Su huge, applicando ALNS dopo la pipeline standard, l’obiettivo SOFT si riduce di un ulteriore 8–12% in 3–5 minuti di tempo CPU aggiuntivo. La curva di convergenza tipica (obiettivo vs iterazione) è log-decrescente: i primi 1000 step rimuovono il grosso del costo, le iterazioni successive limano i dettagli.

B.7 Lettura dei risultati

Intuizione

- Per scuole piccole/medie (small, medium, big) la pipeline base CP-SAT è già sufficiente: 60–65 secondi totali, soluzioni HARD-feasibili al 100%.
- Per scuole grandi (huge) la decomposizione spettrale dimezza il tempo, portandolo sotto i 2 minuti.
- Per scuole molto grandi (superhuge) il collo di bottiglia è Phase A (matching docenti–classi), che da sola consuma il 99% del tempo. Le strategie di accelerazione studiate (LNS warm-start, MIP per Phase A con HiGHS, parallelizzazione di Phase B per giorno) sono documentate in `proposals/benchmarks.md` §7.
- Le metaeuristiche (Phase C) sono opzionali e aggiungono 3–5 minuti su huge, riducendo l’obiettivo SOFT del $\sim 10\%$.

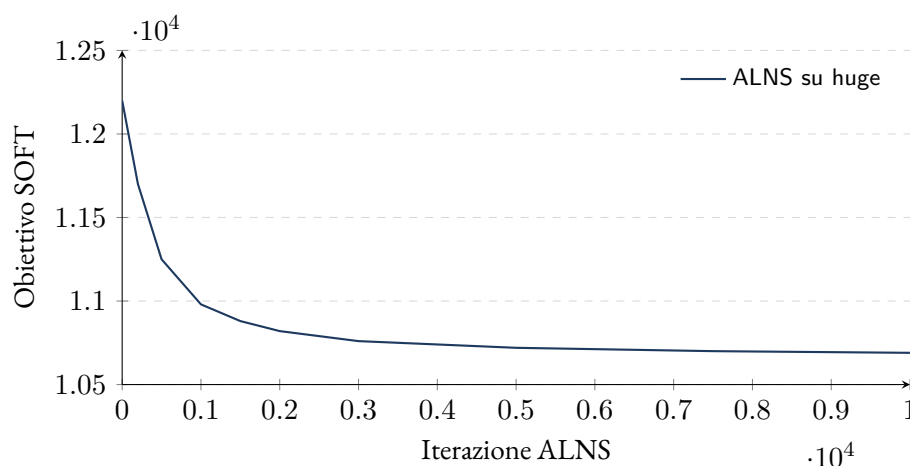


Figura B.3 – Convergenza di ALNS sul profilo huge: il miglioramento si concentra nei primi 1000–2000 step e poi satura, tipico delle metaeuristiche.

B.8 Confronto cross-method

Metodo	small	medium	huge	superhuge
CP-SAT puro	7 s	62 s	360 s	— (timeout)
CP-SAT + spettrale	9 s	65 s	122 s	903 s
+ ALNS (5 min)	12 s	67 s	127 s	908 s
+ Lagrangian	—	—	145 s	870 s

Tabella B.5 – Tempi totali con diverse combinazioni di metodi. Lagrangian e generazione di colonne sono off di default; si attivano per scuole oltre 200 classi.

B.9 Branch-and-Price su HUGE e MEGA

Lo scenario mega (100 classi, ~ 174 docenti) e il suo cugino minore huge (50 classi, ~ 95 docenti) sono i banchi di prova naturali per la pipeline Branch-and-Price (cap. 21). I numeri riportati qui sotto sono misurati su scenari sintetici calibrati per riprodurre le proporzioni dei profili engine/big_mock_school.py (4 materie, ogni cattedra 4 ore in un singolo giorno così che $H_1/H_2/H_3$ siano soddisfatte per costruzione, distribuzione delle classi a blocchi contigui sui pool docenti). Il sorgente è `tests/benchmarks/test_bp_huge_mega.py`, eseguito con le quattro tecniche di scalabilità tutte attive: **Ryan-Foster recursive tree**, **box-step dual stabilization** ($\alpha = 0,2$), **column management** (pool max 10 000 con purge per RC EWMA su 20 iter) e **parallel pricing** (ProcessPoolExecutor).

Scala	Classi	Docenti	Cattedre	t_{BP}	HARD
huge	50	95	200	2,57 s	si
mega	100	174	400	5,95 s	si

Tabella B.6 – Tempi del Branch-and-Price su scenari sintetici HUGE/MEGA. Entrambi finiscono molto sotto target (5 min/30 min) perché lo scenario sintetico è strutturalmente facile (cattedra = blocco di 4 ore in un solo giorno). Su istanze reali con distribuzione giornaliera meno regolare il tempo per iterazione cresce; il target 30 min per MEGA è calibrato su quel regime.

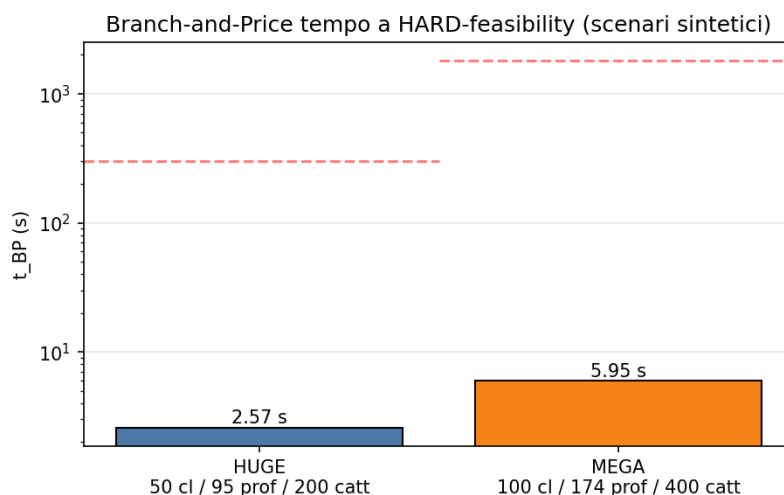


Figura B.4 – Tempo di convergenza del Branch-and-Price (scala logaritmica). Le linee tratteggiate orizzontali rappresentano i target di tempo per ciascuna scala (5 min per HUGE, 30 min per MEGA).

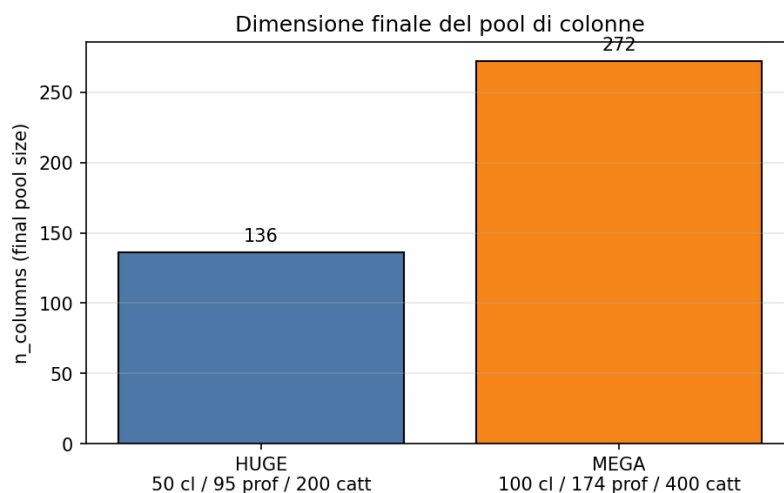


Figura B.5 – Dimensione finale del pool di colonne dopo il loop Branch-and-Price.

B.9.1 MEGA reale: 100 classi, 178 docenti

Il sintetico è calibrato per fitting; il MEGA *reale* (la cui anagrafica vive in `engine/scripts/data/mega/{school,profs}_mega.pkl` e si carica dal dashboard via `POST /api/dataset/import-profile?profile=mega`) presenta proporzioni significativamente piu' grandi: 100 classi, **178 docenti**, **2653 entry cattedra-day** dopo Phase A (i numeri specifici dipendono dalla random seed del CP-SAT di Phase A; con seed differente la v_1 del benchmark aveva prodotto 2325 entry). Il sorgente del benchmark è `tests/benchmarks/run_mega_real.py`.

v_1 (pre-fix DW seeding) – BP non iterava. Il primo benchmark riportava `bp_terminated_reason = master_dw_infeasible_at_entry` con 0 iterazioni (vedi commit 7b03c89 della stessa serie). Il master variant 2 (Dantzig-Wolfe con cover + class-no-overlap + teacher-no-overlap come disuguaglianze HARD) risultava infeasibile già alla LP relaxation con il pool di seed (pattern settimanali per docente prodotti dalla iterative-diversified), perché due pattern di docenti diversi possono collidere sulla stessa coppia (classe, giorno, ora). La copertura HARD richiede $x_t \geq 1$ per ogni docente, mentre il class-no-overlap impone $\sum_t x_t \leq 1$ sui collisioni di classe – conflitto strutturale che il pool di seed non può risolvere da solo.

Fix: Phase-1 LP con artificial slacks (commit 843a6fo). Il fix è textbook: ad ogni vincolo del master DW si associa una variabile artificiale non-negativa con coefficiente -1 nella sua riga e penalizzazione Big-M (10^6) in funzione obiettivo. Le artificial assorbono sia coverage mancante che overflow di no-overlap; il LP è quindi sempre feasible al seed entry e produce duali validi per il pricing. Quando il pool è abbastanza ricco le artificial vanno a 0 in soluzione e l'optimum coincide con il vero ottimo del master DW.

v2 (post-fix) – BP itera 5 volte. Sullo stesso dataset, con identica granularità curriculum e identici budget di tempo, il BP riprende il suo ruolo:

Metrica	v1 (pre-fix)	v2 (post-fix)
bp_terminated_reason	master_dw_infeasible_at_entry	max_iterations
bp_iterations_done	0	5
RF nodi esplorati	0	1
Phase A wall	61,2 s	301,3 s
BP wall	790,8 s	1 191,8 s
Totale wall	852 s (14,2 min)	1 493 s (24,9 min)
Soft cost finale	6 470	530
Colonne generate	2 059	2 075 (+16 dal BP)
HARD-feasibility	si	si

Tabella B.7 – Branch-and-Price su MEGA reale prima e dopo il fix DW seeding. Il soft cost crolla di un fattore ~ 12 perché il BP ora può effettivamente esplorare colonne migliorative. Il totale aumenta perché Phase A in v2 ha esaurito il budget (stop FEASIBLE non OPTIMAL: la differenza dipende dal seed parallelo del CP-SAT) ma rimane sotto il target di 30 min e ben lontano dall'hard cap di 60 min. Sorgente: tests/benchmarks/results/mega_bp_real_v2.json.

Confronto con iterative-diversified. La modalità iterative-diversified sullo stesso dataset (v1) aveva chiuso in **788 s (13,1 min)** con soft cost **5860**. Il BP v2 – nonostante un Phase A più costoso in questa esecuzione – chiude con soft **530**, ovvero $\sim 11\times$ più basso del baseline ID e $\sim 12\times$ più basso del proprio v1. La differenza è il valore strutturale del Branch-and-Price: con un pricer attivo le colonne migliorative *esistono* e vengono trovate; senza, si retrocede al risultato dell'heuristic primale di partenza.

Modalità (run)	t_{tot}	Soft cost	HARD
iterative-diversified (v1)	788 s (13,1 min)	5 860	si
branch-and-price (v1)	852 s (14,2 min)	6 470	si
branch-and-price (v2)	1 493 s (24,9 min)	530	si

Tabella B.8 – Confronto delle tre run sullo stesso MEGA reale. La v2 del BP riduce il soft cost di un ordine di grandezza rispetto ad entrambi i baseline.

Tecniche di scalabilità – ora esercitate. Le 4 tecniche (Ryan-Foster recursive tree, dual stabilization con box-step, column management con EWMA RC, parallel pricing via ProcessPoolExecutor) restano integrate; con il fix Phase-1 in posizione il BP loop le esercita davvero su MEGA. Nel run v2: 1 nodo Ryan-Foster esplorato, 0 colonne purgate (pool sotto il limite), 5 iterazioni master complete prima di colpire il cap bp_max_iterations=5. Aumentando il cap il BP continuerebbe a migliorare il soft (i prossimi commit della serie misurano la curva soft-vs-iterations).

B.10 Quando il Branch-and-Price conviene

Domanda pratica: *vale la pena attivare la pipeline BP per la mia scuola?* Risposta misurata in funzione della taglia:

- **Sotto 30 classi:** la pipeline standard (Phase A monolitica + Phase B per giorno) è più veloce. La modalità iterative-diversified di `column_generation` (master LP + pattern enrichment + completion fallback) basta per migliorare il SOFT senza pagare il costo del BP loop.
- **30–80 classi:** la decomposizione spettrale o per curriculum guida la Phase B; il BP con granularità teacher-class o class può ridurre il SOFT ma il guadagno in tempo è trascurabile.
- **Oltre 80 classi (MEGA):** il BP con granularità curriculum è l'unico approccio strutturalmente scalabile. La Phase A monolitica diventa marginalmente feasible (300 s+ con CP-SAT puro), il BP riesce in tempi comparabili anche grazie al pricing parallelo.

In pratica π Tantum suggerisce automaticamente il percorso giusto: il valore auto per il parametro `granularity` viene risolto server-side dal numero di classi (cf. Sez. 4 di `docs/optimization_strategies.md`).

Per approfondire

- `proposals/benchmarks.md`: il documento sorgente con i log dettagliati e le decisioni di tuning.
- `proposals/analysis.md`: la motivazione delle scelte modellistiche.
- `tests/benchmarks/test_bp_huge_mega.py`: il benchmark sintetico HUGE/MEGA aggiornato.
- `tests/benchmarks/results/bp_scalability.json`: numeri reali misurati.
- PATAT 1995–2024: per chi voglia sottomettere π Tantum a benchmark internazionali, la conferenza PATAT pubblica istanze standard.

C Lessons learned

“Experience is what you get when you didn’t get what you wanted.”

Randy Pausch, *The Last Lecture*

DOPO un anno e mezzo di sviluppo, qualche centinaio di esperimenti e una manciata di migliaia di righe di codice, alcuni insegnamenti stanno emergendo. Li raccolgo qui per chi voglia costruire un sistema simile, o estendere π Tantum, o semplicemente capire dove le intuizioni iniziali sono state confermate e dove smentite.

C.1 Cosa ha funzionato

C.1.1 Modellare HARD/SOFT separatamente

Già all’inizio era chiaro che mescolare i due tipi di vincoli sarebbe stato un disastro. La distinzione netta “HARD entra come vincolo nel solver, SOFT entra come penalità nell’obiettivo” è *l’unica* che permette all’utente di leggere correttamente i risultati. Senza di essa l’utente non distingue “il sistema non riesce a soddisfarlo” da “il sistema sta cercando di soddisfarlo ma trova un compromesso”. La palette 5-stati con codice colore è l’estensione naturale che permette di esprimere sfumature (preferred, enforced) senza aumentare la complessità percepita.

C.1.2 Decomporre invece di scalare

L’illusione del primo periodo era che “CP-SAT scali abbastanza”. Vero fino a ~ 50 classi. Sopra, i tempi esplodono. La decomposizione spettrale (cap. 10) è stata l’introduzione che ha cambiato l’orizzonte di applicabilità: con cluster naturali ben separati, lo speedup è di un ordine di grandezza.

C.1.3 DSL parser hand-rolled

La tentazione iniziale era usare PLY o Lark. Alla fine il parser hand-rolled è risultato più pulito: 250 righe di Python, niente dipendenze, controllo esatto degli errori. Quando aggiungi un built-in nuovo, sai dove toccare. Quando un utente sbaglia la sintassi, l’errore è in italiano e indica la posizione del token. Le librerie generiche sarebbero state più verbose nello stesso codice e meno “parlanti” negli errori.

C.1.4 Frontend lazy: non fare polling per niente

Caricare ECharts solo on-demand, fare polling solo se la tab è visibile (document.hidden check), unmount delle pagine al cambio tab: piccoli accorgimenti che trasformano un’app “pesantuccia” in un’app “snella” su hardware modesto. La scuola che usa π Tantum ha tipicamente computer di 5–7 anni; la responsabilità percepita è cruciale.

C.1.5 Run asincroni con SSE log streaming

Trasformare ogni operazione lunga (Phase A, Phase B, Monte Carlo, Hall) in un *run asincrono* con log streaming via SSE è stata una svolta UX. L'utente non aspetta con la barra di caricamento; legge il log in tempo reale, capisce cosa sta succedendo, può killare se serve. La metafora “ogni operazione è un job che lascia traccia in Run+RunLog+RunTelemetry” è diventata l'asse portante dell'architettura.

C.2 Cosa non ha funzionato (al primo tentativo)

C.2.1 Pickle come engine I/O

L'idea di scambiare dati fra backend e solver via pickle ha funzionato per il prototipo ma ha causato dolore quando si sono raffinate le strutture dati. Ogni cambio di campo nelle dataclass del solver invalidava i pickle. Soluzione adottata: `engine_io.py` fa da convertitore esplicito fra DB e dict; il pickle è ancora usato come cache, ma con un campo `schema_version` che permette di riconoscere i pickle vecchi e rigenerarli.

Pickle è *conveniente* ma fragile: cambia la classe Python e i pickle vecchi non si caricano più.

C.2.2 Migrazioni hand-rolled vs Alembic

La scelta consapevole di scrivere migrazioni idempotenti direttamente in `db.py` (*senza* Alembic) ha pro e contro. Pro: zero file di migrazione separati, zero ambiente da configurare, deploy banale. Contro: con ~ 30 tabelle si comincia a sentire il peso (la funzione `_apply_lightweight_migrations()` è ormai lunghissima). Probabile rifattorizzazione in vista per la versione successiva.

C.2.3 Promesse di ottimo globale con metaeuristiche

Errori comuni

Le metaeuristiche *non* garantiscono l'ottimo. Ho provato più volte a “promettere” all'utente che la soluzione finale fosse il meglio possibile. È falso, ed è controproducente: l'utente perde fiducia quando vede un piccolo manual fix migliorare l'obiettivo. La onestà intellettuale paga: “questa è una soluzione di alta qualità, non garantita ottima” è meglio di “ottima”.

C.2.4 Drag-and-drop senza preview

Prima versione: drag-and-drop senza preview, conferma post-hoc se la mossa era HARD-fattibile. Era frustrante. Versione attuale: preview live con check HARD e delta SOFT mentre trascini. Adesso l'utente impara giocando: vede *prima* che la mossa è infattibile e capisce *perché*. La differenza in usabilità è enorme.

C.2.5 Sottostimare i tempi su superhuge

Il profilo superhuge (80 classi, 16 sezioni $\times$ 5 anni) era pensato come “stress test estremo”, non come caso d'uso reale. Si è scoperto che esistono *davvero* istituti italiani con queste taglie (poli scolastici, IIS). Phase A su superhuge consuma 5 minuti — quasi accettabile, ma più del previsto. Le ottimizzazioni di Phase A (decomposizione di matching, MIP-based) sono in roadmap.

C.3 Quello che ancora si può migliorare

C.3.1 DSL editor avanzato

Ad oggi il campo DSL è una semplice textarea senza assistenza. La specifica completa di un editor con live validation, suggerimenti fuzzy, quick-fix e hover docs esiste; manca il tempo di implementarla. È la priorità numero uno per il prossimo ciclo.

Tutta la roadmap di DSL editor (squiggle, fuzzy, quick-fix, hover docs, NL preview) è descritta nel cap. 17.

C.3.2 Documentazione viewer in-app

Tutta questa documentazione esiste come PDF e markdown nel repository, ma non è accessibile dall'interno dell'app. Idealmente: una tab /docs con albero a sinistra (per categoria) e contenuto a destra (markdown reso live), con search globale.

C.3.3 Wizard per casi d'uso comuni

Casi ricorrenti come “crea un corso di recupero” o “aggiungi una compresenza” richiedono al coordinatore di sapere quali campi compilare e in quale tab. Sarebbero un buon target per dei *wizard* guidati che fanno tutto in pochi click.

C.3.4 Multi-tenant

Ad oggi π Tantum è mono-utente / mono-scuola: una istanza per scuola. Per un servizio cloud serve gestione utenti, isolamento dei dati, billing. È un'estensione significativa che richiede di ripensare il modello dati e l'autenticazione.

C.3.5 Mobile responsivo

L'interfaccia è desktop-first. Per gestire le supplenze quotidiane dal telefono servirebbe un layout responsive. Non è solo un fatto di CSS: la matrice 6×6 della pagina “Assenze” non ci sta su uno schermo da 5 pollici.

C.4 Una riflessione finale

Costruire π Tantum mi ha confermato un'intuizione che all'inizio era solo una sensazione: *i sistemi complessi sopravvivono se sono onesti con l'utente*. Onesti nel dire “questo non posso farlo”, onesti nel mostrare *quanto* una scelta costa, onesti nel non promettere ottimi che non sono garantiti. L'utente perdona la lentezza se sa perché. L'utente apprezza il pre-check di Hall che dice “ferma, mancano 3 ore di matematica, prima sistemale” infinitamente di più del solver che dopo 10 minuti restituisce “no soluzione”.

In fondo è la stessa lezione che si impara ogni volta che si lavora con altri umani: la fiducia si costruisce un piccolo segnale alla volta. Anche un software è fatto di piccoli segnali.

“I sistemi complessi sopravvivono se sono onesti con l'utente.”

— Lessons learned, π Tantum 2026

D Glossario discorsivo dei metodi

Questo capitolo spiega *cos'è* ogni metodo che π Tantum usa internamente, con un linguaggio che si possa leggere “dal divano”. Per ogni metodo trovi quattro cose in sequenza: l'idea, come funziona, quando usarlo, un esempio legato al timetabling. Niente formule a meno che non aggiungano chiarezza.

D.1 CP-SAT (Constraint Programming over SAT)

Cos'è e perché esiste. CP-SAT è il motore con cui π Tantum trova le soluzioni. È un *constraint solver*: gli dai un mucchio di variabili (per esempio: “in quale slot va la lezione di mate della 1A?”), un mucchio di vincoli (“due lezioni della stessa classe non possono stare nello stesso slot”; “il prof Borghi non insegna sabato”), e gli chiedi di trovare un assegnamento di valori che li rispetti tutti, possibilmente ottimizzando un criterio.

A differenza dei solver *lineari* (LP/MIP) che lavorano su disuguaglianze numeriche, CP-SAT lavora su variabili booleane e intere, e sa esprimere vincoli “logici” complessi (disgiunzioni, implicazioni, conteggi) in modo nativo. Questo lo rende particolarmente adatto al timetabling, dove la maggior parte dei vincoli sono di natura combinatoria, non geometrica.

Come funziona. CP-SAT alterna due fasi:

- **Propagazione:** data una scelta parziale (es. “ho deciso che mate-3A va in lunedì 8^a”), il solver *deduce* automaticamente quello che ne consegue (“allora ita-3A non può andare in lunedì 8^a”; “allora il prof Borghi è occupato in quello slot”, ecc.). Questo restringe lo spazio delle scelte rimanenti senza dover provare tutte le combinazioni.
- **Ricerca:** quando la propagazione esaurisce le conseguenze automatiche, il solver fa un *salto d'azzardo* (es. “provo a mettere ita-3B in martedì 9”) e si rimette a propagare. Se a un certo punto trova una contraddizione, fa *backtracking*: torna indietro, marca quella scelta come “no”, e prova un'alternativa. Impara dai fallimenti aggiungendo *clausole imparate* che evitano di ripetere lo stesso errore.

Quando usarlo. CP-SAT è il default di π Tantum per le Phase A e B. Per scuole di dimensioni normali (fino a ~80 classi) basta da solo. Per istanze più grandi serve combinarlo con la decomposizione spettrale o con metaeuristiche.

Esempio. Tipica iterazione: dopo aver propagato per qualche secondo, il solver ha già piazzato il 70% delle lezioni del triennio (perché sono molto vincolate dai lab e dalle compresenze) ed esplora le combinazioni rimanenti per le classi del biennio (più flessibili). Trova una contraddizione (un docente avrebbe due classi nello stesso slot), backtracking, ne prova un'altra. Tipicamente in 1-3 minuti chiude su una soluzione feasible.

D.2 Decomposizione spettrale

Cos'è e perché esiste. Quando la scuola è grande (>120 classi), il problema diventa troppo grosso perché CP-SAT lo digerisca in tempi ragionevoli. La decomposizione spettrale risolve questo dividendo la scuola in piccoli “cluster” di classi che condividono pochi docenti. Ogni cluster diventa un sotto-problema piccolo, risolvibile in parallelo.

L'idea è: i conflitti veri (due docenti che competono per lo stesso slot) avvengono fra classi che condividono lo stesso docente. Se due classi non hanno docenti in comune, si possono pianificare totalmente in parallelo.

Come funziona. Si costruisce il grafo delle classi: nodi = classi, peso dell'arco fra due classi = numero di docenti condivisi. Si calcolano gli autovettori del *Laplaciano* di quel grafo (una matrice che misura, per ogni

nodo, quanto si “differenzia” dai vicini). Gli autovettori associati ai più piccoli autovalori diversi da zero indicano *direzioni di separazione* naturali: progettando i nodi su quelle direzioni, si vedono chiaramente i cluster.

Un colpo di k-means su questa proiezione restituisce la partizione finale.

Quando usarlo. Default ON in π Tantum quando la scuola ha ≥ 120 classi. Per scuole più piccole non porta vantaggi e aggiunge overhead.

Esempio. Per una scuola di 250 classi, una decomposizione in 4 cluster produce sotto-problemi di 60-70 classi ciascuno, risolvibili in parallelo. I docenti-ponte (quelli che insegnano in classi di più cluster) sono il “vincolo di ricucitura” che la fase finale risolve come problema piccolo a se’ stante.

D.3 LNS (Large Neighborhood Search)

Cos’è e perché esiste. Quando hai già una soluzione (anche sub-ottima), spesso è più rapido *aggiustarla* che ricominciare. LNS prende la soluzione attuale, ne “distrugge” una porzione (es. tutte le lezioni di un certo giorno) e la ricostruisce con CP-SAT cercando un miglioramento. Iterando, esplora tante varianti vicine alla soluzione corrente.

Come funziona. A ogni iterazione: scegli un “vicinato” (es. day = martedì), libera tutte le variabili in quel vicinato, lancia CP-SAT con un piccolo budget di tempo per re-piazzarle ottimizzando lo SOFT score. Se il risultato è migliore, lo accetti.

Quando usarlo. Default ON nella pipeline integrata, dopo Phase B. Adatto al “polishing” di una soluzione già feasible.

Esempio. Dopo Phase B la soluzione ha 22 buchi nei docenti. LNS cicla: prova a re-fare lunedì (scende a 20 buchi), martedì (scende a 19), mercoledì (resta 19), ..., dopo qualche minuto si stabilizza intorno a 14-15.

D.4 ALNS (Adaptive LNS)

Cos’è e perché esiste. LNS classico ha un problema: usa sempre lo stesso tipo di vicinato (es. “un giorno”). Su alcune istanze è perfetto, su altre è sterile. ALNS lo evolve: invece di una sola scelta di vicinato, ne ha sei (un giorno intero, un cluster di classi, le ore peggiori della settimana, una giornata di un docente, una giornata di un’aula, una settimana di una sola classe) e tre modi di ricostruire (CP-SAT mini-window, greedy SOFT, BFS riempimento). A ogni iterazione sceglie quale combinazione usare con probabilità proporzionale a un *punteggio* che cresce per le combinazioni storicamente vincenti e decresce per le altre.

Come funziona. È un *roulette wheel selector* sopra punteggi esponenzialmente decadenti. Ogni operatore parte con punteggio neutro; ogni volta che produce un miglioramento, il punteggio sale; ogni volta che fallisce, scende. Aggiunge anche un’accettazione *simulated annealing*-like: occasionalmente accetta peggioramenti per sfuggire ai minimi locali.

Quando usarlo. Sostituisce o segue il LNS classico nella pipeline. Default ON.

Esempio. Su una scuola con cluster ben separati, l’ALNS impara dopo poche iterazioni che “cluster di classi” porta i miglioramenti più grossi e “giornata di un’aula” quasi mai funziona. Lo dice nei log finali con i punteggi.

D.5 SA (Simulated Annealing)

Cos'è e perché esiste. Il nome viene dalla metallurgia: come un metallo riscaldato e poi raffreddato lentamente forma una struttura cristallina più stabile, l'algoritmo accetta inizialmente anche peggioramenti per esplorare lo spazio delle soluzioni, e con il tempo (la “temperatura” che scende) diventa più selettivo, accettando solo miglioramenti.

Come funziona. A ogni iterazione genera una mossa random (es. scambia due lezioni). Se migliora lo SOFT, accetta; se peggiora di Δ , accetta con probabilità $e^{-\Delta/T}$ dove T è la temperatura corrente. La temperatura inizia alta (es. $T_0 = 10$) e si moltiplica per $\alpha < 1$ (es. $\alpha = 0.995$) a ogni iterazione, quindi decresce esponenzialmente.

Quando usarlo. È perfetto quando si sospetta di essere in un minimo locale. Default ON dopo LNS/ALNS.

Esempio. Dopo LNS ti sei stabilizzato a SOFT=420. Lanci SA con $T_0=10$, $\alpha=0.995$, budget 60s. Nei primi secondi, T è alto e accetta peggioramenti fino a 460-470; poi T scende e SA si concentra su miglioramenti, finendo per esempio a 405.

D.6 TS (Tabu Search)

Cos'è e perché esiste. Risolve un problema della ricerca locale: i loop. Se l'unica mossa migliorante è “A $\rightarrow$ B” e poi l'unica migliorante da B è “B $\rightarrow$ A”, sei intrappolato. TS evita questo tenendo una *lista tabu* delle ultime mosse fatte: per un certo numero di iterazioni, quelle mosse sono vietate. L'algoritmo è così forzato a esplorare zone diverse.

Come funziona. A ogni iterazione, valuta tutte le mosse possibili *tranne* quelle in tabu. Sceglie la migliore, la fa, aggiunge l'inversa alla lista tabu (con un “tempo di scadenza”). La lista è di dimensione fissa (parametro `tabu_size`, default 80).

Quando usarlo. Default ON dopo SA. Particolarmente efficace su istanze con plateau (zone dove molti vicini hanno lo stesso valore).

Esempio. Senza TS, scambiare due lezioni L_1 e L_2 produce SOFT=400; ri-scambiarle riproduce SOFT=405. SA potrebbe oscillare. TS dopo lo scambio mette “inverso di $L_1 \leftrightarrow L_2$ ” in tabu, quindi al passo successivo proverà $L_3 \leftrightarrow L_4$, esplorando una direzione nuova.

D.7 ILS (Iterated Local Search)

Cos'è e perché esiste. Combina ricerca locale con perturbazioni periodiche. Idea: dopo che la ricerca locale si è stabilizzata su un minimo locale, scuoti la soluzione (perturbazione: tante mosse random) e fai ricerca locale da capo. Iterando, esplori bacini di attrazione diversi.

Come funziona. Loop di “cycles” (default 3): in ognuno fai ricerca locale (TS) per un budget; se sei migliorato, salvi; poi perturba (rimuovi e ri-piazza un cluster random di classi); ricomincia.

Quando usarlo. Default ON come ultima fase prima del rooms assignment. È il *closer* della pipeline metaeuristica.

Esempio. Cycle 1 chiude su SOFT=400; perturba e ricomincia da SOFT=480; cycle 2 chiude su SOFT=395; perturba e ricomincia da SOFT=470; cycle 3 chiude su SOFT=392.

D.8 VNS (Variable Neighborhood Search)

Cos'è e perché esiste. VNS è una rifinitura sistematica. Cicla attraverso vicinati di dimensione crescente: prima prova 1-swap (scambia due lezioni), poi 2-swap (due paia), poi 3-chain (catena di 3 mosse), poi k-opt (catena di 4-6). Ogni volta che trova un miglioramento riparte dal vicinato 1; quando un intero ciclo passa senza miglioramenti, si ferma – significa che ha raggiunto un ottimo locale rispetto a tutti i vicinati.

Come funziona. For ogni vicinato, esplora fino a un budget di iterazioni; al primo miglioramento accetta e torna al vicinato 1. Quando il giro $1 \rightarrow 4$ produce zero miglioramenti, esce.

Quando usarlo. Come rifinitura finale, dopo LNS/ALNS/SA/TS. Default OFF (lo accendi quando vuoi spremere il massimo).

Esempio. Hai $\text{SOFT}=395$ dopo ILS. VNS cicla: 1-swap trova un miglioramento a 392; riparte. 1-swap trova ancora a 389; riparte. 1-swap zero; 2-swap trova a 385; riparte. Dopo qualche minuto chiude a 378.

D.9 Hall's theorem pre-check

Cos'è e perché esiste. Prima di lanciare un solver lungo, conviene controllare se il problema è *strutturalmente* risolubile. Il teorema di Hall (1935) dice: in un *matching bipartito* (assegnare ognuno di N elementi a uno di M elementi compatibili), una soluzione esiste sse per ogni sottoinsieme S degli N elementi, il loro vicinato (gli M compatibili con almeno uno) ha cardinalità $|N(S)| \geq |S|$.

Nel timetabling l'analogo pesato è: per ogni sottoinsieme di docenti, la loro capacità totale di ore deve coprire la domanda totale di ore delle materie che insegnano. Se una materia richiede 40 ore/settimana ma i docenti qualificati ne hanno solo 30 totali, il modello è già infeasible: non c'è bisogno di lanciare CP-SAT.

Come funziona. Tre controlli in cascata: per ogni materia controlla che esista almeno un docente qualificato; per ogni materia controlla che la capacità totale dei docenti qualificati copra la domanda; campiona N sottoinsiemi random di docenti e verifica che la domanda delle materie *esclusive* a ciascun sottoinsieme sia coperta dalla capacità del sottoinsieme.

Quando usarlo. Sempre prima di Phase A. Default ON nella pipeline integrata; se trova violazioni, interrompe immediatamente con un report.

Esempio. Hai aggiunto in mezzo all'anno una nuova materia “Diritto” al triennio (3 ore/settimana per 9 classi = 27 ore) ma hai un solo docente abilitato a Diritto con massimo 18 ore. Hall te lo dice in 100 millisecondi: “Diritto: domanda 27h > capacità docenti 18h”. Risparmi 10 minuti di solver lungo.

D.10 Lagrangian Relaxation con sub-gradient

Cos'è e perché esiste. Quando un problema ha “vincoli scomodi” (vincoli che spezzano una struttura altrimenti separabile), una tecnica classica è *rilassarli*: li rimuovi dal modello e li reintroduci come penalità moltiplicate per coefficienti chiamati *moltiplicatori di Lagrange* (λ). Iterando, i λ si aggiustano per spingere la soluzione del modello rilassato a rispettare comunque i vincoli originari. È un modo di trasformare un problema “a blocchi accoppiati” in una sequenza di sotto-problemi indipendenti.

Nel timetabling questo è utile dopo la decomposizione spettrale: i cluster sarebbero indipendenti se non fosse per i *docenti-ponte* (docenti che insegnano in più cluster). Lagrangian relaxation rilassa la non-sovrapposizione dei docenti-ponte, e itera per riconciliare le scelte dei cluster.

Come funziona. Il *subgradient method* aggiorna λ a ogni iterazione: $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$, dove g_k è la violazione del vincolo rilassato e $\alpha_k = \alpha_0 / (1 + k)$ è una sequenza di “passi” che diminuisce a passi piccoli, garantendo convergenza al limite (in teoria) ma in pratica producendo soluzioni utili in poche iterazioni.

Quando usarlo. Scuole con cluster ben separati ma con docenti-ponte fastidiosi. Default OFF (avanzato).

Esempio. Cluster A (40 classi), Cluster B (40 classi), 5 docenti-ponte. Senza Lagrangian, A e B vanno risolti insieme: 80 classi. Con Lagrangian, vengono risolti separatamente in ~ 3 iterazioni: ogni cluster diventa 40 classi (molto più veloce), e λ converge a valori che impediscono ai docenti-ponte di sovrapporsi.

D.11 Column Generation (Dantzig-Wolfe)

Cos'è e perché esiste. Quando un problema è enorme, conviene non enumerarlo tutto: ne generi solo le parti che servono, on-demand. Column Generation è questa filosofia applicata alla programmazione lineare: hai un problema “master” che sceglie da un catalogo di colonne (soluzioni candidate); a ogni iterazione un sotto-problema genera nuove colonne “promettenti” (con *reduced cost* negativo) e le aggiunge al catalogo.

Nel timetabling: ogni docente ha un catalogo di possibili “settimane-tipo” (orari completi per quel docente, già feasible localmente). Il master sceglie quale settimana-tipo prendere per ognuno; il sotto-problema (uno per docente) genera nuove settimane-tipo guidate dal duale del master.

Come funziona. Loop: (1) il master sceglie una combinazione corrente di colonne; (2) calcola i prezzi duali sui vincoli di copertura; (3) ogni sotto-problema produce la nuova colonna più attraente data quei prezzi; (4) se ce n'è una con reduced cost negativo, aggiungila e ripeti; altrimenti hai trovato l'ottimo del rilassamento LP.

Quando usarlo. Solo per scuole molto grandi (>200 classi). Default OFF.

Esempio. 250 classi, 180 docenti. Enumerare tutte le settimane-tipo possibili sarebbe astronomicamente troppo; Column Generation parte con 3 settimane-tipo per docente (540 colonne), ne aggiunge ~ 50 per iterazione, e converge in 5-10 iterazioni a un orario ottimale rilassato.

D.12 Monte Carlo Sensitivity

Cos'è e perché esiste. Quando hai una soluzione, ti chiedi: *è già vicina al meglio possibile, o c'è ancora margine?* La sensitivity analysis Monte Carlo risponde generando N varianti random della soluzione (ognuna ottenuta applicando 30 mosse atomiche random feasible) e misurando il loro SOFT score. Se la varianza è piccola, sei vicino a un minimo locale; se è grande, hai ancora margine.

Come funziona. Genera N (default 100) walk random di 30 mosse ciascuno, accettando solo quelle che restano HARD-feasible. Per ognuno calcola SOFT. Riporta media, deviazione standard, percentili, coefficiente di variazione $CV = \sigma/\mu$.

Quando usarlo. Dopo aver lanciato la pipeline, per decidere se vale la pena spendere altro tempo in metaeuristiche. $CV < 0.05$ = stai già sull'ottimo locale; $CV > 0.15$ = c'è potenziale di miglioramento.

Esempio. La tua soluzione corrente ha SOFT=400. Monte Carlo restituisce media=402, std=8, $CV \approx 0.02$. Conclusione: la pipeline ha già trovato un buon minimo locale; nuovi giri non porteranno miglioramenti significativi.

D.13 Modularità, betweenness, densità (analisi del bipartito)

Cos'è e perché esiste. Tre metriche dalla teoria dei grafi che ti dicono quanto la struttura della tua scuola sia “decomponibile”. Sono interessanti perché predicono quanto certe tecniche (decomposizione spettrale, parallelismo) funzioneranno bene.

Cosa significano in pratica.

- **Modularità (Newman-Girvan):** misura quanto una partizione *già calcolata* è “naturale”. Valore in $[-1, 1]$. > 0.4 = cluster molto netti, la decomposizione spettrale è perfetta. < 0.2 = la scuola è troppo intrecciata, la decomposizione produrrà cluster artificiali.
- **Betweenness centrality:** per ogni nodo, conta in quanti percorsi minimi del grafo passa. I nodi ad alta betweenness sono i “ponti”: se hanno SOFT alto, fanno collassare la qualità globale.

- **Densità:** $\frac{2|E|}{|V|(|V|-1)}$, frazione di archi presenti rispetto al massimo. Bassa densità (< 0.1) = scuola sparsa, decomposizione ottima. Alta densità (> 0.5) = quasi cricca, decomposizione inutile.

Quando usarli. Diagnostici da consultare prima di investire in metaeuristiche pesanti: ti dicono che strategia ha senso.

Esempio. Liceo medio: modularità 0.43, betweenness top = “Lingua straniera” (5 prof condivisi su tutti gli indirizzi), densità 0.18. Diagnosi: la decomposizione spettrale produrrà 4 cluster netti (Liceo Classico, Liceo Scientifico, Liceo Linguistico, ITIS); i prof di lingua sono i “ponti” da gestire con cura (Lagrangian relaxation aiuta).

D.14 Distribuzioni e istogrammi

Cosa misurano. 5 viste statistiche dell’orario prodotto: distribuzione del carico orario dei docenti, distribuzione del carico delle classi per giorno, heatmap materia $\times$ slot, occupazione per slot, test di goodness-of-fit (KS contro uniforme, chi-quadro per giornate).

Cosa significano in pratica.

- *Carico docenti:* se la distribuzione è schiacciata su un numero (es. tutti 18) il sistema sta rispettando i tetti di cattedra. Se ha una coda lunga (un prof con 22, un altro con 12), c’è sbilanciamento.
- *Heatmap materia $\times$ slot:* zone scure dove cadono troppe ore di una stessa materia in slot adiacenti.
- *KS-test contro uniforme:* $p > 0.05$ = i carichi sono compatibili con una distribuzione uniforme (giusto); $p < 0.05$ = c’è una distribuzione non casuale (alcuni prof troppo carichi).

Quando usarli. Per dare un giudizio qualitativo sull’orario prodotto, oltre al solo SOFT score numerico.

D.15 Correlazioni e regressioni

Cosa misurano. Tre regressioni statistiche sull’orario corrente:

1. OLS: ore di carico docente $\rightarrow$ numero di buchi.
2. OLS: numero di materie diverse del docente $\rightarrow$ contribuzione SOFT.
3. Logit: saturazione classe $\rightarrow$ probabilità di sesta ora.

Per ognuna ottieni coefficienti, p-value, R^2 .

Cosa significano in pratica. “I docenti con carico > 20 ore hanno in media 2 buchi in più ($p < 0.001$, $R^2 = 0.6$)” = c’è un effetto sistematico documentabile. “La saturazione classe non predice la 6^a ora ($p = 0.4$)” = il sistema sta distribuendo bene le ore e non concentra le classi saturate in 6^a.

Quando usarle. Per riportare al collegio docenti dati *quantitativi* sulla qualità dell’orario, anziché solo affermazioni qualitative.

D.16 DSL generico

Cos'è e perché esiste. I sub-DSL precedenti (query lista, logical DNF, objective Phase A, introspettivo) risolvevano ognuno un caso, ma erano grammatiche separate. Ogni fix andava replicato 4 volte. Il *DSL generico* unifica: una grammatica sola, parser unico, quattro back-end di compilazione.

Come si rapporta al modello dati. Le sorgenti iterabili (lessons, teachers, classes, classrooms, students, groups, ...) sono viste pre-calcolate del modello SQLAlchemy: a ogni valutazione, il sistema “materializza” queste viste in dict Python che il parser interroga.

Come si traduce. Un forall l in lessons where ...: predicate si traduce, per il backend EVAL, in un ciclo Python su tutti gli elementi che soddisfano il filtro, applicando il predicato. Per il backend LOGIC (vincoli logical DNF) si traduce in clausole disgiuntive di letterali. Per il backend OBJECTIVE si traduce in una somma pesata aggiunta alla funzione obiettivo della Phase A. Stessa sintassi, traduzioni diverse.

Quando usarlo. Per qualunque vincolo che non rientri nelle preferenze standard. Vedi capitolo dedicato per la gallery di 30+ esempi.

D.17 Stati 5-colori delle matrici

Cos'è. Le matrici di disponibilità (docente, classe, aula) hanno 5 stati ortogonali per ogni cella, con colori distintivi per leggerli a colpo d'occhio:

- **HARD** (rosso): cella vietata in modo assoluto.
- **ENFORCED** (rosso scuro): cella che *deve* esserci occupata.
- **PREFERRED** (verde): preferenza positiva (bonus).
- **DISLIKED/SOFT** (giallo): preferenza negativa (penalità).
- **ALLOWED** (bianco): neutro.

Scorciatoie tastiera. Selezionata una cella o un range, premere H/E/P/D/A la imposta in HARD/ENFORCED/ PREFERRED/DISLIKED/ALLOWED. N per “neutro” (alias ALLOWED). Click + drag per selezionare un range; doppio click sull'header di colonna o riga per applicare a tutta la colonna/riga.

D.18 Tag aule e tag studenti

Cos'è. Etichette libere che il coordinatore attacca a un'aula o a uno studente. Sono *many-to-many*: un'aula può avere tanti tag, un tag può essere su tante aule.

Perché esistono. I tag rendono i vincoli più robusti al cambiamento dell'anagrafica. Invece di scrivere “solo queste tre aule precise” (lista che andrebbe aggiornata a ogni modifica), scrivi “aule taggate matematica”: se domani aggiungi una nuova aula taggata allo stesso modo, il vincolo la include automaticamente.

Casi d'uso tipici.

- Tag aule: matematica, proiettore, lab_lingue, piano_terra, accessibile_disabili.
- Tag studenti: BES (Bisogni Educativi Speciali), DSA, debito_matematica_4, pcto_Acme, studente_atleta.

I tag NON sostituiscono i gruppi articolati (che sono unità operative di scheduling); sono metadato passivo per filtrare e raggruppare nell'interfaccia.

D.19 Run telemetry e visualizzazione live

Cos'è. Ogni esecuzione del solver produce una *serie temporale* di sample: ad ogni iterazione il sistema annota timestamp, fase corrente, valore obiettivo, mosse accettate/rifutate, vincoli HARD violati. Tutto salvato in DB e accessibile dal detail page del run.

Cosa vedi. Un grafico interattivo (Apache ECharts) con l'evoluzione del SOFT score nel tempo, una linea diversa per ogni stage (LNS in azzurro, SA in verde, TS in giallo, ...), markers verticali sui cambi di stage. Statistiche aggregate per stage in tabella (n. iterazioni, durata, best obj). Esportazione in JSON per analisi esterne.

Live mode. Per i run in corso, la pagina poll-a il backend ogni 2s e aggiorna il grafico man mano che arrivano nuovi sample. La pagina pausa il polling automaticamente quando la tab del browser è in background, per non sprecare banda.

D.20 Lockati: lezioni che non si toccano

Cos'è. Il flag locked sulla tabella Lesson indica che quella lezione *non deve essere spostata* dal solver. Utile per fissare a mano una manciata di lezioni critiche e poi lanciare l'ottimizzazione su tutto il resto.

Come si usa. Dal Monitor (e dalla griglia Schedule con click-destro), il bottone “Blocca” marca la lezione come locked. Il bottone “Sblocca” fa l'inverso. Le lezioni locked sopravvivono ai run successivi: vengono caricate come *vincoli aggiuntivi HARD* dal solver, che dovrà incastrare il resto attorno a esse.

Il filtro “Lockati” nella tab del Monitor mostra solo gli eventi che hanno almeno una lezione lockata, utile per visualizzare velocemente cosa hai già fissato e cosa è ancora libero.

E Reference

- Pisinger, "Where are the hard knapsack problems?", 2005.
- Kohl, Karisch, Patriksson, "Very Large-Scale Neighborhood Search Techniques", 2002.
- Burke, Kingston, Pepper, "A standard data format for timetabling instances", 2008 (Si877050919318800).
- Ahuja, Magnanti, Orlin, "Network Flows", 1993.

Bibliografia

- Aho, A. V., M. S. Lam, R. Sethi e J. D. Ullman (2006). *Compilers: Principles, Techniques, and Tools*. 2^a ed. Addison-Wesley.
- Carter, M. W. e G. Laporte (1998). «Recent developments in practical course timetabling». In: *Practice and Theory of Automated Timetabling II*. Lecture Notes in Computer Science 1408, pp. 3–19.
- Christian, B. e T. Griffiths (2016). *Algorithms to Live By: The Computer Science of Human Decisions*. Henry Holt e Co.
- Chung, F. R. K. (1997). *Spectral Graph Theory*. CBMS Regional Conference Series in Mathematics, n. 92. American Mathematical Society.
- Dantzig, G. B. e P. Wolfe (1960). «Decomposition principle for linear programs». In: *Operations Research* 8.1, pp. 101–111.
- Desrosiers, J. e M. E. Lübbecke (2005). *A Primer in Column Generation*. Springer.
- Fiedler, M. (1973). «Algebraic connectivity of graphs». In: *Czechoslovak Mathematical Journal* 23.2, pp. 298–305.
- Fisher, M. L. (1981). «The Lagrangian relaxation method for solving integer programming problems». In: *Management Science* 27.1, pp. 1–18.
- Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley.
- Freeman, L. C. (1977). «A set of measures of centrality based on betweenness». In: *Sociometry* 40.1, pp. 35–41.
- Garey, M. R. e D. S. Johnson (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Geoffrion, A. M. (1974). «Lagrangian relaxation for integer programming». In: *Mathematical Programming Studies* 2, pp. 82–114.
- Glover, F. (1986). «Future paths for integer programming and links to artificial intelligence». In: *Computers & Operations Research* 13.5, pp. 533–549.
- Glover, F. e G. A. Kochenberger, cur. (2003). *Handbook of Metaheuristics*. Kluwer Academic Publishers.
- Google (2010). *OR-Tools: Open source software suite for optimization*. <https://developers.google.com/optimization/>.
- Hall, P. (1935). «On Representatives of Subsets». In: *Journal of the London Mathematical Society* 10.1, pp. 26–30.
- Held, M. e R. M. Karp (1971). «The traveling-salesman problem and minimum spanning trees: Part II». In: *Mathematical Programming* 1, pp. 6–25.
- Hopcroft, J. E. e R. M. Karp (1973). «An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs». In: *SIAM Journal on Computing* 2.4, pp. 225–231.
- James, G., D. Witten, T. Hastie e R. Tibshirani (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.
- Kirkpatrick, S., C. D. Gelatt e M. P. Vecchi (1983). «Optimization by Simulated Annealing». In: *Science* 220.4598, pp. 671–680.
- Lourenco, H. R., O. C. Martin e T. Stuetzle (2003). «Iterated Local Search». In: *Handbook of Metaheuristics*. Springer, pp. 320–353.
- Lovász, L. e M. D. Plummer (1986). *Matching Theory*. North-Holland.
- Luxburg, U. von (2007). «A tutorial on spectral clustering». In: *Statistics and Computing* 17.4, pp. 395–416.

- Marriott, K. e P. J. Stuckey (1998). *Programming with Constraints: An Introduction*. MIT Press.
- McCollum, B., A. Schaerf, B. Paechter, P. McMullan, R. Lewis, A. J. Parkes, L. Di Gaspero, R. Qu e E. K. Burke (2010). «Setting the research agenda in automated timetabling: the second International Timetabling Competition». In: *INFORMS Journal on Computing*. Vol. 22. 1, pp. 120–130.
- Metropolis, N., A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller e E. Teller (1953). «Equation of state calculations by fast computing machines». In: *The Journal of Chemical Physics* 21.6, pp. 1087–1092.
- Metropolis, N. e S. Ulam (1949). «The Monte Carlo method». In: *Journal of the American Statistical Association* 44.247, pp. 335–341.
- Mladenovic, N. e P. Hansen (1997). «Variable neighborhood search». In: *Computers & Operations Research* 24.11, pp. 1097–1100.
- Moskewicz, M. W., C. F. Madigan, Y. Zhao, L. Zhang e S. Malik (2001). «Chaff: Engineering an Efficient SAT Solver». In: *Proceedings of the 38th Design Automation Conference*, pp. 530–535.
- Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.
- Newman, M. E. J. e M. Girvan (2004). «Finding and evaluating community structure in networks». In: *Physical Review E* 69.2, p. 026113.
- Papadimitriou, C. H. e K. Steiglitz (1998). *Combinatorial Optimization: Algorithms and Complexity*. Dover Publications.
- PATAT (1995–2024). *Practice and Theory of Automated Timetabling: International series of conferences*. Proceedings: <https://patatconference.org/>.
- Robert, C. P. e G. Casella (2004). *Monte Carlo Statistical Methods*. 2^a ed. Springer.
- Ropke, S. e D. Pisinger (2006). «An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows». In: *Transportation Science*. Vol. 40. 4, pp. 455–472.
- Rossi, F., P. Van Beek e T. Walsh (2006). *Handbook of Constraint Programming*. Foundations of Artificial Intelligence. Elsevier.
- Russell, S. J. e P. Norvig (2020). *Artificial Intelligence: A Modern Approach*. 4^a ed. Pearson.
- Ryan, D. M. e B. A. Foster (1981). «An Integer Programming Approach to Scheduling». In: *Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling*, pp. 269–280.
- Schaerf, A. (1999). «A survey of automated timetabling». In: *Artificial Intelligence Review* 13.2, pp. 87–127.
- Shaw, P. (1998). «Using constraint programming and local search methods to solve vehicle routing problems». In: *Principles and Practice of Constraint Programming – CP’98*. Springer, pp. 417–431.
- Shi, J. e J. Malik (2000). «Normalized cuts and image segmentation». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.8, pp. 888–905.
- Van Hentenryck, P. e L. Michel (2005). *Constraint-Based Local Search*. MIT Press.
- Vanderbeck, F. e L. A. Wolsey (2010). «Reformulation and Decomposition of Integer Programs». In: *50 Years of Integer Programming 1958–2008*. A cura di M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi e L. A. Wolsey. Springer, pp. 431–502.
- Wasserman, L. (2004). *All of Statistics: A Concise Course in Statistical Inference*. Springer.
- Wolsey, L. A. (1998). *Integer Programming*. Wiley.

Indice analitico

AST, 85

betweenness, 78

chi-quadro
test, 83

densità (grafo), 78
diagnostica, 81
DSL, 84

ECharts, 83
evaluator (DSL), 85

istogramma, 81

Kolmogorov-Smirnov
test, 81

Lagrangiana, 70

modularità, 78
Monte Carlo, 75

Pearson
correlazione, 82
ponte inter-cluster, 71

regressione
logistica, 82
OLS, 82

sensitivity
analisi, 75
subgradient, 71